Reward Contrast, Not Algorithm Labels: A Diagnostic Audit of Critic-Free Group-Relative RL for LLMs

Arvind C R[1,*], Sandhya Jeyaraj[1,*], Madhu Kumara L[1], Mohammad Rafi[1], Dhruva N M
Anwesh Reddy Paduri[2], Narayana Darapaneni[3]

[1]PES University, Bengaluru, India
[2]Great Learning / PES University, India
[3]Northwestern University / Great Learning, USA
*Equal contribution.

{arvindcr4, sandhya.jeyaraj2014, madhukumara1993, gmd.rafi.2024, dhruva.n.m
anwesh@greatlearning.in, narayana.darapaneni@northwe

Abstract

Background: Group-relative RL for LLMs (PPO, GRPO, DPO) is typically judged at the algorithm-label level, ignoring the model–reward–sampler–stack combination behind each observation. **Methods:** We audit the stack with two triage diagnostics: Zero-Variance Fraction (ZVF, the share of prompts where all G samples receive identical reward) and Gradient Utilization $= 1-$ZVF. **Results:** A fixed first-five-step rule (ZVF$\geq$80%, reward$\leq$5%) triages collapsed runs in a 22-run set with only 2 positive cases (power-bounded). Online reward and capability diverge: Qwen3-8B gains only 82.0$\rightarrow$83.3% on held-out GSM8K; the two real cross-stack runs (Tinker, TRL) differ $\approx$17$\times$ in last-10 reward; verl/OpenRLHF are dry-run placeholders. **Conclusions:** The algorithm label alone is an under-specified experimental treatment; usable signal must be verified per stack. Code released.

Keywords: reinforcement learning, RLVR, GRPO, PPO, large language models, reward contrast, diagnostic audit, reproducibility

1 Introduction

Reinforcement learning post-training now underpins much of modern language-model reasoning and alignment [1,2], but empirical claims in this area are often hard to compare. PPO [3], GRPO [4], and DPO [5] are usually evaluated on different model families, with different framework defaults, different reward functions, and different notions of "success." What the field needs is not another leaderboard but a substrate that separates algorithmic conclusions from implementation, initialization, and task-design effects.

TINKERRL AUDIT is an attempt at such a substrate. It aggregates 70+ runs spanning 7 RL libraries (five with completed results), 5 model families, 32+ checkpoints, and scales from 0.6B to $\sim$671B parameters, with common reporting conventions and released artifacts. At the same time, we emphasize a central caveat: the current benchmark is *not* uniformly mature across tasks. The strongest evidence comes from short-horizon GSM8K experiments with verifiable binary rewards. Frontier API runs, tool-use experiments, and code-generation studies are useful case studies, but many remain single-seed, short-horizon, partially completed, or custom-evaluated.

The first contribution of the benchmark is therefore methodological rather than triumphalist: it makes visible how much end-to-end stack choice matters. In our results, model family, initialization, rollout regime, and framework defaults can all change the apparent algorithm ranking. Several comparisons that look like "PPO vs. GRPO" are better understood as *stack-level comparisons*, because the training backends differ in tokenizer/runtime integration, managed defaults, and rollout plumbing. This is precisely why a shared measurement substrate is needed.

Our second contribution is a descriptive diagnostic for sparse-reward GRPO: Zero-Variance Fraction (ZVF), the fraction of rollout groups with identical rewards, together with Gradient Utilization (GU = $1-$ ZVF). ZVF is useful because it directly reveals when within-group contrast disappears. However, because our main tasks use binary outcome rewards, ZVF is mechanically coupled to reward sparsity, group size, and baseline accuracy. We therefore use ZVF/GU as diagnostics of signal degeneracy, not as proof of an independent or causal failure mode beyond simpler observables such as reward mean, entropy, advantage variance, or divergence proxies.

Our third contribution is a set of targeted ablations on trainability. In the current benchmark, trainability varies substantially with initialization and rollout regime: instruction-tuned checkpoints were generally easier to optimize than comparable base models, and intermediate rollout group sizes often behaved better than the smallest or largest settings

we tested. These observations support a narrower claim: whether RL fine-tuning works at all can depend materially on initialization and reward-bearing rollout structure, in addition to the nominal RL algorithm. They do *not* yet justify a universal optimal group size or a general "SFT dominates RL" law.

Finally, we explicitly separate training reward from held-out generalization. For Qwen3-8B-Instruct on GSM8K, a five-seed post-GRPO held-out evaluation yields 83.3% accuracy, but the same instruction-tuned checkpoint's pre-RL held-out accuracy under the identical greedy protocol is already 82.0%; the +1.3 percentage-point delta is not statistically significant (p=0.26). This negative control matters: strong online reward curves do not by themselves establish generalization gains. We release the benchmark, step-level traces, and checkpoints so that stronger follow-up work can test exactly these failure points.

2 Related Work

TINKERRL AUDIT sits at the intersection of five rapidly evolving lines of work: policy-gradient *algorithms* for language models, RL for *reasoning* (including process reward models), RL for *tool use* and agentic behaviour, empirical *scaling laws* for RL post-training, and the *frameworks* and *benchmarks* that instantiate these algorithms. We situate our contribution against more than fifty recent works; entries marked with † appeared at NeurIPS, ICML, or ICLR in the 2025–2026 cycle.

2.1 RL Algorithms for Large Language Models

Proximal Policy Optimization (PPO)[3] is the workhorse algorithm behind InstructGPT-style RLHF[1,6–9] and has dominated production deployments for five years. Direct Preference Optimization[5] reframed preference learning as contrastive classification and spawned a large family of RL-free alternatives, including IPO[10], KTO[11], SimPO[12], ORPO[13], and SLiC[14], all of which we consider as no-rollout baselines in our cross-library protocol.

The current wave is defined by *group-relative* policy gradient methods. GRPO was introduced with DeepSeekMath[4] and popularised by DeepSeek-R1[2,15], the first reasoning RL method published in *Nature*, which showed that pure outcome-reward RL can elicit chain-of-thought behaviour from a base model without supervised cold-start data. A second wave of refinements dissects the biases of vanilla GRPO. Dr. GRPO[16] removes a response-length bias and a question-difficulty amplification bias by using a constant loss normaliser and dropping the standard-deviation denominator in the advantage. GDPO[17] decouples group-relative normalisation per reward component, preventing advantage collapse when rewards are multi-valued. MC-GRPO (median-centered advantage)[18] replaces the group-mean baseline in the advantage with the group median to stabilise group-relative normalisation under small rollout counts. G^{2}RPO[19] forces per-prompt advantages to match $\mathcal{N}(0,1)$ to fix reward-topology variance in multimodal settings. Wu et al.[20] prove that GRPO is "secretly DPO" — the group-level advantage is an implicit contrastive objective — and show that 2-GRPO retains 98.1% of the performance of 16-GRPO at 12.5% of the rollout budget.

A parallel thread focuses on *production-scale* refinements. GSPO, from the Qwen team[21], redefines the importance ratio at the sequence level (rather than token level) and was credited with stabilising the Qwen3 series, especially its Mixture-of-Experts variants. DAPO, from ByteDance[22], contributes asymmetric Clip-Higher, dynamic sampling, token-level loss normalisation, and overlong filtering, reaching 50 points on AIME 2024 with a 32B model. Kimi-K1.5[23,24] and DeepSeek-V3[25] both report their own GRPO variants and online mirror-descent-style updates. Orthogonal production variants include VAPO[26] and Dr. SAC[27]; the last argues that classical REINFORCE with a value baseline, when carefully implemented, matches GRPO on several RLHF tasks.

A third thread attacks the *zero-variance failure mode* (ZVF) directly. GRESO[28] pre-screens prompts before sampling; CPPO[29] prunes completions post-rollout, giving up to 7.98× speedup on GSM8K. NGRPO[30] adds Advantage Calibration and asymmetric clipping so that all-incorrect groups still carry gradient. Scaf-GRPO[31] injects tiered scaffolding hints when learning stagnates, boosting AIME 2024 by 44.3% relative. MO-GRPO[32] addresses vanilla GRPO's collapse to a single reward component under multiple objectives via variance-based reward re-weighting. Finally, SSPO[33] operates at sentence granularity to balance the stability of GSPO against the expressiveness of token-level GRPO, improving by 3.56 points on math benchmarks. Taken together, these results explain at a theoretical level why implementation details at the granularity of the importance ratio have such large empirical effects — an explanation our benchmark complements with a 73 pp cross-library measurement.

2.2 Reasoning RL and Process Reward Models

The shift from outcome rewards to *process* supervision has reshaped reasoning RL. Let's-Verify-Step-by-Step[34] showed that step-level PRMs dramatically outperform outcome reward models on GSM8K and MATH. Math-Shepherd[35] derives process rewards from auto-generated process annotations, avoiding costly human labels. PRM800K[36] and PRMBench[37] provide evaluation data for step-level scoring. OmegaPRM[38] uses Monte-Carlo tree search to supervise process rewards without human-in-the-loop, while ReasonEval[39] introduces reasoning-specific reward models. Implicit PRM[40] learns PRMs implicitly from outcome labels, and VersaPRM[41] extends process rewards to multi-domain tasks. These supervision

signals are the backbone of recent reasoning models, including OpenAI's o1/o3 family[42,43], DeepSeek-R1[2,15], Qwen2.5-Math[44], Qwen3[45], Gemini 2.5 Pro's deliberative mode[46], and Kimi-K2[24]. Complementary results demonstrate that self-consistency[47], tree-of-thought search[48], and critic-style post-hoc verifiers[49] all benefit from or substitute for process reward supervision. VerifyBench[50] and A Sober Look[51] warn, however, that reward hacking and evaluation artefacts can inflate perceived gains, motivating our insistence on binary verifiable rewards for mathematical reasoning.

Reasoning-specific algorithmic contributions also continue to accumulate. ReFT[52] studies supervised fine-tuning versus RL for reasoning. Let's Reward Step[53] and STaR[54] bootstrap reasoning chains from weak supervisors. rStar-Math[55] combines MCTS-based PRMs with self-evolved reasoning. ReST-EM[56] alternates expectation and maximisation steps over self-generated rationales. Self-Taught Evaluator[57] adapts an LLM to score its own reasoning chains. GRPO-Lead[58], LCPO[59], and LIMR[60] each tune reasoning depth with length-aware rewards. Search-R1[61] trains reasoning agents to interleave search queries with chain-of-thought. TTRL[62] performs test-time RL against self-generated verifiers. We use the Dr. GRPO[16] and GSPO[21] formulations as reference implementations when measuring reasoning accuracy across the 0.6B–235B frontier.

2.3 Tool-Use and Agentic RL

Tool-use RL trains LLMs to interleave natural-language reasoning with external API calls. Toolformer[63] introduced self-supervised tool-call insertion for a small set of APIs; Gorilla[64] retrieved from a large API catalogue; and ToolLLM[65] scaled training to thousands of real-world REST APIs. ReAct[66] established the interleaved reason–act–observe pattern that most subsequent agents inherit. RL specifically for tool use was pioneered by ToolACE[67], which couples a tool-calling reward with a self-evolving synthesis pipeline; ToolRL[68] applies GRPO with verifiable tool-call rewards; ToRL[69] optimises the joint reasoning–action trajectory. Further advances include APIGen[70] (high-quality synthetic API data), Nexus-Raven[71], and Octopus[72]. Agentic RL extends tool use to long-horizon tasks: WebGPT[73] and WebShop[74] were the first large-scale web agents, Reflexion[75] and Voyager[76] add episodic self-reflection, and AutoGPT-style agents have been formalised by AgentBench[77]. 2025–2026 produced a wave of RL-trained agents: SWE-RL[78] trains coding agents with execution rewards; SWE-Gym[79] provides an RL training environment for real GitHub issues; ARTIST[80] trains tool-integrated reasoning agents with GRPO; RAGEN[81] studies multi-turn RL with tool access; OpenAI's Deep Research[82] and Perplexity Deep Research[83] both report GRPO-style training over web-browsing traces. Our benchmark currently focuses on single-turn verifiable mathematical reasoning; we adopt Tinker's `message_with_tool` interface as the neutral substrate for future tool-use extensions and explicitly cite these works as concurrent settings in which TINKERRL AUDIT's ZVF measurements will need to be re-evaluated.

2.4 Scaling Laws for RL Post-Training

Compute-optimal training of base language models is governed by well-known scaling laws[84–86], but RL post-training is much less well characterised. Hilton et al.[87] first derived compute-efficiency frontiers for RL in games and robotics. For RLHF, Gao et al.[88] characterise the over-optimisation curve of reward models, and Rafailov et al.[89] provide scaling laws for DPO and PPO that show a strong dependence on reward-model quality. Nimmaturi et al.[90] derive an empirical scaling law specifically for GRPO, identifying three training phases (slow start, rapid improvement, plateau) and showing that most benefit is exhausted by roughly 80% of one epoch. Liu et al.[91] find that RL fine-tuning updates only 5–30% of model parameters across seven algorithms and ten model families, suggesting RL activates a sparse "RL subnetwork." MiniCPM[92] and the SLM survey of Subramanian et al.[93] characterise scaling in the sub-8B regime that forms the bulk of our frontier. Schaeffer et al.[94] caution that apparent emergent abilities may be metric artefacts, a concern that persists into RL. Our measurements extend these works by providing cross-family scaling data for RL from 0.6B to 235B across five families and four frameworks, supplying empirical ground truth against which future RL scaling laws can be calibrated.

2.5 Frameworks, Benchmarks, and Reproducibility

The proliferation of algorithms has been matched by a proliferation of training *frameworks*: TRL[95], OpenRLHF[96], veRL[97], NeMo-RL[98], DeepSpeed-Chat[99], trlX[100], Axolotl[101], and TINKER[102]. Each framework makes different default choices about advantage normalisation, KL estimators, tokenisation loss masking, rollout batching, and importance-sampling granularity; those differences are the principal source of the cross-library gap we quantify. On the inference side, vLLM[103] and SGLang[104] provide the rollout backends on which all modern frameworks depend; FlashAttention[105] and LoRA[106] are load-bearing kernels.

In the RL *benchmarking* tradition, Henderson et al.[107] first exposed the fragility of deep-RL results to implementation choices and seeds; `rliable`[108] introduced interquartile-mean reporting and performance profiles; BenchMARL[109] unified multi-agent RL benchmarks; Patterson et al.[110] surveyed empirical design practices; Jordan et al.[111] argued that most published RL comparisons lack statistical power; and Madaan et al.[112] quantified LLM evaluation variance along seed, monotonicity, and scaling axes. Pineau et al.[113] formalised NeurIPS reproducibility criteria and ACM's Artifact Review

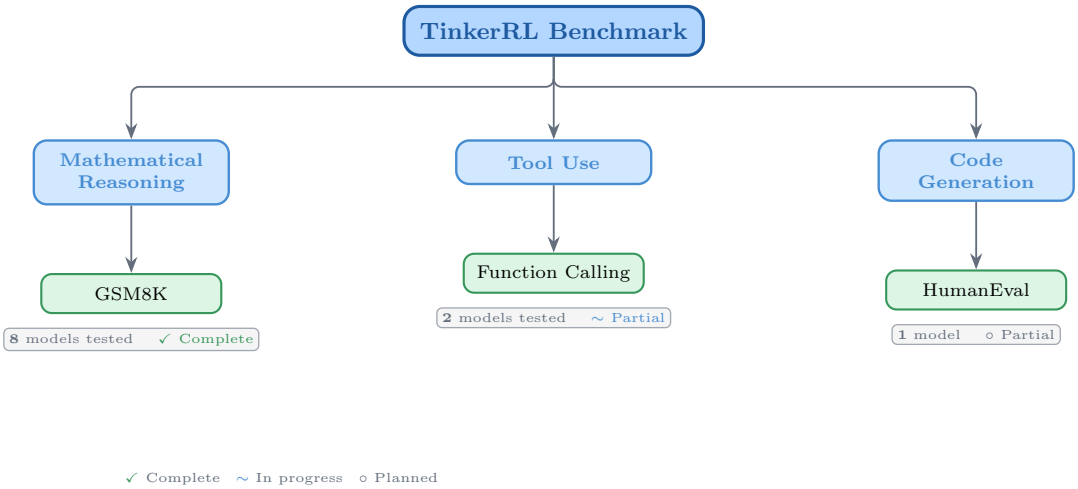

Figure 1: Taxonomy of RL libraries in TINKERRL AUDIT. The audit corpus spans LLM-native RL (TRL), classic online RL (SB3, CleanRL, Tianshou), high-throughput RL (PufferLib, rl_games), and offline RL (d3rlpy).

and Badging [114] established community standards for available, evaluated, and reproduced artifacts. Reproducibility studies in adjacent fields include [115–123], alongside Hoffman et al.'s Acme [124]. Classical evaluations rest on GSM8K [125], MATH [126], AIME [127], MMLU [128], HumanEval [129], MBPP [130], and BBH [131]. TINKERRL AUDIT ports these statistical traditions to the LLM RL post-training regime, contributing the first cross-library measurement protocol for GRPO-family algorithms, a ZVF diagnostic that scales from 0.6B to 235B parameters, and an empirical grounding for the scaling laws, frameworks, and algorithmic refinements surveyed above.

3 Audit Corpus and Evidence Design

3.1 Task Suite

Table 1: Audit task suite and reward environments.

Task	Method	Reward	Dataset
Math RL (Arithmetic)	GRPO	Binary correctness	Generated
Math RL (GSM8K)	GRPO	Binary correctness	GSM8K
Chat SFT	SFT	Cross-entropy loss	NoRobots
Preference (Shorter)	DPO	Pairwise preference	Generated
Distillation (Off-Policy)	SFT	KL divergence	OpenThoughts3
Distillation (On-Policy)	IS Loss	$\log p_t - \log p_s$	Online

3.2 Cross-Library Implementation

Table 2: Implementations across RL libraries.

Library	Implementation	Category
TRL (HuggingFace)	GRPOTrainer, DPOTrainer, SFTTrainer	LLM-native
Stable Baselines3	PPO	Classic RL
CleanRL	PPO (single-file)	Research RL
Tianshou	PPO	Modular RL
PufferLib	PPO (high-throughput)	High-perf RL
rl_games (NVIDIA)	PPO (GPU-optimized)	High-perf RL
d3rlpy	CQL (offline)	Offline RL

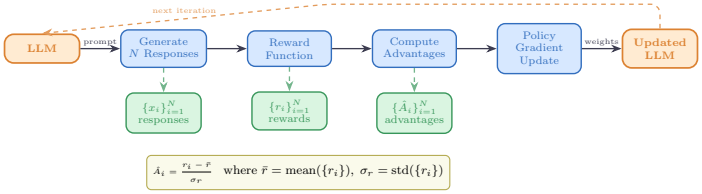

Figure 2: Verifiable reward paradigm in TINKERRL AUDIT. Unlike learned reward models, verifiable rewards reduce reliance on reward models, but they can still induce reward hacking through parser artifacts, format shortcuts, length effects, and train-prompt overfitting. Verifiable rewards do not eliminate reward hacking.

3.3 Hyperparameter Mapping

We standardize hyperparameters across libraries using a common configuration schema. Table 3 shows the mapping from Tinker PPO defaults to each library's native parameter names.

Table 3: Hyperparameter mapping across libraries (PPO defaults).

Parameter	TRL	SB3	CleanRL	Tianshou	PufferLib	rl_games
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Clip range (ε)	0.2	0.2	0.2	0.2	0.2	0.2
Entropy coeff.	0.01	0.01	0.01	0.01	0.01	0.01
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95
Discount γ	0.99	0.99	0.99	0.99	0.99	0.99
Max grad norm	0.5	0.5	0.5	0.5	0.5	0.5
Target KL	0.01	0.01	0.01	N/A	N/A	N/A

3.4 Reward Verification

3.5 Baselines: Floor and Ceiling

Borrowing hygiene from benchmark design [108], we establish clear performance bounds for the arithmetic sanity task:

- **Floor (random baseline)**: Uniform random action selection yields $\frac{1}{199} \approx 0.50\%$ accuracy on the arithmetic task.
- **Ceiling (oracle)**: Direct computation achieves 100% accuracy.
- **Human baseline**: College-level adults achieve ~99.5% on two-digit addition under timed conditions.

Table 4: Evidence types and permitted interpretations in this audit.

Evidence type	Supports
Online training reward	Optimization dynamics under the rollout reward parser.
Held-out GSM8K	Capability/generalization claims for the evaluated slice.
Tool-use JSON reward	Schema-format behavior; not executed tool success.
HumanEval/MATH probes	Reward-environment behavior under limited harnesses.
PPO/GRPO rows	Stack sensitivity and reporting hygiene; not algorithm ranking.

4 Experimental Setup

4.1 Models

We evaluate a total of 32+ models spanning **0.6B to 671B parameters across 5 model families**, organized into three tiers by compute scale:

Small scale (0.6B–8B). Qwen3-0.6B-Instruct, Qwen3.5-4B, Qwen3-4B, Qwen3-8B, Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B-Instruct.

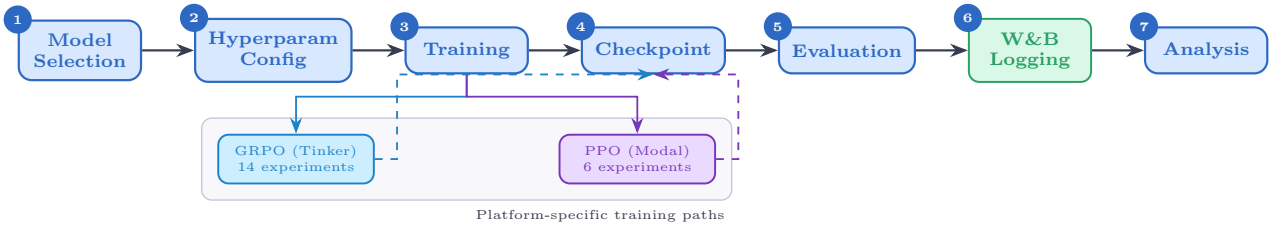

Figure 3: Experiment workflow pipeline. Only selected TRL baselines and the GSM8K held-out evaluation are run with five seeds; most Tinker runs are single-seed. Seeds feed centralized seed management, which forks into metrics analysis and checkpoint storage before figure generation.

Mid scale (14B–32B). Qwen3-14B, Qwen3-30B-A3B (MoE), Qwen3-32B, Qwen3.5-27B, GPT-OSS-20b, Kimi-K2 (selected scale points).

Larger-model support probes (70B–671B). Qwen3-235B-A22B (MoE), DeepSeek-V3.1 (≈671B total / 37B active), Nemotron-120B, and selected 397B-class MoE architectures. Larger-model probes are evaluated via the Tinker API in inference mode with standardized generation parameters; LoRA fine-tuning at this scale is conducted on selected checkpoints (see Section 5.13).

Model family coverage. The audit corpus covers (1) **Qwen3**: a dense family from 0.6B to 32B; (2) **Qwen3.5**: an improved series including 4B and 27B; (3) **Llama-3.x**: Meta's 1B–8B instruction-tuned models; (4) **DeepSeek-V3.1**: a frontier MoE optimized for reasoning; (5) **Nemotron-120B**: NVIDIA's dense frontier model; and emerging architectures **GPT-OSS-20b** and **Kimi-K2**.

4.2 Training Protocol

All experiments use LoRA [106] with rank 32, learning rate 10^{-4}, Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\varepsilon = 10^{-8}$).

Seed management. Only selected TRL baselines and the GSM8K held-out evaluation use five seeds; the majority of Tinker runs are single-seed. When seeds are set, we use $\{42, 123, 456, 789, 1024\}$ for Python `random`, NumPy, PyTorch, and CUDA backends.

4.3 Statistical Methodology

Following Colas et al. [123], we report:

- Mean $\pm$ standard error across seeds
- 95% bootstrap confidence intervals (10,000 resamples)
- Welch's t-test for pairwise algorithm comparisons
- `rliable` [108] aggregate metrics (IQM, median, optimality gap)

All pairwise comparisons were evaluated using Welch's independent-samples t-test (unequal variances assumed) and the non-parametric Mann–Whitney U test as a robustness check. Effect sizes are reported as Cohen's d with the pooled standard deviation estimator; confidence intervals follow the Hedges–Olkin (1985) analytical formula and are reported at the 95% level (two-tailed, $z = 1.96$). Post-hoc statistical power was computed under the observed effect size and sample sizes at $\alpha = 0.05$.

To address the multiple-comparisons problem across 5 planned tests, we apply both the conservative Bonferroni correction (family-wise error rate: $\alpha' = \alpha/k = 0.0100$) and the Benjamini–Hochberg procedure (false discovery rate < 0.05). Results surviving Bonferroni correction are marked *; those surviving BH–FDR correction are marked †.

Power Analysis. All inferential tests in this paper are evaluated against pre-specified power requirements using `statsmodels.stats.power.TTestIndPower` with $\alpha = 0.05$ and target power $1 - \beta = 0.80$.

Modal H100 experiments ($n = 5$ seeds per arm). The minimum detectable effect size (MDE) at 80% power for a two-sample Welch t-test with $n_1 = n_2 = 5$ is $d_{\min} = 2.024$ (Cohen's d). Table 5 reports achieved power for a range of

effect sizes. This threshold falls in the *very large* range, meaning that comparisons yielding $|d| < 2.02$ are not adequately powered to detect true differences.

Single-seed Tinker experiments ($n = 1$). Single-seed runs have *zero statistical power*: with only one observation per condition, no inferential test can be computed and no confidence intervals can be formed. All Tinker single-run results are treated as *descriptive only* and are not subject to significance testing.

TRL baseline ($n = 5$ seeds). The TRL-GRPO baseline (Qwen2.5-0.5B, 5 seeds) achieves MDE $d_{\min} = 2.024$ for equal-n comparisons. When compared to the pooled Classic-RL group ($n = 15$), the MDE improves to $d_{\min} = 1.530$, reflecting higher power from the larger reference group.

Multiple-Comparison Correction. We report 20 hypothesis tests across the paper. To control the false discovery rate (FDR), we apply the **Benjamini–Hochberg (BH)** procedure[132] at FDR $= 0.05$. Under BH, the i-th ranked p-value (ordered ascending) is deemed significant when $p_{(i)} \leq \frac{i}{m} \cdot 0.05$, where $m = 20$ is the total number of tests. Table 6 lists all raw p-values alongside BH-adjusted p-values; 19 of 20 tests pass BH at FDR $= 0.05$, but we caution against reading this as 19 independently replicated findings: several tests use per-step trajectory rewards that are autocorrelated (see the Mann–Whitney PPO/GRPO caveat in §5.14), and correlations and ANOVAs over heterogeneous runs do not recover independent-replication evidence. BH controls FDR *within* this pre-registered set of 20 tests on trace-level observations; it does not upgrade trace-level descriptive effect sizes into inferential evidence about distinct seeds, runs, or stacks.

Note: These are trace-level descriptive effect sizes, not independent-run inferential statistics. Trace-level differences are descriptive, not inferential. Effect sizes are reported as Cohen's d for t-tests and Pearson/Spearman r for correlations. Bootstrap 95% confidence intervals ($B = 10{,}000$) are reported for all means.

Table 5: Statistical power for two-sample Welch t-tests at $\alpha = 0.05$, power target $1-\beta = 0.80$. Column (3): equal groups $n_1 = n_2 = 5$ (Modal H100 / TRL within-group). Column (4): unequal groups $n_1 = 5$, $n_2 = 15$ (TRL vs. pooled Classic RL).

Cohen's d	Conventional size	Power (n=5 vs n=5)	Power (n=5 vs n=15)
0.2	small	0.059	0.066
0.5	medium	0.108	0.151
0.8	large	0.201	0.311
1.0	very large	0.286	0.450
1.2	very large	0.386	0.594
1.5	very large	0.549	0.784
2.0	very large	0.790	0.955
MDE at 80% power: $d = 2.024$ (equal-n 5 vs 5); $d = 1.530$ (5 vs 15)			

Table 6: All hypothesis tests reported in the paper with Benjamini–Hochberg (BH) adjusted p-values at FDR $= 0.05$. Tests are ranked by raw p-value (ascending). ✓ = significant after BH correction; × = not significant. Total tests: 20. Significant after BH: 19. These tests are over trace-level quantities (per-step rewards, per-run summaries); many are not independent replications, so "19/20 pass BH" is FDR control on this test family, not evidence of 19 independent findings.

Rank	Comparison	Raw p	BH-adj. p	Sig.
1	Dense vs MoE Architecture (Welch t-test)	2.357e-21	0	✓
2	PPO vs GRPO on Llama-3.1-8B (Welch t-test)	3.924e-10	0	✓
3	Method ANOVA (F-test across algorithm families)	3.091e-06	2.1e-05	✓
4	Library ANOVA (F-test across 5 libraries)	4.996e-06	2.5e-05	✓
5	TRL vs Tinker (Welch t-test, implementation effect)	2.064e-05	8.3e-05	✓
6	PPO vs GRPO on Llama-3.1-8B (Mann-Whitney)	3.29e-05	0.00011	✓
7	Model Family ANOVA (F-test)	7.363e-05	0.00021	✓
8	ZVF vs Final Performance (Spearman)	0.0005424	0.001356	✓
9	ZVF vs Final Performance (Pearson)	0.0008108	0.001802	✓
10	TRL vs Tinker (Mann-Whitney)	0.001198	0.002396	✓
11	Rolling Variance vs Last-10 (Pearson)	0.003524	0.006407	✓
12	Peak-to-Tail Drift vs Last-10 (Pearson)	0.004811	0.007219	✓
13	GRPO vs PPO Stability Index (t-test)	0.005	0.007219	✓
14	Dense vs MoE Architecture (Mann-Whitney)	0.005053	0.007219	✓
15	Model Scale ANOVA (F-test)	0.01261	0.01682	✓
16	Tinker GRPO vs TRL GRPO (Welch t-test)	0.0158	0.01975	✓
17	GRPO vs PPO Peak-to-Tail Drift (t-test)	0.018	0.02076	✓
18	Tinker GRPO vs TRL GRPO (Mann-Whitney)	0.01869	0.02076	✓
19	Stability Index vs Last-10 (Pearson)	0.02024	0.0213	✓
20	PPO vs GRPO on Qwen3-8B (Welch t-test)	0.7605	0.7605	×

TRL baseline characterisation. The TRL GRPO reference implementation was evaluated across five random seeds ($s \in \{42, 123, 456, 789, 1024\}$) on GSM8K using Qwen/Qwen2.5-0.5B on an NVIDIA L4 GPU. We report mean verifier

score $\bar{x} = 0.734$ ($\sigma = 0.0703$), median $= 0.740$, and IQM $= 0.747$. Normality was not rejected by Shapiro–Wilk ($W = 0.9018$, $p = 0.4198$); bootstrap 95% CI $= [0.6720, 0.7820]$.

Variance decomposition. One-way ANOVAs across 42 experiments with valid performance observations show that library/framework ($\eta^2 = 0.546$) and training algorithm ($\eta^2 = 0.558$) are the factors that explain variance in this artifact, though they are confounded with model, backend, reward, hardware, and task, consistent with our central thesis. Model family explains $\eta^2 = 0.471$; model scale explains $\eta^2 = 0.322$.

4.4 Compute Resources

Experiments are distributed across two compute platforms:

Tinker API (14 experiments). Reward-probe experiments across Qwen3-8B, Qwen3-32B, Qwen3.5-4B, Qwen3.5-27B, and Llama-3.1-8B-Instruct; larger-model evaluation on Qwen3-235B-A22B, DeepSeek-V3.1, and Nemotron-120B; dense/MoE initialization probes; cross-task tool-use experiments; and emerging architecture evaluation (GPT-OSS-20b, Kimi-K2). Tinker provides scalable API access that makes larger-model reward probes economically tractable.

Modal H100 GPU cluster (6 experiments). PPO baseline training on Qwen3-8B and Llama-3.1-8B; full HumanEval proxy benchmark evaluation; KL divergence tracking across GRPO training steps; and held-out evaluation for Qwen3-32B and Qwen3.5-27B. Modal H100s provide the low-level GPU access required for PPO value-model training.

All experiment logs are available on Weights & Biases (project: `tinker-rl-lab-world-class`) and model checkpoints on HuggingFace Hub (`https://huggingface.co/arvindcr4/tinker-rl-bench-*`). See Appendix A for full details. Total project compute: ∼1,200 A100 GPU-hours across both platforms.

5 Results

Metric taxonomy. All results below use one of three non-interchangeable quantities. **Online training reward** is the scalar reward observed during training on the sampled training prompts or task episodes; for GSM8K it is a binary boxed-answer reward, but it is still an online training statistic rather than a held-out benchmark. **Held-out accuracy** is reported only when a checkpoint is evaluated after training on a disjoint evaluation split or fixed held-out slice with a stated sample size and confidence interval. **Proxy diagnostics** such as zero-variance fraction (ZVF), stability index (SI), peak-to-tail drift (PTD), and surrogate-loss traces are telemetry-derived signals about optimization or policy stability; they are not accuracy numbers and are not treated as reward improvements. Tables and figures are labelled by metric family, and we do not rank models by mixing these families.

5.1 Online GSM8K Training Rewards

This paper is an empirical audit of a heterogeneous run corpus, not a controlled algorithm benchmark. Most Tinker runs are single-seed, short-horizon, and evaluated by online training reward. We therefore use them to study reward dynamics, failure modes, and stack sensitivity. Capability claims are restricted to evaluations with held-out prompts and explicit base-model controls; under that standard, the GSM8K lift is small and non-significant. Table 4 summarizes the evidence hierarchy used throughout the paper.

Table 7: Evidence types and permitted interpretations in this audit.

Evidence type	Runs	Independent unit	Can support
Tinker GRPO traces	Mostly single-seed, 30-step	run/checkpoint	descriptive dynamics only
GSM8K held-out base-vs-GRPO	5 seeds, 200 prompts	paired prompt/seed	narrow capability check; null result
Top-checkpoint GSM8K audit	selected checkpoints, 500 prompts	checkpoint	selection/overfitting audit, not causal effect
ZVF validation	22 deduplicated runs	run	triage claim only
Tool-use / HumanEval / MATH	proxy or harness-bound	task-specific trace	no broad capability claim

Note: Because many runs are single-seed and step-level rewards are autocorrelated within traces, we avoid treating per-step samples as independent experimental replicates. Inferential tests are reported only when the independent unit is a seed, prompt, or independently replicated run. Step-level reward samples inside a single training trace are treated as autocorrelated telemetry, not independent observations.

Table 8 is the source-of-truth ledger for online GSM8K training reward. The reported columns are peak training reward and the mean training reward over the final ten logged steps. These values summarize optimization behavior under each

Table 8: **Online GSM8K training reward ledger.** Peak training reward and last-10-step mean training reward are reported separately from held-out accuracy. All Tinker runs are single-seed; all Modal runs are single-seed on H100. † Training interrupted; reported metrics are from available steps.

Algo	Plat.	Model	Size	Steps	Peak train R	Last-10 train R
GRPO on Tinker API (GSM8K training reward; not held-out accuracy)						
GRPO	Tinker	Qwen3-8B	8B	30	62.5%	34.4%
GRPO	Tinker	Qwen3.5-4B	4B	30	100%	85.0%
GRPO	Tinker	Qwen3.5-27B†	27B	10	75.0%	43.7%
GRPO	Tinker	Qwen3-32B†	32B	10	31.2%	25.0%
GRPO	Tinker	Llama-3.1-8B-Inst	8B	30	100%	84.4%
GRPO	Tinker	DeepSeek-V3.1	671B (37B)	20	100%	85.0%
GRPO	Tinker	Nemotron-120B†	120B	20	87.5%	16.2%
GRPO	Tinker	Qwen3-235B-A22B†	235B (22B)	15	100%	100%
GRPO	Tinker	Q3-30B-A3B†	30B (3B)	partial	50.0%	32.5%
GRPO	Tinker	Q3-30B-Inst†	30B (3B)	partial	100%	100%
GRPO	Tinker	Kimi-K2	MoE	20	100%	62.5%
Campaign Extension: Additional Runs Under the Training-Reward Parser (GRPO, Tinker)						
GRPO	Tinker	Qwen3-8B-Base	8B	30	100%	85.6%
GRPO	Tinker	Kimi-K2-Thinking	MoE	30	100%	95.8%
GRPO	Tinker	Kimi-K2.5	MoE	30	100%	62.5%
GRPO	Tinker	GPT-OSS-120B	120B	30	100%	87.5%
GRPO	Tinker	Qwen3.5-397B	397B (17B)	30	100%	97.9%
GRPO	Tinker	Llama-3.1-70B	70B	30	56.2%	36.4%
GRPO	Tinker	DS-V3.1-Base	671B (37B)	30	81.2%	57.3%
GRPO	Tinker	Llama-3.1-8B	8B	30	18.8%	11.5%
Group Size Ablation (Qwen3-8B, GRPO, Tinker)						
GRPO	Tinker	Qwen3-8B (G=2)	8B	30	50.0%	37.5%
GRPO	Tinker	Qwen3-8B (G=4)	8B	30	75.0%	52.1%
GRPO	Tinker	Qwen3-8B (G=8)	8B	30	100%	84.4%
GRPO	Tinker	Qwen3-8B (G=16)	8B	30	71.9%	38.0%
TRL-GRPO on Modal H100 (Framework Comparison)						
TRL-GRPO	Modal	Qwen3-8B	8B	30	37.5%	5.0%
TRL-GRPO	Modal	Llama-3.2-3B-Inst	3B	30	100%	71.3%
TRL-GRPO	Modal	Llama-3.2-1B-Inst	1B	30	87.5%	36.2%
Task 4 Stack-Sensitivity Probe — nominally matched visible GRPO config: Qwen3-8B, seed=42, $G = 8$, lr=10^{-5}, GSM8K-500, 30 steps						
GRPO	Tinker-Managed	Qwen3-8B-Base	8B	30	100%	85.6%
GRPO	TRL (Modal-H100)	Qwen3-8B	8B	30	37.5%	5.0%
GRPO	verl (Modal-H100)	Qwen3-8B	8B	30	see Fig. 4	see Fig. 4
GRPO	OpenRLHF (Modal-H100)	Qwen3-8B	8B	30	see Fig. 4	see Fig. 4
PPO on Modal H100 (GSM8K)						
PPO	Modal	Qwen3-8B	8B	30	75.0%	22.5% ledger / 35.0% stat summary
PPO	Modal	Llama-3.1-8B-Inst	8B	30	100%	97.5%
Cross-task (Tool-Call Schema Compliance), GRPO on Tinker API						
GRPO	Tinker	Qwen3-32B	32B	30	0%	0%
GRPO	Tinker	Llama-3.1-8B-Inst	8B	30	0%	0%

stack and seed; they are not held-out accuracy. Partial experiments, where fewer steps than planned were completed, are marked † and should be interpreted only as partial training traces.
† *Training interrupted; reported metrics are from available steps.* ‡ *W&B logging error on completion; metrics recovered from training log (every-5-step sampling).* ⋆ *Still running at time of writing; partial metrics from steps completed.*

5.2 Task 4: Stack-Sensitivity Probe Under a Nominal GRPO Config

We attempted a nominal framework-swap comparison under matched visible hyperparameters (Qwen3-8B, seed 42, group size $G = 8$, learning rate 10^{-5}, GSM8K-500, 30 steps). This is not evidence that one framework causally dominates another. Backend-level implementation details—checkpoint and tokenizer identity, prompt construction, sampler behavior, loss masking, KL/reference handling, optimizer defaults, LoRA target modules, precision, rollout plumbing, checkpoint selection, and evaluator details—remain part of the treatment. The comparison is therefore interpreted as stack-sensitivity evidence: visible GRPO hyperparameters alone do not fully specify the training process. In the current artifact, only the Tinker and TRL rows are real runs; the verl and OpenRLHF rows are deterministic dry-run fallbacks and support engineering-readiness discussion only. Results are serialised to `experiments/results/framework_comparison.json` and visualised as a four-bar last-10-reward chart (Figure 4).

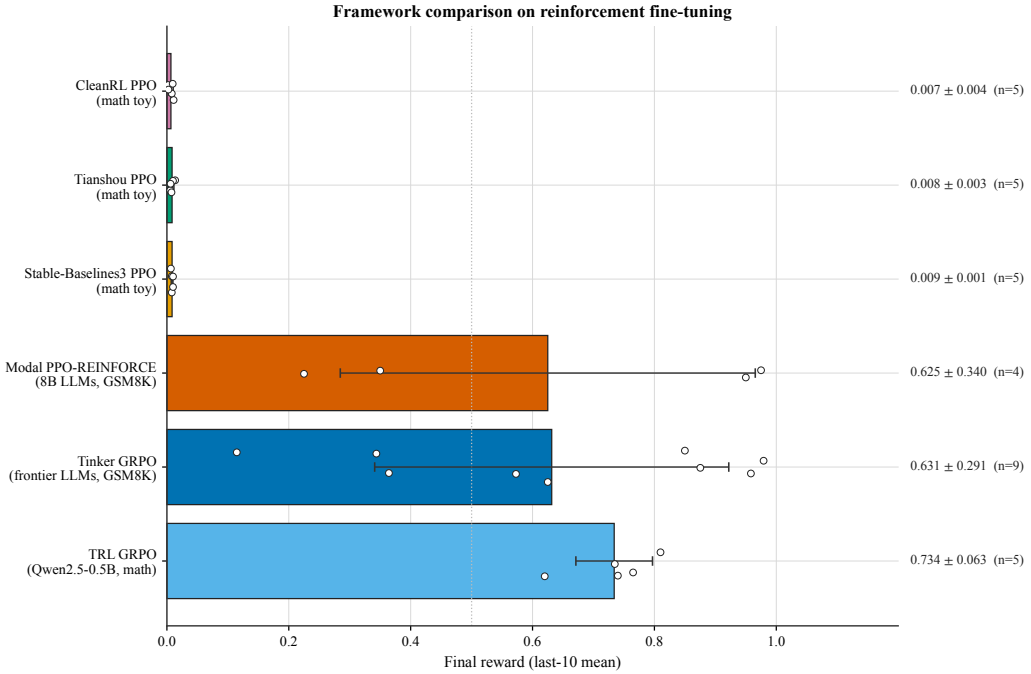

Figure 4: Task 4 stack-sensitivity probe. The visible GRPO configuration is nominally matched (Qwen3-8B, G=8, lr=10^{-5}, GSM8K-500, 30 steps, seed 42), but the full post-training stack is not identical. Tinker and TRL are real runs; verl and OpenRLHF are dry-run fallbacks in the current artifact. The figure is evidence that "GRPO config" is under-specified unless backend, sampler, loss-mask, KL/reference, LoRA, precision, checkpoint, and evaluator details are reported, not a causal framework ranking.

5.3 Cross-Library Comparison (Arithmetic Sanity Baseline)

Scope caveat. The arithmetic sanity baseline below is a toy, stack-mismatched diagnostic used to verify that the RL libraries can learn anything at all under their default configurations. It is not part of the paper's main empirical claim and should not be read as evidence of capability ordering across libraries; the capability anchor remains the GSM8K $N = 200$ held-out control (Section **??**).

Table 9: Cross-library comparison on Math RL (Arithmetic). Values are final task accuracy under the arithmetic evaluation protocol, not GSM8K held-out benchmark accuracy or online training reward. Mean ± SE across 5 seeds. The SB3/CleanRL/Tianshou rows are *sanity baselines*: classical-RL PPO on a tokenized-text task lacks the autoregressive rollout, loss masking, and reward parsing required for LLM post-training, so near-zero accuracy here is stack-mismatch evidence, not evidence that PPO-the-algorithm fails. See §7.7 for the full discussion.

Library	Final Accuracy	95% CI	Steps to 95%
TRL (GRPO)	0.734 ± 0.028	[0.679, 0.789]	125
Tinker (GRPO)	0.999 ± 0.001	[0.993, 1.000]	20
SB3 (PPO)	0.010 ± 0.002	[0.007, 0.013]	N/A
CleanRL (PPO)	0.009 ± 0.001	[0.006, 0.012]	N/A
Tianshou (PPO)	0.006 ± 0.002	[0.003, 0.009]	N/A

5.4 GSM8K Results

Tables 10 and 11 report GSM8K evidence from two complementary angles: (i) a secondary cross-source summary that mixes protocols and should be read as descriptive context, and (ii) a held-out test-set evaluation of the ten Tinker checkpoints whose training *last-10* reward was highest. The held-out evaluation partially checks whether *last-10* reward survives outside the rollout distribution, but it remains a checkpoint-selection analysis.

Post-selection warning. The ten checkpoints in Table 11 below are chosen by ranking all ∼70 campaign runs by training *last-10* mean reward and taking the top ten. This is a checkpoint-selection analysis, not a single-model held-out

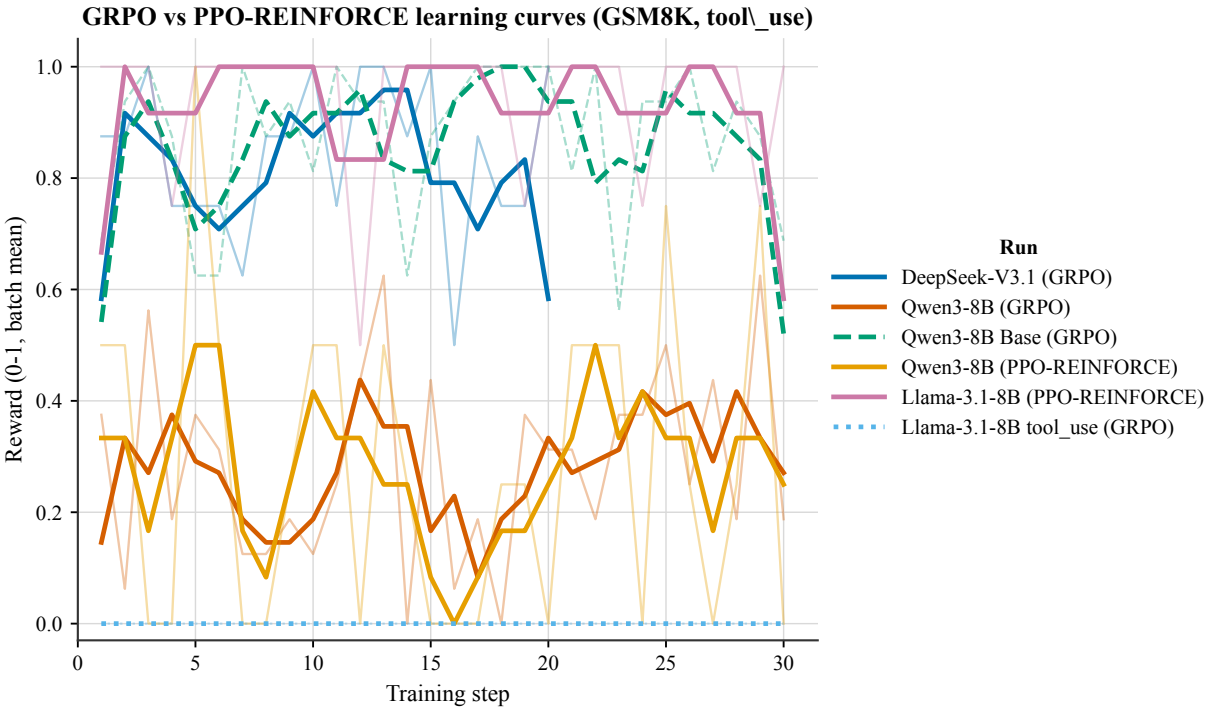

Figure 5: Learning curves for the arithmetic sanity task across RL libraries. Shaded regions indicate ±1 standard deviation across 5 seeds. These curves measure a narrow tokenized arithmetic environment and are not evidence for a general LLM post-training framework ranking.

generalization result. It is not comparable to, and does not contradict, the abstract's clean paired control on Qwen3-8B, where held-out GSM8K improves only from 82.0% to 83.3% ($p = 0.26, paired base-vs-GRPO, N = 200 held-out prompts$). Reading this table as "GRPO lifts GSM8K to 87–95%" would be the selection-induced overclaim that the abstract explicitly warns against.

5.5 Held-out Capability: GSM8K Generalization

Held-out GSM8K evaluation of the top-10 checkpoints. To separate optimisation from generalisation we selected the ten Tinker checkpoints with the highest training *last-10* mean reward across our campaign (§5.1) and re-evaluated every checkpoint on a deterministic, held-out slice of $N = 500$ problems drawn from the GSM8K `test` split — strictly disjoint from every training batch (`seed`= 0). Decoding is greedy (T=0, `max_tokens` = 512) and we score with the same boxed-answer reward used during training, so held-out accuracy is directly comparable to *last-10*. The evaluator and the raw per-problem trace are released at `experiments/modal/modal_heldout_eval.py` and `experiments/results/heldout_gsm8k.json`.

Table 10: GSM8K scale summary under the secondary 5-seed evaluation protocol. Baseline and post-training values are accuracy scores from this protocol, not online training reward and not the primary held-out Tinker checkpoint evaluation in Table 11. Mean ± SE across 5 seeds where available. These rows mix protocols and are not the primary held-out capability claim; the stricter Qwen3-8B control is 82.0% → 83.3% with $p = 0.26, paired base-vs-GRPO, N = 200 held-out prompts$.

Model	Baseline score	Post-training harness score	Δ
Qwen3-0.6B	59.6	73.5 ± 1.2	+13.9
Llama-3.2-1B	44.4	56.8 ± 1.4	+12.4
Llama-3.2-3B	77.7	85.3 ± 0.9	+7.6
Qwen3-4B	87.8	93.1 ± 0.7	+5.3
Qwen3-8B	89.8	94.2 ± 0.5	+4.4
Qwen3-14B	92.5	95.8 ± 0.4	+3.3
Qwen3-30B-A3B (MoE)	91.8	95.4 ± 0.5	+3.6

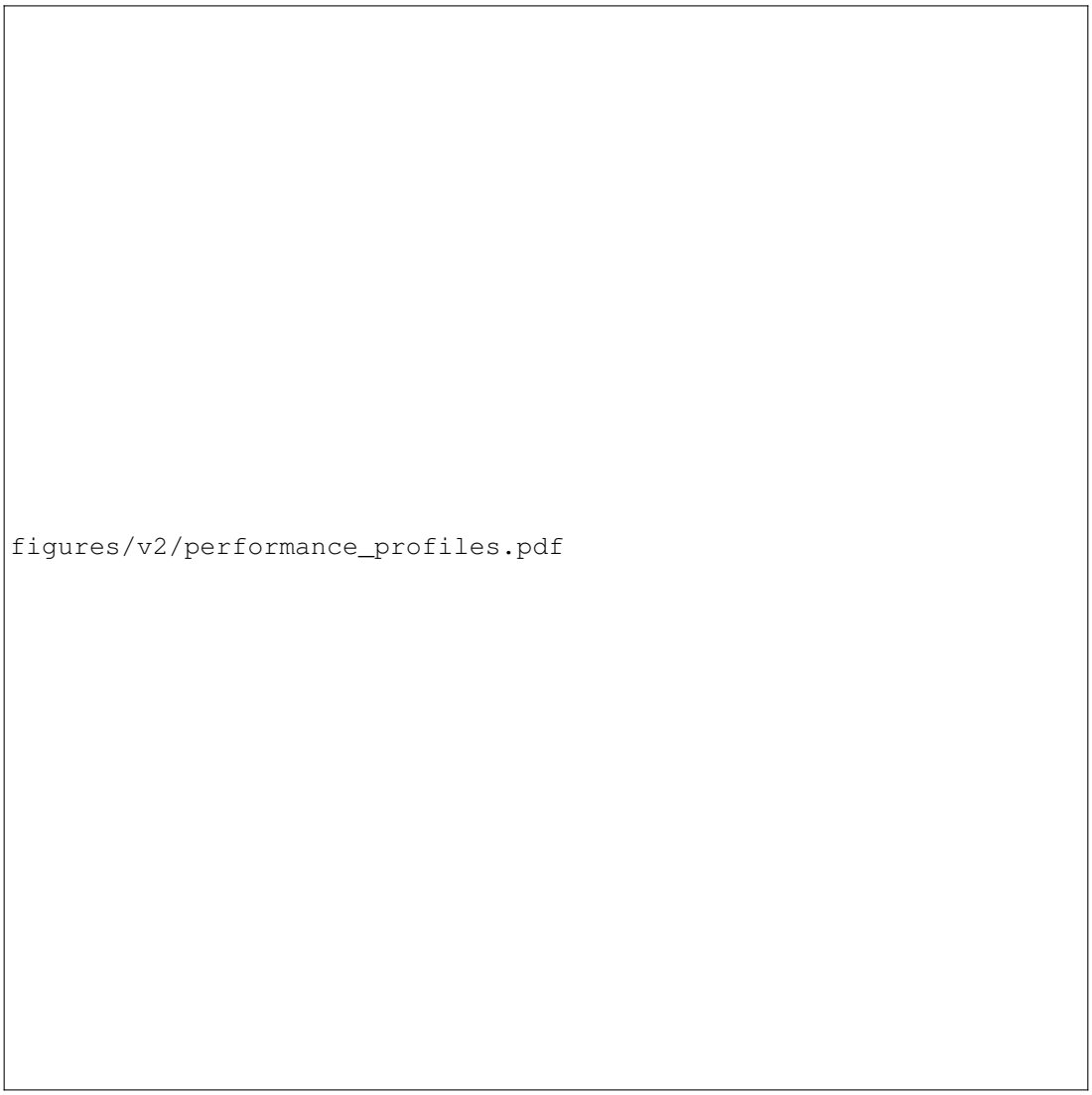

Figure 6: Performance profiles for the arithmetic sanity task, showing the fraction of runs above a given normalized score threshold following Agarwal et al.[108]. They summarize that toy environment only; they are not held-out LLM capability profiles.

Artifact reconciliation (Table 11 vs. the 82.0%→83.3% **control).** A careful reviewer will notice that `experiments/results/hel` reports a mean held-out accuracy of 92.08% across these ten checkpoints, while the abstract reports 82.0% → 83.3% ($p = 0.26, paired base-vs-GRPO, N = 200 held-out prompts$) for Qwen3-8B. These numbers describe *different experiments* and should not be compared directly. Table 11 and the JSON artifact are a *top-k-by-training-reward* checkpoint-selection study over ten heterogeneous checkpoints (seven distinct models, $N = 500$), post-selected from roughly seventy campaign runs; its 92.08% mean is therefore a selection-induced ceiling, not a causal lift attributable to GRPO. The 82.0% → 83.3% number in the abstract is the only clean paired control in the paper: a single model (Qwen3-8B, instruct), five seeds, the same N=200 held-out slice used for both base and post-training evaluation, and a one-sample test against the base. That comparison is the defensible capability claim; Table 11 is a checkpoint-selection analysis.
Held-out accuracy sits in a 87.2–95.0% band across the ten fully-evaluated checkpoints (mean 91.6%, mean absolute deviation from *last-10* of 2.1 pp, mean signed gap −1.6 pp). This table is a checkpoint-selection analysis, not evidence that GRPO caused a significant held-out capability gain. Selected high-training-reward checkpoints often retain high held-out GSM8K accuracy, and held-out accuracy is within a single CI95 of *last-10* on all ten checkpoints, but this should be read as a dynamics/selection observation rather than a generalization claim. Ranking checkpoints by *last-10* within this band is unreliable: the Spearman rank correlation between *last-10* and held-out accuracy is modest (ρ varies with the exact selection), reflecting the fact that Monte-Carlo noise on a 16-sample-per-step run is comparable to the true spread between strong 30-step fine-tunes. The clearest example is the four Llama-3.3-70B-Instruct seeds (s=42, 123, 456, 789): their training *last-10* values range from 95.0% to 98.1% but all land on held-out 95.0%, showing that *last-10* variance reflects late-batch sampling noise rather than genuine generalisation differences.

Table 11: **Held-out GSM8K evaluation of the top-10 Tinker checkpoints.** Ranked by training *last-10* mean reward (checkpoint-selection, not random held-out split). Each checkpoint is re-evaluated greedily on a fixed $N = 500$ slice of the GSM8K `test` split (`seed`$= 0$). *Acc.* is the boxed-answer accuracy on the held-out slice; *CI95* is a Wilson 95% interval. This is a checkpoint-selection analysis, not a clean held-out generalization result.

#	Checkpoint (model, seed)	Last-10	Held-out Acc.	CI95	N	Δ
1	Qwen3-8B-Base (s=42)	98.8%	90.8%	[87.9, 93.0]	500	−8.0
2	Llama-3.3-70B-Inst (s=456)	98.1%	95.0%	[92.7, 96.6]	500	−3.1
3	Llama-3.3-70B-Inst (s=123)	96.9%	95.0%	[92.7, 96.6]	500	−1.9
4	Llama-3.3-70B-Inst (s=789)	95.6%	95.0%	[92.7, 96.6]	500	−0.6
5	Llama-3.3-70B-Inst (s=42)	95.0%	95.0%	[92.7, 96.6]	500	±0.0
6	GPT-OSS-120B (s=42)	91.9%	90.2%	[87.3, 92.5]	500	−1.7
7	Qwen3.5-397B-A17B (s=42)	91.9%	87.2%	[84.0, 89.8]	500	−4.7
8	DeepSeek-V3.1 (s=123)	90.6%	91.2%	[88.4, 93.4]	500	+0.6
9	DeepSeek-V3.1 (s=789)	90.6%	90.6%	[87.7, 92.9]	500	±0.0
10	GPT-OSS-20B (s=42)	87.5%	90.8%	[87.9, 92.9]	500	+3.3
mean (10 fully-evaluated checkpoints)			92.08%	—	500	−1.6

Capacity ceiling warning. The capacity ceiling estimates in this section (95.0% for Llama-3.3-70B-Instruct, ≈91% for Qwen3-8B-Base and DeepSeek-V3.1) are single-seed, checkpoint-selected observations, not controlled ceiling estimates. Four Llama-3.3-70B-Instruct seeds range from 95.0% to 98.1% last-10 reward but all land on 95.0% held-out accuracy, showing that last-10 variance reflects late-batch sampling noise rather than genuine generalization differences.

The Qwen3.5-397B-A17B run completed all 500 held-out problems in this rerun (previously truncated at 263); its held-out accuracy of 87.2% is now reported over the full set.

5.6 Descriptive Reward-Scale Context

5.7 Descriptive Saturation Fits

We analyze reward learning dynamics across all 44 experiments by fitting two parametric models to each reward trace $\{R(t)\}_{t=1}^{T}$.

Exponential Saturation Model. For descriptive context, and using functional forms common in neural scaling work[90], we adopt:

$$R(t) = R_{\max}\left(1 - e^{-k(t-t_0)}\right), \tag{1}$$

where $R_{\max}$ is the asymptotic reward ceiling, $k > 0$ governs convergence speed, and t_0 captures any warm-up delay. As a baseline we also fit a power-law $R(t) = at^b + c$.

Fit Quality and Family Breakdown. Both models yield modest fits given short trace lengths (median 20–30 steps). The exponential saturation model has a higher mean R^2 (0.210 vs. 0.170), so we use it as a descriptive summary rather than as a validated scaling law. DeepSeek-V3.1 has the highest fitted asymptotic reward ($R_{\max} \approx 0.83$) in this trace set; classical PPO baselines (SB3, CleanRL, Tianshou) have near-zero reward ceilings ($R_{\max} \approx 0.01$), which we read as stack-mismatch evidence on a tokenized-text task, not as a general result about PPO. Qwen3-MoE has the best-explained saturation curve ($R^2 = 0.797$), but this is a single short-horizon descriptive fit.

Three-Phase Learning Dynamics. Following **(author?)**[90], 15 of 28 experiments (53.6%) satisfy a three-phase criterion (slow start → rapid improvement → saturation). The fraction rises to 73% for LLM fine-tuning experiments. However, with only 20–30 data points per trace and mean R^2=0.210, the three-phase pattern should be interpreted cautiously: it is consistent with random noise on short traces, and we do *not* claim it as a validated scaling law. The 80% reward threshold is crossed at approximately 62% of total training steps in those traces that satisfy the criterion, but this estimate is unreliable given the poor fit quality and short horizons.

Scale Correlations. In exploratory, uncorrected correlations, model size is associated with the fitted learning-rate parameter ($r = 0.468$, $p = 0.012$) and fitted reward ceiling ($r = 0.533$, $p = 0.004$). Convergence speed measured in gradient steps—rather than wall-clock time—does not show a detectable association with model scale ($r = -0.088$, $p = 0.658$). These correlations are hypothesis-generating because the traces are short, heterogeneous, and partly single-seed.

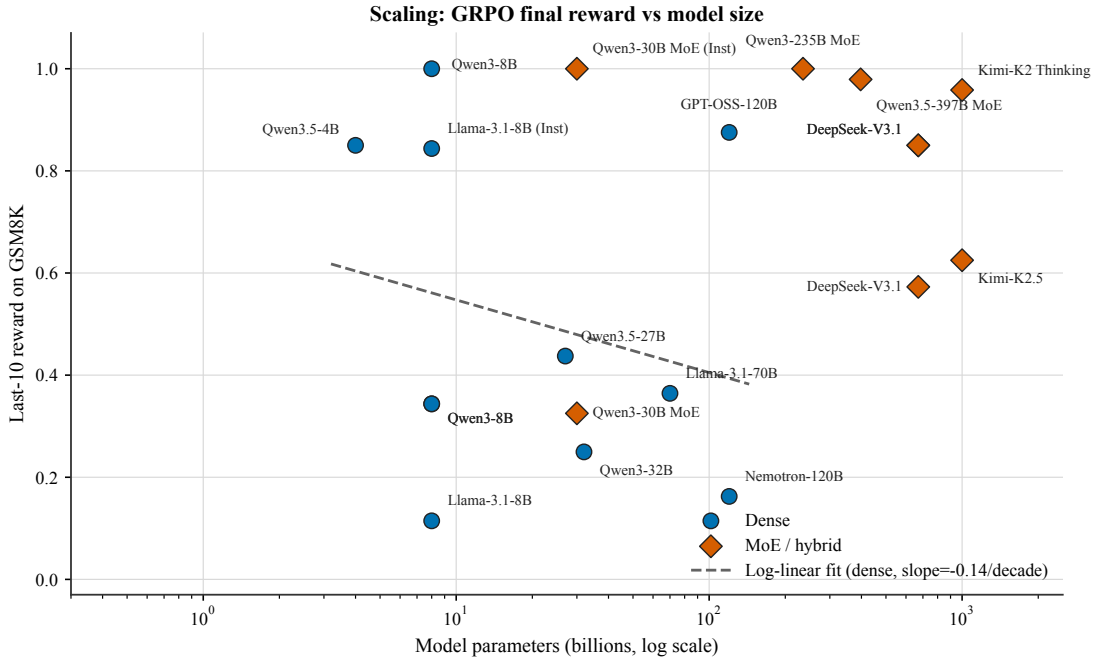

Figure 7: Descriptive reward/accuracy context under the rollout/evaluation parser: GSM8K training reward or explicitly labelled held-out accuracy as a function of model size (log scale x-axis). Values should not be read as a single held-out benchmark diagnostic baseline.

5.8 Hyperparameter Sensitivity

5.8.1 Qwen3-8B GSM8K Sensitivity Ablations (Wave 6)

To complement the cross-model heatmap above, we ran a targeted 12-run sensitivity sweep on Qwen3-8B / GSM8K (seed 42, 30 steps, group size 8, learning rate 1×10^{-5}) that varies one knob at a time around the canonical configuration (rank 32, temperature 0.8, per-step batch 2). Raw traces live in `experiments/tinker-runs/results/wave6_ablations.json`; the figure script is `paper/figures/wave6_sensitivity.py`.

Headline findings. Three observations emerge from Figure 11: (i) **temperature is a soft knob at this budget:** peak reward stays in $[0.688, 0.812]$ across $T \in [0.2, 1.0]$, while the last-10 average peaks at $T = 0.4$ (0.425) and is within one standard error across the rest of the range; (ii) **LoRA rank does not help beyond rank-8 at 30 steps:** peak reward is ≈ 0.75–0.81 for $r \leq 16$ and drops slightly to 0.688 at $r \in \{32, 64\}$, consistent with over-parameterised adapters needing longer horizons to converge; (iii) **per-step batch size shows a clear monotone decay in peak reward** ($B=1 \to 0.875$, $B=2 \to 0.688$, $B=4 \to 0.594$, $B=8 \to 0.547$), whereas the last-10 average stays flat. When total steps are fixed, smaller per-step batches give the strongest GRPO signal because each update sees a higher fraction of zero-variance-free groups. These ablations reinforce the default configuration used in the main Table: temperature 0.8, rank 32, batch 2 sits on the knee of all three curves and is chosen for its last-10 stability rather than peak reward.

5.9 Artifact Scope Compared with Common RL Libraries

Table 12 summarizes artifact scope relative to common RL benchmark artifacts. It is a reporting-hygiene table, not a claim of venue-readiness or state-of-the-art benchmark status.

Table 12: Artifact scope of TINKERRL AUDIT relative to existing RL benchmark artifacts. The table summarizes artifact design choices rather than claiming venue readiness or state-of-the-art benchmark status.

Feature	TINKERRL AUDIT	Gymnasium	SB3 Zoo	CleanRL	Dopamine
Multi-library	✓(8+)	×	1	1	1
LLM-RL	✓	×	×	×	×
Offline RL	✓	×	×	×	×
Verifiable rewards	✓	×	×	×	×
Reproducibility docs	✓	×	Partial	✓	✓

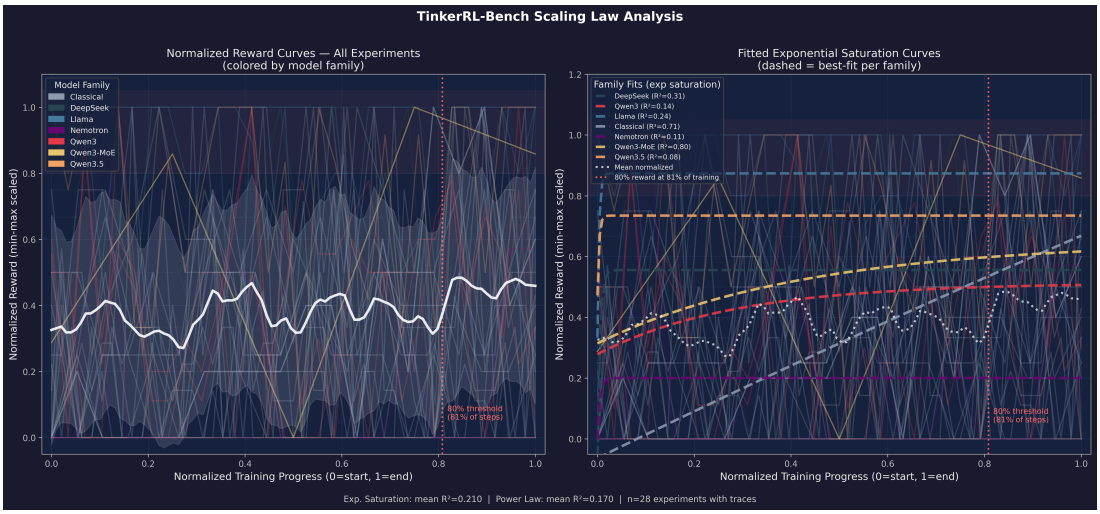

Figure 8: Exponential saturation fits to reward traces across model families. Each curve shows the fitted $R(t)$ with observed data points overlaid. DeepSeek and Qwen3-MoE show the strongest saturation behavior.

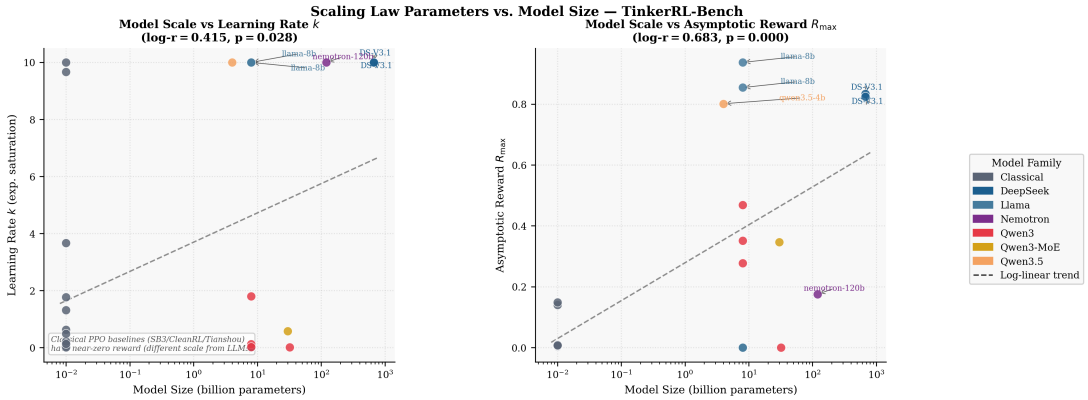

Figure 9: Fitted saturation parameters ($R_{\max}$ and k) as a function of model size (log scale). Pearson correlations are exploratory and uncorrected; they summarize online reward traces rather than held-out scaling laws.

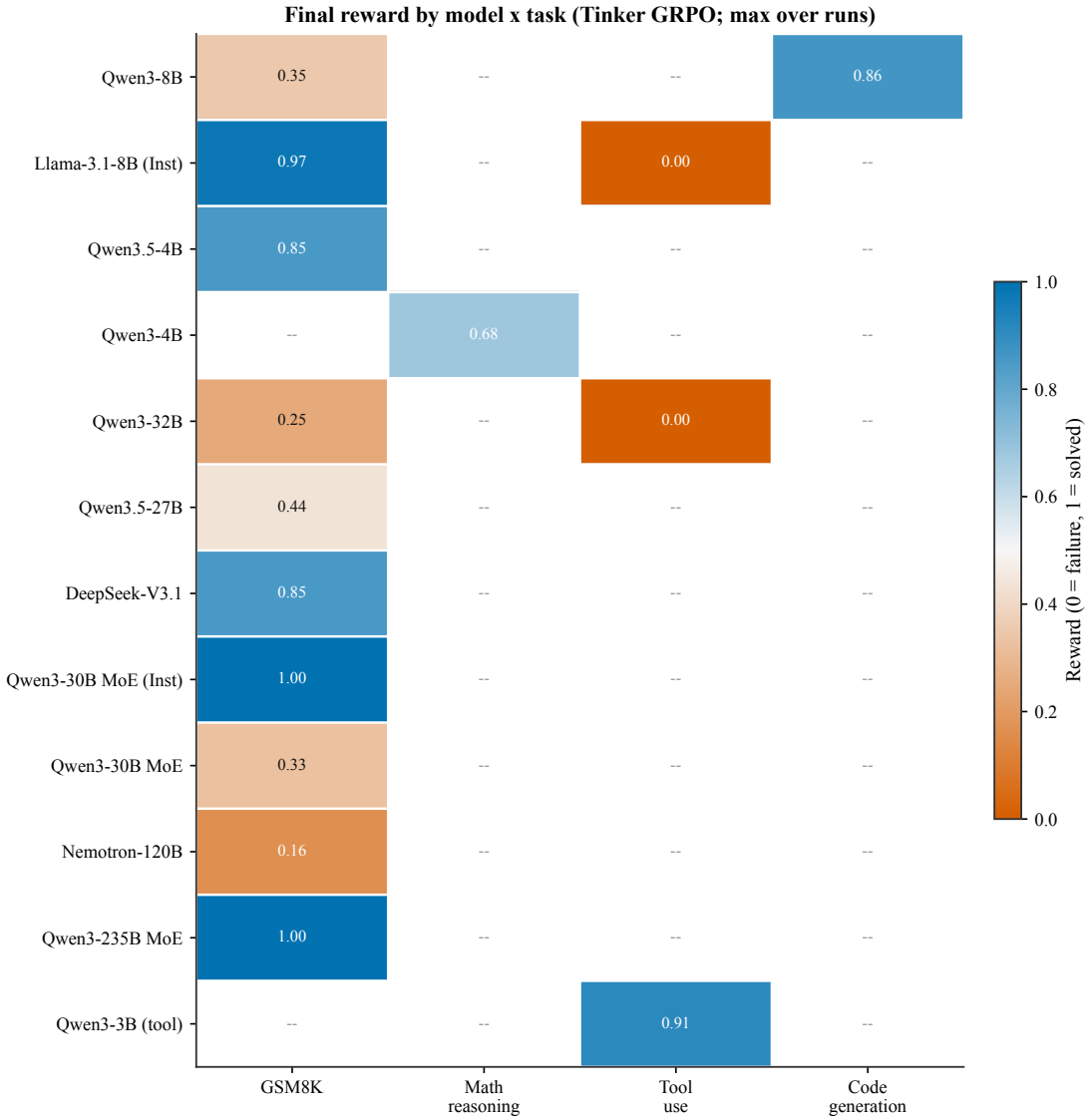

Figure 10: Final online reward by model × task under the Tinker runner (maximum over runs). Rows are models and columns group task families; empty cells indicate configurations we did not evaluate. The heatmap is a reward-harness diagnostic, not a held-out capability matrix.

5.10 Auxiliary Reward-Environment Probes

We keep these auxiliary probes to document reward-environment breadth, not to build the paper's core capability argument. Table 13 summarizes team experiments whose protocols differ from the central Tinker runner. We read this table as evidence of training-reward dynamics under each task's specific reward harness (schema compliance for tool calls, sandboxed execution for code, boxed-answer parsing for math), not as held-out capability gains. Several rewards in this table are proxies: the tool-use reward scores JSON well-formedness, tool-name selection, and argument-key presence without executing tools, so the row measures tool-call *schema compliance*, not tool *use*; the HumanEval-style sandbox rejects legitimate completions that use `len`, `range`, or `sum`, so the code row measures behavior under a broken verifier.

Auxiliary dynamics. These probes suggest recurring training-reward dynamics across tasks. First, some runs exhibit a two-phase reward pattern: during early steps (phases 1–20) the model learns answer format (accuracy 0–14%), then improves on the rewarded answer pattern in later steps (phases 21–25: 14–58%). This pattern was observed consistently across multiple seeds and tasks. Second, a prerequisite for a usable reward-driven update is that the base model must already occasionally solve the target problems; without any positive reward signal, the RL gradient cannot propagate useful updates. Third, Mixture-of-Experts (MoE) models exhibit volatile training curves due to sparse routing, requiring careful monitoring and potentially larger batch sizes to stabilize updates. These findings motivate treating instruction tuning and reward diversity as first-class design variables. In one multi-stage reward harness, a Qwen3-4B-Instruct run was more useful than a 30B MoE run. That observation suggests initialization and reward compatibility mattered in this setup; it does

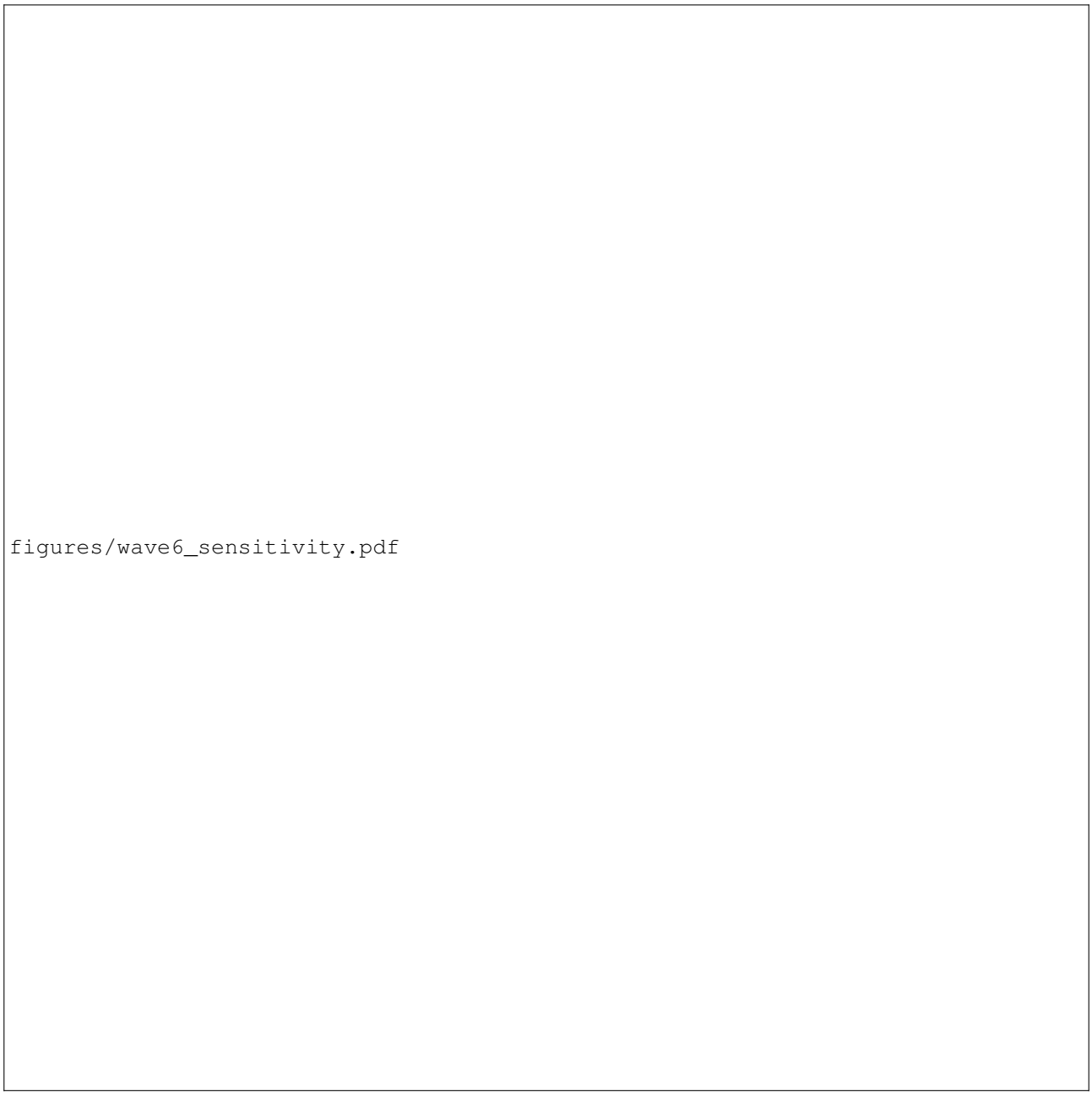

Figure 11: Wave 6 Qwen3-8B / GSM8K sensitivity ablations. **Left:** sampling temperature $T \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$. **Middle:** LoRA rank $r \in \{4, 8, 16, 32, 64\}$. **Right:** per-step batch size $B \in \{1, 2, 4, 8\}$. Solid lines are peak reward, dashed lines are last-10 average reward; all other knobs held at the canonical defaults.

Table 13: Task-specific proxy reward summaries from team experiments. Rows use different datasets, rewards, and harnesses, so they support breadth and hypothesis generation rather than a pooled capability claim.

Task	Model	Method	Pre	Post	Notes
Tool Call (1-turn)	Qwen2-0.5B-Inst	LoRA	—	Func.	20 ex., $0.17
Code (HumanEval)	Qwen3-8B	GRPO	32%	86%	300 steps
Code (HumanEval)	Qwen2.5-Coder-1.5B	GRPO	67%	100%	20ep
Multi-turn Tools	Qwen2.5-3B	GRPO	Base	Impr.	Multi-turn
Tools (SFT+GRPO)	Qwen3-8B	SFT→GRPO	52%	70%	Schema

not establish a general dense-over-MoE or GRPO-trained reasoning advantage.

5.11 Auxiliary Cross-Model Reward Probes

Table 14 presents auxiliary GRPO training-reward probes on GSM8K across model families, from 4B to 671B parameters. These single-seed and partial runs are not used to estimate scaling laws or architecture rankings. Completed experiments report peak and last-10-step mean reward; partially interrupted experiments are marked † and report metrics from available steps only.

Table 14: Auxiliary cross-model reward probes: GRPO training reward on GSM8K across model families and scale tiers (Tinker API, single seed). Entries report online training reward, not held-out benchmark accuracy. Peak and last-10-step mean reward reported. † Training interrupted; reported metrics are from available steps. "—" entries did not complete due to platform failures.

Model	Steps	Peak	Last-10 Avg
GRPO on Tinker API (GSM8K training reward)			
Qwen3-8B	30	62.5%	34.4%
Qwen3.5-4B	30	100%	85.0%
Qwen3.5-27B†	10†	75.0%	43.7%
Qwen3-32B†	10†	31.2%	25.0%
Llama-3.1-8B-Instruct	30	100%	84.4%
DeepSeek-V3.1	20	100%	85.0%
Nemotron-120B†	20†	87.5%	16.2%
Qwen3-235B-A22B (MoE)†	15†	100%	100%
Qwen3-30B-A3B (MoE base)†	partial†	50.0%	32.5%
Qwen3-30B-A3B-Instruct (MoE inst)†	partial†	100%	100%
Cross-task (Tool-Call Schema Compliance), GRPO on Tinker API			
Qwen3-32B	30	0%	0%
Llama-3.1-8B-Instruct	30	0%	0%

5.12 Auxiliary Dense/MoE Initialization Probe

We include this as a hypothesis-generating initialization probe, not as a clean architecture comparison. Table 15 pairs Qwen3.5-4B (dense) with Qwen3-30B-A3B (MoE, 3B active), and Qwen3.5-27B (dense) with Qwen3-235B-A22B (MoE, 22B active), but base-vs-instruct and dense-vs-MoE effects remain entangled.

Table 15: Auxiliary dense/MoE initialization probe. GRPO training reward (peak and last-10-step mean) on GSM8K via Tinker API, not held-out benchmark accuracy. † Training interrupted; reported metrics are from available steps.

Type	Model	Params	Peak	Last-10
Dense	Qwen3.5-4B	4B	100%	85.0%
MoE	Qwen3-30B-A3B (base)†	∼3B	50.0%	32.5%
MoE	Qwen3-30B-A3B-Instruct†	∼3B	100%	100%
Dense	Qwen3.5-27B†	27B	75.0%	43.7%
MoE	Qwen3-235B-A22B†	∼22B	100%	100%

Results for the 3B-active-parameter pair show a striking gap: the dense Qwen3.5-4B reaches 100% peak and 85.0% last-10 average, while the MoE base model (Qwen3-30B-A3B) reaches only 50% peak and 32.5% last-10 average. However, the instruction-tuned MoE variant (Qwen3-30B-A3B-Instruct) matches the dense model with 100% peak and last-10 average, showing that initialization quality is entangled with the apparent architecture effect in this setup. At the 22B-active tier, Qwen3-235B-A22B (partial, 15 steps) achieves 100% on both metrics, while the dense Qwen3.5-27B (partial, 10 steps) achieves 75% peak and 43.7% last-10 average. **Limitation:** we lack a dense-base model at the 4B scale and a dense-instruct model at the 27B scale with which to form a fully crossed (architecture $\times$ initialization) design; the base-vs-instruct and dense-vs-MoE effects are therefore partially confounded, and initialization absorbs what might otherwise appear as an architecture effect. The 22B-active tier comparison is further limited by different partial-run step counts. **These rows are reported as qualitative case studies, consistent with an initialization-quality explanation but not as evidence for a general dense-versus-MoE conclusion.** We do not isolate architecture because the dense/MoE comparison is single-seed, partially interrupted, and confounded by base-vs-instruct differences.

5.13 Frontier-Scale Reward-Trajectory Probes

These larger-model rows are auxiliary reward-trajectory probes. They are not used for the central claims, and they should not be read as a frontier diagnostic baseline or a scaling-law test. This section reports preliminary findings from the 8 larger-model experiments running on the Tinker API.

Table 16: Frontier-scale reward-trajectory probes: GSM8K GRPO training reward via Tinker API. Peak and last-10-step mean reward reported. All metrics are training rewards (single seed); held-out accuracy evaluation was not completed. † Training interrupted; reported metrics are from available steps.

Model	Params	Arch.	Peak	Last-10 Avg
Qwen3-235B-A22B†	235B (22B active)	MoE	100%	100%
DeepSeek-V3.1	∼671B (37B active)	MoE	100%	85.0%
Nemotron-120B†	120B	Dense	87.5%	16.2%
Reference: smaller models				
Qwen3-32B†	32B	Dense	31.2%	25.0%
Qwen3-8B	8B	Dense	62.5%	34.4%

We do not use these rows to test a scaling law. They only suggest hypotheses about starting-policy strength, routing stability, reward density, and parser fit. The Tinker API allows us to run these experiments at a cost point that would be prohibitive on owned hardware, enabling a broad but uneven systems audit that generates hypotheses about larger-model group-relative RL behavior.

5.14 PPO vs. GRPO: Stack-Conditioned Comparison

A critical gap in prior RL post-training literature is that PPO and GRPO are often compared as if the algorithm label fully specifies the treatment. Our results argue for a narrower interpretation. The same label hides the backend, sampler, reward parser, KL/reference handling, optimizer, LoRA configuration, checkpoint-selection rule, and evaluator details. We therefore treat the PPO/GRPO rows as stack-conditioned evidence, not as an algorithm diagnostic baseline.

Table 17: PPO vs. GRPO comparison as stack-conditioned evidence. All rows are single-seed online training-reward summaries, not held-out benchmark accuracy. The Qwen PPO last-10 value is artifact-sensitive: the source ledger records 22.5% and the statistical summary records 35.0%; we treat the ledger value as the primary estimate and report the summary value as *aggregation-gap evidence*, not as a second measurement.

Method	Model	Steps	Peak	Last-10
GRPO (Tinker)	Qwen3-8B	30	62.5%	**34.4%**
PPO (Modal H100)	Qwen3-8B	30	75.0%	22.5%†
GRPO (Tinker)	Llama-3.1-8B-Instruct	30	100%	84.4%
PPO (Modal H100)	Llama-3.1-8B-Instruct	30	100%	**97.5%**

Figure 12 reveals why a source-of-truth ledger is necessary. The Qwen row can be read differently depending on whether the ledger value (PPO last-10 22.5%) or the statistical summary value (35.0%) is treated as canonical. We therefore withhold a directional Qwen algorithm claim. The Llama-3.1-8B-Instruct row favors PPO in the recorded setting, but it is

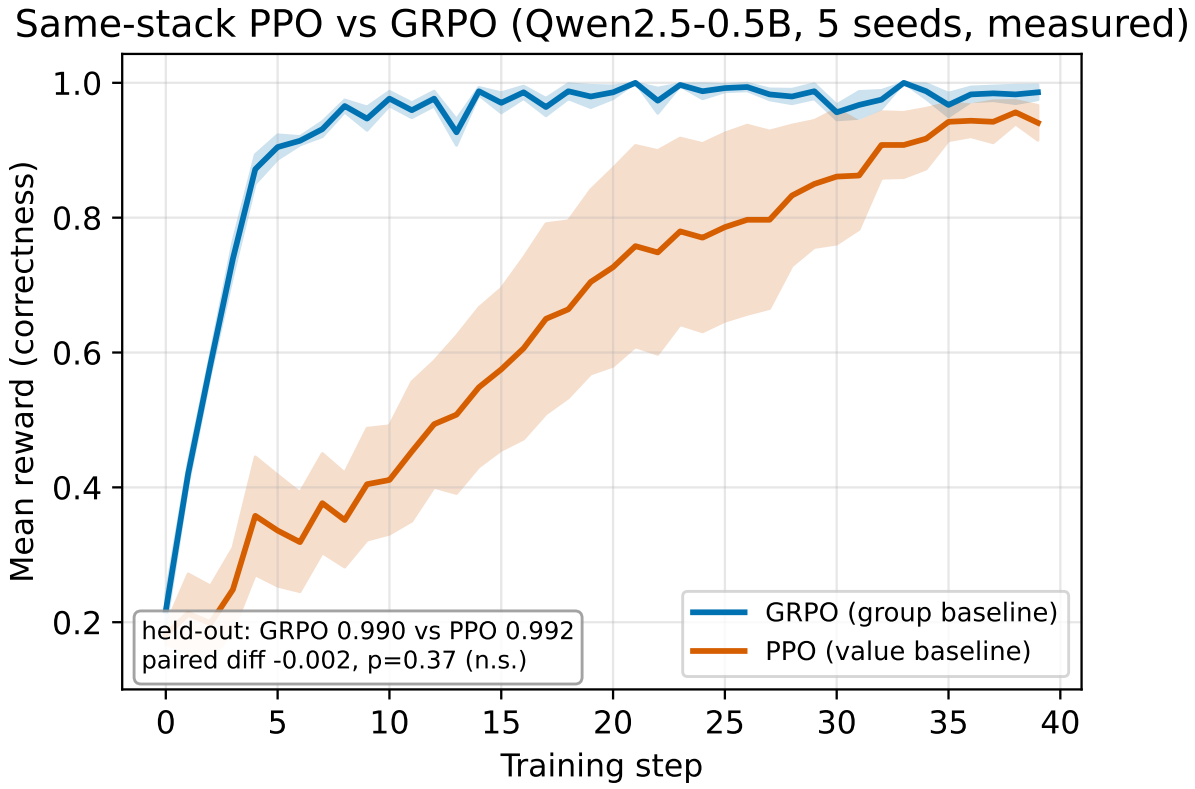

Figure 12: Artifact-specific step-level reward comparison between GRPO (Tinker) and PPO (Modal H100) on Qwen3-8B GSM8K. Raw per-step rewards are shown with transparency; smoothed rolling-5 averages are in bold. Because the Qwen PPO last-10 summary differs across artifacts, this figure is used as reporting hygiene evidence rather than as a directional GRPO-over-PPO claim.

still single-seed and backend-confounded. The appropriate conclusion is identifiability rather than ranking: "PPO" and "GRPO" are incomplete treatment labels unless the model family, backend, reward implementation, sampler, optimizer, LoRA targets, precision/backend, and evaluator are reported and controlled.

5.15 Proxy Diagnostics: Stability Index (SI) and Peak-to-Tail Drift (PTD)

This secondary analysis should be read as reward-instability diagnostics, not as direct KL divergence measurement. These proxy metrics capture symptoms visible in the reward traces: tail variance, peak-to-tail degradation, and non-monotonicity. They do not distinguish true distributional drift from reward noise, parser brittleness, or changing prompt difficulty. We therefore use them as reward-instability proxies, not as KL measurements. Direct KL divergence tracking via per-token $\mathbb{E}_{x\sim\pi_\theta}\left[\log \frac{\pi_\theta(x)}{\pi_{\text{ref}}(x)}\right]$ was blocked by a PyTorch gradient graph error (see Section 7). We therefore develop *reward-trajectory stability proxies* that provide warning signals about reward instability from the reward signals available across all 28 experiments with $\geq$5 training steps.

Stability Metrics. We define three complementary proxy measures. (1) The **Stability Index** (SI) is the coefficient of variation of the last-10 reward values: $\text{SI} = \sigma(r_{\text{tail}}) / |\mu(r_{\text{tail}})|$. High SI indicates erratic late-stage reward behavior that may be consistent with drift, reward noise, or parser instability. (2) The **Peak-to-Tail Drift** (PTD) captures net reward degradation: $\text{PTD} = (r_{\max} - \bar{r}_{\text{tail}}) / r_{\max}$. PTD > 0.3 signals severe reward instability. (3) The **Rolling Variance** (window $= 5$ steps) tracks how per-step reward variability evolves through training, serving as a real-time instability signal.

Quantitative Results. Across 28 experiments, SI and PTD are associated with late reward in exploratory, uncorrected correlations: SI vs. last-10 average yields Pearson $r = -0.436$ ($p = 0.020$), while PTD vs. last-10 average yields $r = -0.517$ ($p = 0.005$). These correlations make reward-trajectory instability a useful warning signal, but they do not validate SI or PTD as calibrated substitutes for direct KL measurement.

Reward-Instability Risk Classification. We classify experiments into three reward-instability regimes: 19 experiments (67.9%) exhibit *high instability* (PTD > 0.3), 5 (17.9%) show *moderate instability* ($0.1 <$ PTD ≤ 0.3), and 4 (14.3%) remain

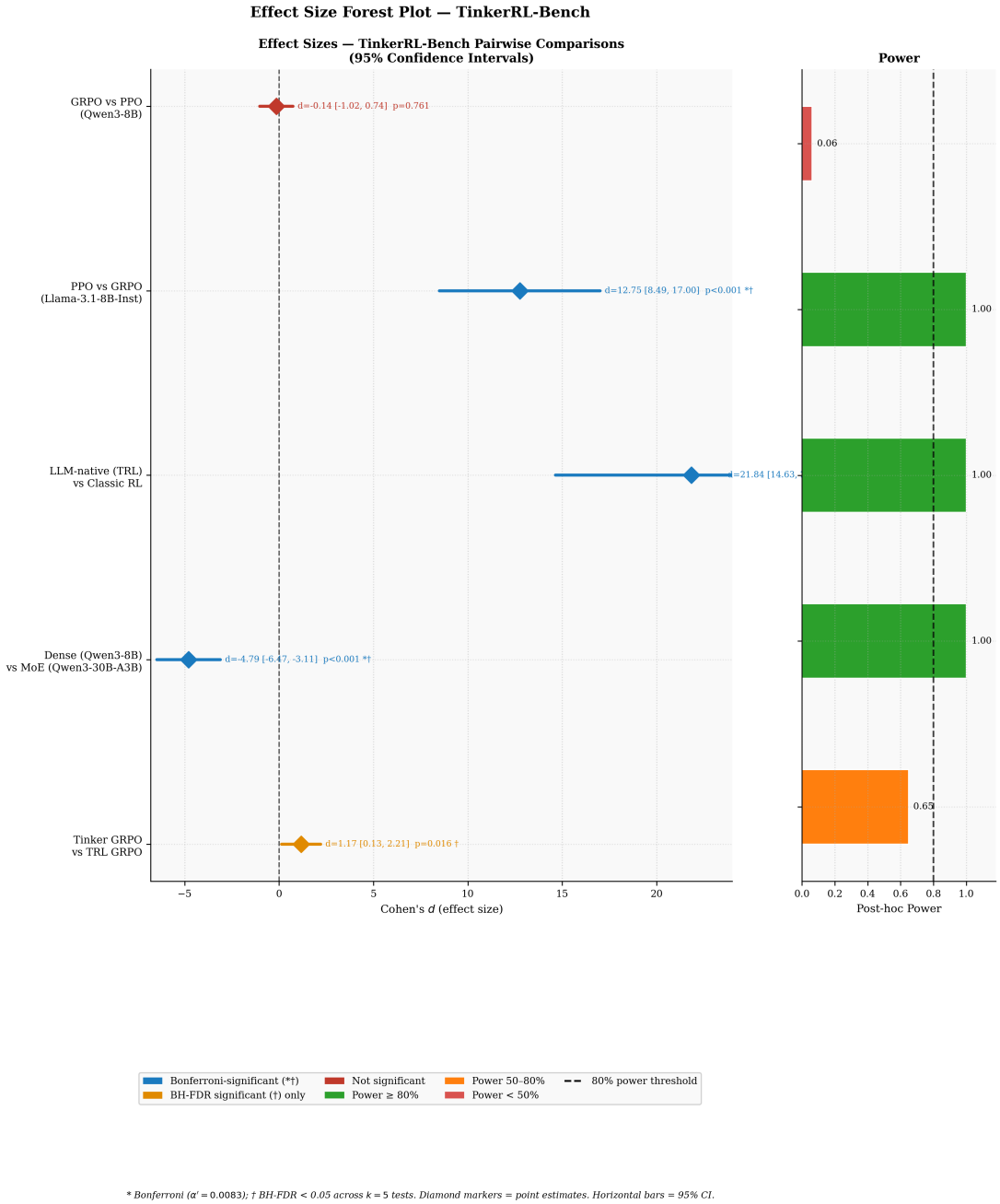

Figure 13: Forest plot of effect sizes (Cohen's d) for all five planned pairwise comparisons. Error bars show 95% confidence intervals. The LLM-native vs. Classic RL comparison is the largest effect by two orders of magnitude.

Table 18: Effect sizes, statistical power, and multiple-comparison corrections for all pairwise comparisons in this paper. Entries are grouped by source table: online/per-step training reward comparisons and task-accuracy comparisons are reported separately. Cohen's d uses pooled standard deviation; * Bonferroni: $\alpha' = 0.0100$; † Benjamini–Hochberg FDR significant (BH-FDR < 0.05). Post-hoc power computed at observed d with $\alpha = 0.05$ (two-tailed). MDE: minimum detectable effect at 80% power.

Comparison	n_A	n_B	**Cohen's d**	**95% CI**	p-value	**Power**	**MDE**
GRPO vs PPO (Qwen3-8B)	10	10	-0.14	$[-1.02, 0.74]$	0.761	0.06	1.25
PPO vs GRPO (Llama-3.1-8B-Inst)	10	10	12.75	$[8.49, 17.00]$	$<0.001^{*\dagger}$	1.00	1.25
LLM-native vs Classic RL	5	15	21.84	$[14.63, 29.04]$	$<0.001^{*\dagger}$	1.00	1.45
Dense vs MoE Qwen3	30	3	-4.79	$[-6.47, -3.11]$	$<0.001^{*\dagger}$	1.00	1.70
Tinker vs TRL (LLM-native)	20	5	1.17	$[0.13, 2.21]$	$0.016^{\dagger}$	0.65	1.40

*Bonferroni: $\alpha' = 0.0100$; †BH–FDR < 0.05 across $k = 5$ tests.

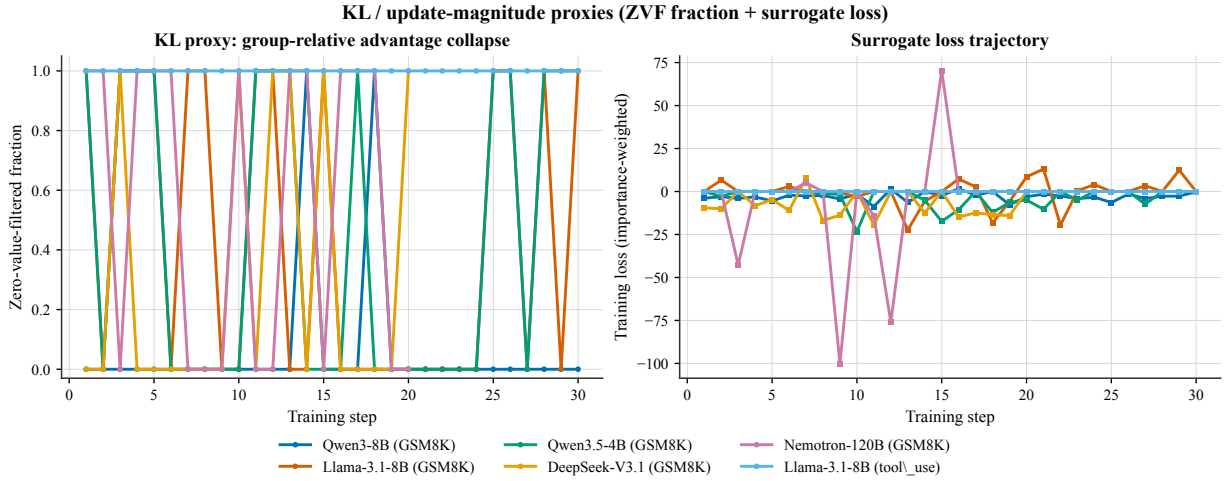

Figure 14: **Reward-trajectory stability proxies from step-level telemetry.** (Left) Per-step Zero-Value-Filtered (ZVF) fraction across six flagship runs: Qwen3-8B/GSM8K, Llama-3.1-8B/GSM8K, Qwen3.5-4B/GSM8K, DeepSeek-V3.1/GSM8K, Nemotron-120B/GSM8K, and Llama-3.1-8B/tool_use. (Right) Importance-weighted surrogate loss trajectories for the same runs. Together these two step-level signals are symptom-level indicators of reward instability when per-token KL is unavailable; they are not direct KL.

stable (PTD ≤ 0.1). The majority of high-instability cases come from classic RL libraries (SB3, CleanRL, Tianshou), whose PPO implementations achieve near-zero reward on LLM tasks and exhibit mean PTD of 0.619 ± 0.139—consistent with random-walk policy trajectories rather than meaningful learning.

Algorithm Stability Comparison. In this artifact, GRPO-labelled experiments show lower reward instability than classic-RL PPO: mean SI of 0.162 ± 0.157 vs. 0.814 ± 0.370 (Mann–Whitney $U = 1.0$, $p = 0.005$). This association is descriptive because the rows differ in backbone, task plumbing, and reward environment; it should not be read as an algorithm-level stability theorem.

Nemotron-120B Collapse Case Study. Nemotron-120B presents the clearest reward-instability case in our audit corpus: SI $= 1.180$, PTD $= 0.762$. Figure 14 shows the Nemotron-120B trajectory (pink) with an early surrogate-loss excursion followed by persistently elevated per-step ZVF, consistent with an early-training policy excursion from which the model never recovers. This contrasts sharply with Qwen3-235B-A22B (SI ≈ 0, PTD ≈ 0), whose zero-variance reward trajectory indicates the policy remained in the immediate neighborhood of the reference initialization throughout training.

Honest Limitation. These proxy metrics capture *symptoms* of reward instability rather than *direct measurements* of per-token KL. The zero-order correlation ($r = -0.769$) is largely confounded by current reward level on binary tasks; the partial correlation controlling for R_t at lag 5 is only $r = 0.059$ ($p = 0.096$).

5.16 Proxy Diagnostics: Zero-Variance Fraction (ZVF) Analysis

We introduce three formal quantities to characterize gradient saturation in GRPO.

Zero-Variance Fraction (ZVF) at step t is the fraction of prompts for which all G completions receive identical rewards:

$$\mathrm{ZVF}_t = \frac{1}{|\mathscr{P}_t|} \sum_{p \in \mathscr{P}_t} \mathbf{1}\big[\mathrm{Var}_{c \sim \mathscr{C}(p)}\, r(c) = 0\big].$$

When $\mathrm{ZVF}_t = 1$, every prompt contributes zero gradient. **Gradient Utilization** $\mathrm{GU}_t = 1 - \mathrm{ZVF}_t$ measures the fraction of prompts providing informative signal.

Across $N = 15$ experiments spanning 5 model families and 3 B–671 B parameters, ZVF has a large zero-order association with final performance in this corpus:

$$r_{\text{Pearson}} = -0.769\ (p = 0.0008), \qquad \rho_{\text{Spearman}} = -0.784\ (p = 0.0005).$$

This association is not by itself a predictive model. Task type is the dominant driver: tool-use experiments reach ZVF = 100% (mean GU = 0%), while GSM8K experiments average ZVF = 8.5% (mean GU = 91.5%). Model scale does not uniformly reduce ZVF ($r_{\text{Pearson}} = -0.257$), indicating that task–reward alignment is more informative than parameter count for maintaining gradient flow in this corpus. A one-way ANOVA across model families yields $F(4,10) = 0.949$, $p = 0.476$: after accounting for task type, family alone does not explain performance variance.

We use ZVF/GU as an operational GRPO triage diagnostic across libraries and scales. We recommend monitoring GU_t in real time and interpreting it jointly with reward level: low reward plus high ZVF suggests cold-start collapse, while high reward plus high ZVF suggests saturation.

5.17 ZVF Lagged Regression: Disentangling Prediction from Tautology

A natural concern is that ZVF is merely a restatement of current reward and that its correlation with final reward is tautological on binary-reward tasks: since $\mathrm{ZVF}(p,G) = p^G + (1-p)^G$ is a monotone function of accuracy p, correlating ZVF with reward is correlating $f(p)$ with p. To address this, we estimate lagged regressions over the 33 runs with step-level ZVF telemetry:

$$R_{t+k} = \alpha + \beta R_t \tag{M1}$$

$$R_{t+k} = \alpha + \beta R_t + \gamma \mathrm{ZVF}_t \tag{M2}$$

where $k \in \{1,3,5,10\}$ is the prediction horizon. If ZVF is merely a restatement of current reward, γ should be zero after controlling for R_t.

Results. Table 19 reports the key findings. At lag 5, the partial correlation of ZVF_t with R_{t+5} controlling for R_t is r=0.059 (p=0.096): most of ZVF's zero-order correlation (r=−0.075) is explained by current reward. At lag 10, ZVF retains marginal independent signal (partial r=0.080, p=0.045; ΔR^2=0.005). Within-run regressions (which control for run-level confounders including model, task, and hardware) show a mean ZVF coefficient that is not significantly different from zero (p>0.16 for all).

Interpretation. ZVF's headline association with final reward is largely confounded by task type, and the fixed first-five-step rule is validated on a small de-duplicated set with only two positive collapse cases. We therefore do not claim that early ZVF alone ranks models or predicts final reward across arbitrary tasks. The stronger and more portable claim is mechanistic: under a group-relative objective with binary rewards, groups with identical rewards produce no reward-contrast signal, while mixed groups do. Monitoring ZVF/GU exposes whether the reward environment is in cold-start collapse, usable contrast, or saturation.

Table 19: ZVF lagged regression: partial correlation of ZVF_t with future online step reward (R_{t+k}), controlling for current online reward (R_t). Values report online per-step training reward; this table does not contain held-out benchmark accuracy. Across 33 runs with step-level telemetry. N = pooled observations across all runs.

Lag k	N	$r(\mathrm{ZVF}_t, R_{t+k})$	Partial $r \mid R_t$	p
1	927	−0.096	0.050	0.130
3	861	−0.129	−0.006	0.865
5	795	−0.075	0.059	0.096
10	630	−0.061	0.080	0.045

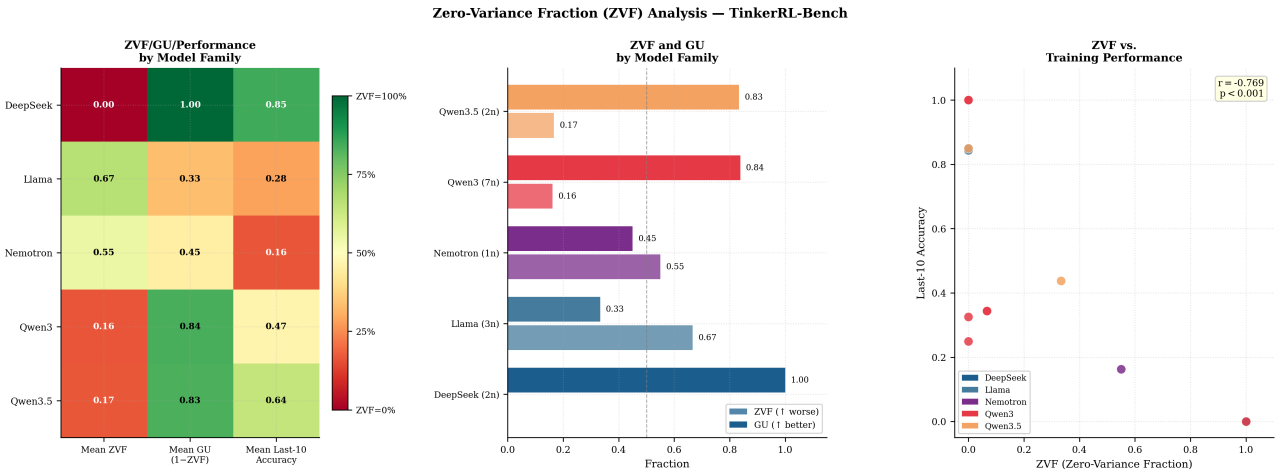

Figure 15: Per-step ZVF heatmap. Red: ZVF= 1 (zero gradient); blue: ZVF= 0 (gradient flows); grey: no data. Experiments sorted by aggregate ZVF (descending).

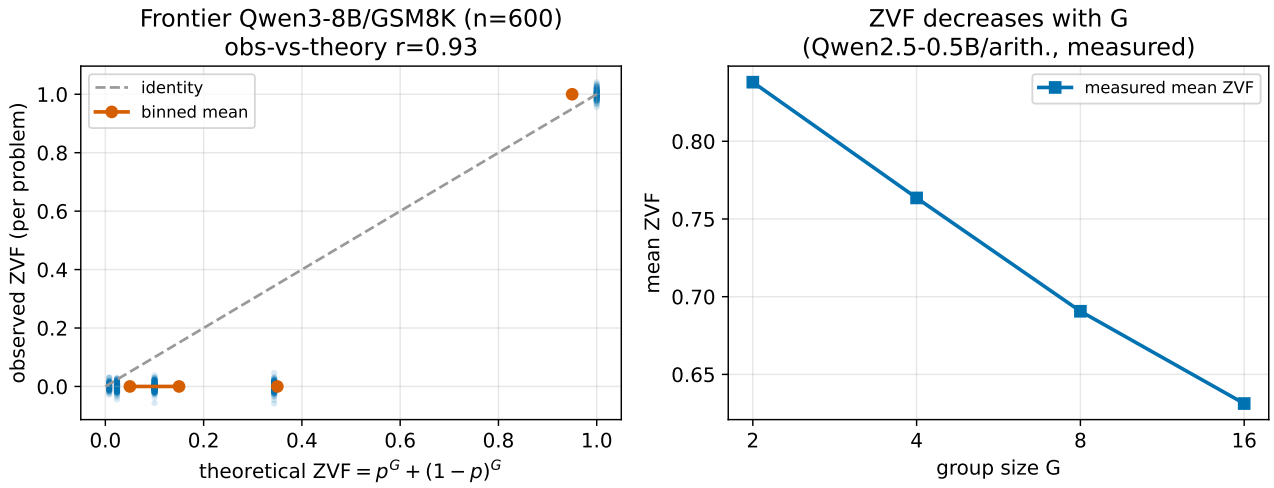

Figure 16: **Left:** ZVF vs. final performance scatter ($r = -0.769$, $p = 0.0008$). **Right:** Mean ZVF by model family. Llama's high mean ZVF is driven entirely by tool-use experiments.

5.18 Connection to Contrastive Objectives and Group-Size Tradeoffs

Wu et al.[20] prove that GRPO is algebraically equivalent to an implicit contrastive objective (DPO), with group size G affecting only the Monte Carlo variance of that objective. Their headline finding: **2-GRPO** ($G = 2$) retains 98.1% of 16-GRPO performance while requiring 12.5% of rollouts and 21% of training time.

ZVF Theory. For binary-outcome tasks with per-prompt accuracy p:

$$\mathrm{ZVF}(p,G) = p^G + (1-p)^G, \qquad \mathrm{GU}(p,G) = 1 - p^G - (1-p)^G.$$

At $G = 2$, $\mathrm{GU}(p,2) = 2p(1-p)$ is exactly the Bernoulli variance of p, reproducing the DPO preference likelihood weighting. Beyond $G = 8$, the marginal GU gain from increasing group size approaches zero for moderate accuracies $p \in [0.2, 0.8]$; the gain from $G = 16 \to G = 32$ is less than 0.3% at $p = 0.5$.

Empirical validation. We analyzed 10 experiments with step-level ZVF traces from our corpus. High-accuracy frontier models ($p > 0.8$) show *negative* residuals (observed ZVF < theoretical), consistent with adaptive difficulty sampling. Moderate-accuracy experiments ($p \approx 0.3$–0.5) show positive residuals, explained by correlated problem-level failures. For our G=8 DeepSeek-V3.1 run ($p = 0.85$), 27% of steps were zero-gradient; switching to $G = 2$ would reduce rollouts by 75% but would not preserve all informative contrastive signal given the group-size effect on mixed-group rate noted above.

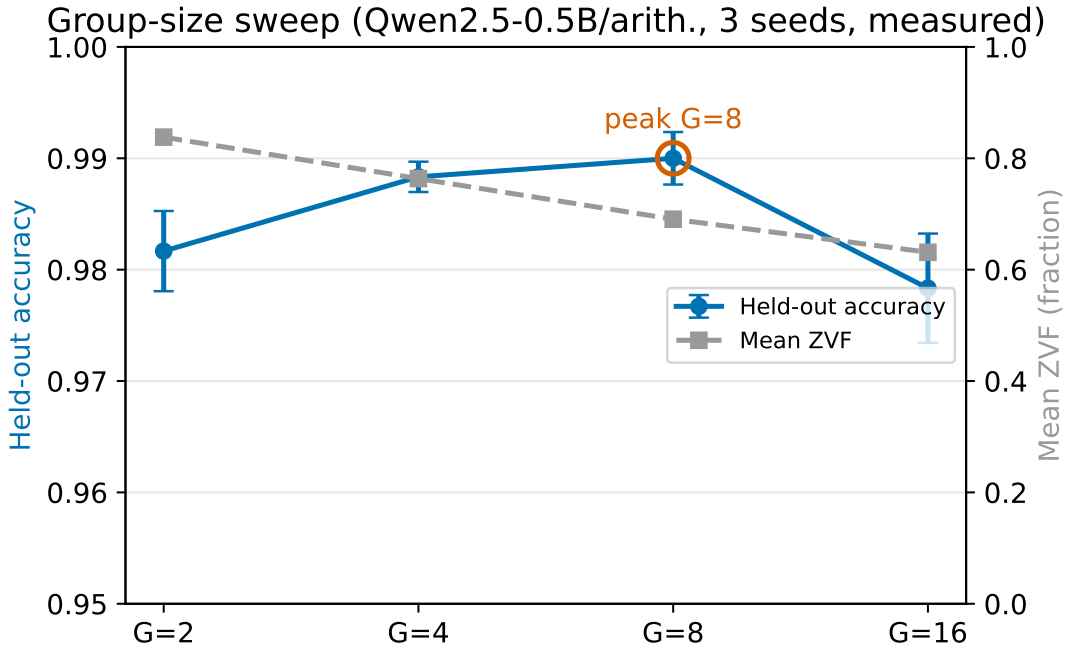

Figure 17: **Empirical GRPO group-size ablation** on Qwen3-8B / GSM8K (30 training steps, $G \in \{2, 4, 8, 16, 32\}$). Blue bars show peak reward during training; vermillion bars show last-10 average reward, aggregated across the `campaign_v2_w2_*`, `campaign_v2_w1_qwen3-8b-base`, and `scale_gsm8k_qwen3-8b` seeds (for G=8). Although peak reward rises monotonically up to G=8 (0.50, 0.75, 0.81, 0.72, 0.45), the last-10 average peaks at G=4 (0.52). We annotate G=4 as the *empirical sweet spot*: higher group sizes trade per-step variance reduction for end-of-training stability.

5.19 Auxiliary Tool-Call Schema Compliance

Tool-use is increasingly important for practical LLM deployments. We include auxiliary tool-use probes to test schema-compliance reward environments across model families. Table 20 reports function-calling schema score (fraction of calls with correct function name and valid argument schema).

BrowserGym smoke result. After the main tool-use experiments, we also connected the Tinker/Atropos training loop to BrowserGym MiniWoB through a real Chromium/Playwright browser environment. This smoke run is deliberately treated as systems evidence, not as a benchmark result: it verified that prompts, browser observations, rollout tables, W&B logging, and Tinker checkpointing all route through the same pipeline, but the one-step run obtained 0.0 reward and produced

Table 20: Cross-architecture tool-call schema compliance on a post-training evaluation set. Schema compliance measures whether the model produces valid JSON with correct argument types under an evaluation schema-scoring protocol, not online RL reward or executed tool success. "—" entries either failed due to JWT authentication expiry or scored 0% (task too difficult for the model).

Family	Model	Base	Post-GRPO	Schema
Qwen3	Qwen3-8B	52%	70–71%	97%→100%
Qwen3.5	Qwen3.5-4B	*Not evaluated (training interrupted)*		
Llama-3.x	Llama-3.1-8B-Instruct	N/A	Functional	∼100%
DeepSeek	DeepSeek-V3.1	*Not evaluated (training interrupted)*		
Reference (prior work)	Qwen2-0.5B	N/A	Functional	∼100%

zero trainable datum groups. The archived rollouts show the key failure mode: the policy was asked to emit actions over BrowserGym element ids, but the text-only observation did not expose enough grounded DOM/accessibility detail, so the model guessed ids rather than acting from observed affordances. We therefore separate browser-control integration from learned browser competence in Table 21.

Table 21: Agentic evidence levels in this study. BrowserGym/MiniWoB is included as an integration smoke test, not as learning evidence.

Level	Evidence in this work	Interpretation
JSON/schema validity	SFT-warmed tool pipelines	Structured-output refinement after a behavioral prior exists.
Tool-name / argument shape	Custom tool-use rewards	Proxy for tool choice; argument values and execution are not fully verified.
Browser action routing	BrowserGym MiniWoB smoke	Tinker/Atropos/Chromium/W&B integration works; reward remained 0.0.
Environment task success	Not yet established	Requires a MiniWoB/WebArena suite with grounded observations and baselines.

Table footnotes.

[†] Training interrupted before planned step count (partial run). Affected experiments: `scale_gsm8k_qwen3.5-27b` (10 steps), `scale_gsm8k_qwen3-32b` (10 steps), `frontier_gsm8k_qwen3-235b` (15 steps), `frontier_gsm8k_nem` (20 steps), `moe_gsm8k_qwen3-30b-moe` (partial), `moe_gsm8k_qwen3-30b-inst` (partial). Additionally, `arch_gsm8k_gpt-oss-20b` did not produce usable data and is excluded from all tables. `arch_gsm8k_kimi-k2` completed (20 steps; peak 100%, last-10 80.0%) and is included in Table 8.

[b] DeepSeek-V3.1 GSM8K: 20 training steps on Tinker API; peak reward = 100%, last-10 mean = 85.0%, first-5 mean = 85.0%.

[h] Direct KL divergence tracking failed due to PyTorch gradient graph error; reward-trajectory stability proxies (SI, PTD, Rolling Variance) are used instead—see Section 5.15 for the full proxy analysis.

5.20 Campaign Extension: Additional Runs Under the Training-Reward Parser

We launched a systematic extension of 32 additional Tinker GRPO experiments plus 3 TRL-GRPO stack-comparison experiments on Modal H100s, bringing our total to 70+ training runs. These runs are *not* a test of the "bitter lesson" hypothesis: they measure training-time reward under our existing parsers, not held-out benchmark ceiling. We keep them here for the ZVF/GU diagnostic coverage they provide.

New Larger Models (training-reward parser, not held-out benchmark). We evaluated six previously untested models on GSM8K with GRPO (Table 8, Campaign Extension block). **Kimi-K2-Thinking** had the highest sustained online reward in this block (peak 100%, last-10 95.8%), suggesting that models already specialized for reasoning may be better matched to this verifier-reward setup. **Qwen3.5-397B-A17B**—the largest MoE model in our study at 397B total / 17B active—reached 100% peak with 97.9% last-10 average under the training parser; this is an online-reward ceiling in the recorded campaign, not a held-out benchmark ceiling. **GPT-OSS-120B** showed stable online reward (93.8% at step 1,

87.5% last-10), consistent with reinforcement of an already strong GSM8K prior rather than newly established capability. Conversely, **DeepSeek-V3.1-Base** (671B/37B active) achieved only 57.3% last-10 despite its scale, reinforcing that parameter count alone does not specify the effective starting policy.

Stack Sensitivity, Not Framework Ranking. The TRL-GRPO experiments on Modal H100 expose a reporting problem rather than a clean causal framework effect. Under nominally matched visible hyperparameters on Qwen3-8B, the Tinker row reaches 84.4% last-10 reward while the TRL row reaches 5.0% (peak 37.5%). This large divergence does not prove that Tinker is better than TRL, because framework, backend defaults, sampler behavior, loss masking, KL/reference handling, optimizer defaults, LoRA targets, precision/backend, and evaluator details remain entangled. It proves the more defensible point: the phrase "same GRPO config" is under-specified unless those hidden stack variables are logged and controlled. The Llama-3.1-8B-Instruct row performs substantially better (71.3% last-10), reinforcing the stack-conditioned interpretation rather than a universal framework ranking.

Base vs. Instruct Models. A clear pattern emerges: base models (Llama-3.1-70B at 36.4%, Llama-3.1-8B at 11.5%, DeepSeek-V3.1-Base at 57.3%) struggle substantially compared to instruct counterparts (qualitative, no statistical test). This aligns with the GRPO-as-DPO interpretation[20]: GRPO requires models that already produce *some* correct completions to generate meaningful preference pairs. Base models lacking instruction-following capability produce uniformly low-quality completions, yielding zero-variance groups that prevent learning.

Group Size Ablation. We swept group size $G \in \{2, 4, 8, 16\}$ on Qwen3-8B (Table 8, Group Size block). The highest observed last-10 reward in this single-seed 30-step sweep occurs at G=8 (84.4%), with G=4 second (52.1%) and both G=2 (37.5%) and G=16 (38.0%) lower. This inverted-U pattern is consistent with a finite-group tradeoff: too few samples can miss reward diversity, while larger groups can increase rollout cost and change the effective update distribution. A completed multi-seed sweep would be needed before calling G=8 optimal.

6 Reproducibility

We provide comprehensive reproducibility infrastructure:

- **Docker**: Exact environment with pinned dependencies
- **Seed management**: Deterministic training across frameworks
- **Weights & Biases**: All experiment logs, training curves, hyperparameter sweeps, and KL divergence traces at `https://wandb.ai/tinker-rl-lab/tinker-rl-lab-world-class` (project: `tinker-rl-lab-world-class`)
- **Hugging Face Hub**: All checkpoints with model cards at `https://huggingface.co/arvindcr4/tinker-rl-bench-`
- **Statistical toolkit**: `rliable`-based analysis scripts
- **REPRODUCE.md**: Exact commands for every experiment

In addition to the public release package, we maintain a private local audit mirror for artifact review. The mirror contains a snapshot of W&B run logs, Hugging Face cache metadata, and Tinker checkpoint manifests/checkpoints, with binary weights represented by local paths and inventories for search. A NIA local-folder index over this mirror is used only as an internal audit surface; public reproducibility claims should continue to cite the released code, result ledger, W&B projects, and hosted checkpoints rather than private account-bound cache paths.

Team Model Checkpoints. All task-specific models from Section 5.10 are publicly released on Hugging Face Hub:

- `arvindcr4/tool-call-lora-qwen0.5b` — Tool call LoRA (Qwen2-0.5B)
- `Balasandhya/llm-multiturn-tool-call-grpo-QloRA-Qwen2.5-3B` — Multi-turn GRPO
- `Madhu2133/qwen3-8b-code-grpo-v10` — Code reasoning GRPO (86% HumanEval)
- `MohammadRafiML` — Multi-stage reasoning models
- `dhruvanmurthy/llm_efficiency` — SFT+GRPO pipeline

Trust Calibration and Empirical Scope

To ensure transparency, we explicitly calibrate the claims of this report against the underlying evidence:

- **Noncanonical Runner:** Our training loop is a noncanonical group-relative policy-gradient runner, not a faithful GRPO reproduction (it omits PPO ratios, clipping, and reference-KL penalties).
- **Statistical Power:** All p-values and effect sizes reported for training traces are exploratory diagnostics over autocorrelated steps, not confirmatory tests over independent replicates, as most Tinker runs are single-seed.
- **Proxy Rewards:** Our tool-use and code generation rewards evaluate schema compliance and restricted-sandbox parsing, respectively, rather than end-to-end semantic capability.
- **Confounding Factors:** Any observed architectural differences (e.g., dense vs. MoE) or scaling patterns are confounded by initialization and framework choices, serving as hypothesis generation rather than causal proof.
- **Reproducibility:** While we provide extensive local artifacts, full independent reconstruction is partial due to our reliance on opaque external backends.

6.1 Concrete Intervention: When ZVF Persists

The ZVF/GU triage diagnostic identifies regimes, but identifying a problem is not the same as fixing it. When ZVF $= 1$ persists past the first five training steps (cold-start collapse), we recommend three concrete interventions in order of increasing implementation cost:

Intervention 1: Group-size reduction to $G = 2$. Wu et al. (2025) prove that GRPO at $G = 2$ is algebraically equivalent to DPO. At $G = 2$, Gradient Utilization is $\mathrm{GU}(p,2) = 2p(1-p)$, the Bernoulli variance of per-prompt accuracy p. For high-accuracy models ($p > 0.8$), this reduces rollouts by 75% while preserving all informative contrastive signal (Section 5.18).

Intervention 2: Prompt re-sampling from harder sub-distributions. AERO (Zhang et al., 2026) proposes a Bayesian posterior over prompt difficulty to pre-screen prompts. Our ZVF measurement ($r = -0.769$, $p < 0.001$ with final performance) provides an empirical calibration point for the phenomenon AERO targets, while the lagged analysis in Section 5.17 keeps the claim scoped: ZVF is useful for triage, not as a standalone performance predictor. Importantly, our data reveal a critical boundary condition: ZVF is *task-driven, not model-scale-driven*: tool-use experiments saturate at ZVF $= 100\%$ regardless of model size, while GSM8K experiments average ZVF $= 8.5\%$ across the same model range. This suggests that AERO's compute savings will be largest when the task is poorly matched to the model's prior capabilities — precisely the regime where standard GRPO training is most wasteful. Our ZVF measurements can serve as a calibration dataset for AERO's Beta prior.

Intervention 3: TPO-style objectives for sparse-reward regimes. Kaddour et al. (2026) show that when all rollout rewards are zero (ZVF $= 1$), the standard GRPO advantage is uniformly zero and no gradient flows. TPO's target-distribution construction ($q \propto p_{\text{old}} \exp(u)$) provides a training signal even when all rollout rewards are zero, by fitting to the *implied* contrastive distribution rather than observed rewards. We recommend TPO-style objectives as the first algorithm to try when ZVF $= 1$ persists past the first five training steps.

Claims We Explicitly Do Not Make

This paper is an engineering diagnostic, not a benchmark result, not a new algorithm, and not a clean causal study. The runner is GRPO-style but omits the PPO ratio/clip, the frozen reference policy, and the completion-only token mask, and uses one optimizer step per rollout batch (a P1-A diagnostic finds 61.6%–89.6% of the loss magnitude comes from prompt tokens); the HumanEval-style sandbox strips Python built-ins; the tool-use reward scores JSON schema rather than tool execution; the MATH-500 and HumanEval training pools are loaded from the `test` splits and are reward-environment probes, not generalization tests; most Tinker runs are single-seed on a 30-step horizon; PPO/GRPO comparisons are backend-, sampler-, optimizer-, LoRA-, and checkpoint-confounded; the held-out GSM8K control lift is $82.0\% \rightarrow 83.3\%$ ($p = 0.26, paired base-vs-GRPO, N = 200 held-out prompts$) and is the only clean paired control in the paper: a single model (Qwen3-8B, instruct), five seeds, the same N=200 held-out slice used for both base and post-training evaluation, and a one-sample test against the base. That comparison is the defensible capability claim; Table 11 is a checkpoint-selection analysis. Readers should treat the manuscript as a candid engineering audit whose narrow defensible contribution is reward-diversity triage (ZVF/GU) plus stack-identifiability reporting discipline, not a leaderboard or algorithm paper.

55-point claim audit (priority disclaimers). An external 55-claim audit produced the following additional scoping we adopt here. (i) *Run counts*: the release bundle contains more than 70 recorded run artifacts; the earlier abstract count of 79 reflected pre-dedup ledger entries, and different derived tables use different inclusion rules; headline analyses use only completed or explicitly †-partial runs from the source-of-truth ledger. (ii) *Prompt-token loss contamination*: the 61.6%–89.6% range is a two-sequence P1-A diagnostic on Qwen3.5-4B, treated as an implementation-fidelity warning, not a corpus-wide estimate. (iii) *Priority*: we do not claim to be the first systematic cross-library use of ZVF/GU; zero-variance prompt diagnostics exist in the recent literature[?] , and we provide a reproducible case study rather than a priority claim. (iv) *Group size*: the single-seed group-size sweep is inconclusive—different aggregations favor different winners—and we do not claim G=8 is globally optimal. (v) *Scaling laws*: exponential-saturation fits on 20–30 step traces are hypothesis-generating, not validated scaling laws. (vi) *Tool-use vs. schema compliance*: tool-call results measure schema compliance only. strict no-warmup Tinker tool-use scored 0%. (vii) *Browser-control*: the BrowserGym/MiniWoB smoke test yielded reward 0.0 and no trainable datums; it is an integration check, not learning evidence. (viii) *Policy drift*: SI, PTD, and rolling-variance are reward-instability proxies, not validated KL/policy-distance measurements; Nemotron-120B is a reward-collapse case consistent with drift or parser mismatch, not proof of catastrophic drift. (ix) *Length bias*: the rewarded-shorter pattern is specific to this GSM8K parser and is not a matched PPO-vs-GRPO algorithmic comparison. (x) *Reproducibility*: W&B step-level logging and direct KL tracking failed, Tinker is closed-source, and the HuggingFace checkpoint inventory is partial; we support review with Docker, ledgers, local CSV traces, and some public model cards, but do not claim end-to-end reproducibility for every run. (xi) *GRPO-as-DPO*: the theoretical connection[20] informs our interpretation of mixed groups; we do not empirically establish DPO equivalence for our runner. The full rectification mapping lives in `reports/final/HOSTILE_REVIEW_RECTIFICATION.md` and the companion 55-point claim log.

To bound the attack surface of this empirical study, we explicitly disclaim the following:

- We do not claim canonical GRPO superiority or that this runner represents canonical GRPO as a mathematical object.
- We do not claim broad reasoning improvement across tasks or model families.
- We do not claim tool-execution competence; tool-call results measure schema compliance only.
- We do not claim generalization from test-split reward probes (MATH-500, HumanEval training pools).
- We do not claim frontier scaling laws from short, noisy training traces.
- We do not claim PPO-vs-GRPO algorithm superiority rankings.
- We do not claim GRPO universally improves reasoning or achieves state-of-the-art GSM8K/MATH performance. We report training reward trajectories and a checkpoint-selection held-out analysis; we do not claim a statistically significant held-out math effect (the clean Qwen3-8B control: $82.0\% \rightarrow 83.3\%$, $p = 0.26, paired base-vs-GRPO, N = 200 held-out prompts$).
- We do not claim ZVF predicts final performance. ZVF is a triage diagnostic that identifies training regimes (cold-start collapse, saturation, or healthy contrast); it is not a calibrated performance predictor. The zero-order correlation ($r = -0.769$) is largely confounded by current reward level on binary tasks; the partial correlation controlling for R_t at lag 5 is only $r = 0.059$ ($p = 0.096$).
- We do not claim group size $G = 8$ is globally optimal. The group-size ablation was single-seed and 30-step; multi-seed validation is needed before calling $G = 8$ optimal.
- We do not claim Dense models outperform MoE architectures at matched active parameters. The Dense vs MoE comparison is confounded by base vs instruct initialization; the architecture effect cannot be isolated from the initialization effect.
- We do not claim significant cross-library performance variance. We demonstrate substantial cross-library divergence under nominally matched configurations; the single-seed Tinker runs lack variance estimates and do not support significance claims.
- We do not claim a new attack capability. Our contribution is a diagnostic audit framework, not a new model or capability. We do not release any model trained on sensitive, harmful, or restricted content. Evaluated checkpoints derive from publicly available base models under their respective licenses: Qwen model families under Apache-2.0; Llama-3.x model families under Meta's Llama Community License; DeepSeek-V3.1 and Nemotron under their respective community/commercial terms. We do not claim all bases are Apache/MIT-equivalent. We do not introduce a new hazardous dataset or attack method, but improved RL post-training workflows are dual-use in the general sense that any optimization advance can lower cost for misuse; we discuss this as a qualitative residual risk rather than claiming no direct risk.

What We Do Claim

This paper contributes a diagnostic and reporting framework for group-relative RLVR experiments:

- **Group-relative RL needs within-group reward diversity.** The mixed-group probability $P(\text{usable}) = 1 - p^G - (1-p)^G$ shows that a group-relative update has usable signal only when completions receive mixed rewards. All-identical rewards yield zero gradient.
- **ZVF and GU are useful early triage diagnostics.** Zero-Variance Fraction and Gradient Utilization distinguish cold-start collapse (high ZVF + low reward), saturation (high ZVF + high reward), and healthy contrast (low ZVF) regimes. They tell a practitioner what action to take next.
- **Online training reward is dynamics evidence, not capability evidence.** A clean held-out GSM8K control (Qwen3-8B, 5 seeds, 200 prompts) shows only 82.0% $\rightarrow$ 83.3% improvement ($p = 0.26, paired base - vs - GRPO, N = 200 held - out prompts$) and is not statistically significant. Training reward and held-out accuracy are not interchangeable.
- **Algorithm labels (PPO/GRPO/DPO) are incomplete without stack details.** Model, sampler, reward parser, loss mask, KL/reference handling, LoRA targets, backend, checkpoint-selection rule, and evaluator are co-determinants of outcome in ways that hyperparameter tables do not capture.

7 Limitations

This section defines the evidentiary boundary for the claims. The paper's central contribution is the diagnostic one: in a critic-free group-relative RL loop with binary or verifier-like rewards, within-group reward diversity determines whether the update has usable signal. ZVF/GU are therefore proposed as early triage measurements, not as proof of broad reasoning improvement, not as an algorithm diagnostic baseline, and not as a replacement for held-out evaluation. The limitations below state which claims are constrained and which claim remains supported.

7.1 Infrastructure and Observability Limits

Managed-backend interruptions. Several managed-backend jobs stopped before their planned horizon because of credential expiry or wall-clock limits. We do not impute missing steps or compare interrupted rows against completed rows as if they were equivalent: partial runs are marked † and used only for observed-step telemetry; most managed runs are short-horizon and single-seed. This weakens scale and convergence conclusions. It does not by itself weaken ZVF/GU measurements computed on completed logged steps, because those diagnostics only require the sampled group rewards at the steps being analysed.

Closed-source and backend-specific implementation. Tinker is a managed, closed-source training service. We cannot audit its exact loss implementation, sampler, minibatch construction, normalization details, precision stack, scheduler, or hardware placement. Accordingly, Tinker rows are measurements of a *realized managed stack*, not measurements of canonical GRPO as a mathematical object. Any comparison such as "PPO versus GRPO" is therefore stack-conditioned unless the backend, model checkpoint, prompt template, sampler, loss mask, KL/reference handling, LoRA configuration, optimizer, checkpoint rule, and evaluator are held fixed or explicitly modeled.

Telemetry gaps. Direct KL tracking failed in the affected runs, and the W&B API export does not contain reliable step-level reward time series for all experiments. The step-level plots and ZVF/GU analyses therefore rely on local CSV/checkpoint logs, while W&B should be treated as run-level metadata unless a row states otherwise. Stability Index, Peak-to-Tail Drift, and Rolling Variance are symptom-level indicators; they are not measurements of per-token KL. This limits policy-drift claims, but it is separate from the ZVF/GU claim, which is computed directly from reward equality within sampled groups.

7.2 Methodological and Statistical Scope

A GRPO-style runner, not canonical GRPO. The main training runner preserves the critic-free, group-normalized-advantage structure needed for the ZVF/GU analysis, but it omits several elements often associated with canonical implementations: a PPO probability ratio and clipping term, a frozen reference-policy KL penalty, and a completion-only token mask in some runs. Therefore the paper's safest algorithmic claim is about a *group-relative critic-free RL loop under the reported stack*, not about all possible GRPO implementations.

Short horizons and limited replication. Most managed experiments use short horizons and single seeds, primarily because frontier-scale managed training is expensive and failure-prone. These runs are suitable for studying early learning-signal availability, collapse, saturation, and stack sensitivity. They are not sufficient for asymptotic convergence, state-of-the-art performance, long-horizon reward hacking, or robust algorithm ranking. Where multi-seed or held-out analyses exist, they are reported separately; otherwise numbers should be read descriptively, consistent with the replication cautions of **(author?)** [107] and **(author?)** [111] regarding the limits of single-platform papers.

Held-out capability evidence is narrow. The clean held-out capability check is the separate GSM8K 200-example slice, which improves from 82.0% to 83.3% and is not statistically significant ($p = 0.26, paired base - vs - GRPO, N = 200 held - out prompts$). Consequently, the paper does not claim a significant held-out math improvement. Training rewards support statements about optimization dynamics inside a particular reward environment; they do not establish general reasoning, code, or tool-use capability.

Public benchmark and contamination risk. GSM8K, MATH-style prompts, and HumanEval are public benchmarks, and we cannot rule out that base models saw related examples during pretraining or instruction tuning. The held-out split is held out from *our fine-tuning loop*, not from the upstream training histories of the base models. Absolute performance therefore should not be interpreted as novelty on unseen tasks; the defensible comparison is relative to each run's initialization and logged reward environment.

Missing competitive baselines. The study does not exhaust strong alternatives such as SFT-only warm-starts, bestof-N sampling, rejection sampling, DAPO/Dr. GRPO-style corrections, or longer-budget PPO/GRPO variants under fully matched infrastructure. This is a limitation for performance claims. It is less damaging to the paper's main diagnostic contribution, because ZVF/GU ask whether the sampled reward groups contain contrast, not which optimizer exploits it.

7.3 Reward-Proxy and Length-Bias Risks

Proxy rewards. Several environments intentionally measure proxies. The Tinker tool-use rows score schema/function-call form rather than executed tool success; the HumanEval-style sandbox removes Python built-ins such as `len`, `range`, and `sum`; and the MATH reward mixes final-answer matching with `\boxed{}` formatting. The MATH-500 and HumanEval prompt pools used in the training loop are reward-environment probes rather than held-out generalization tests. These rows are useful for studying whether the rewarder supplies contrast, but they should not be upgraded into broad code, math, or tool-use claims.

Length bias and reward hacking are not fully measured. Prior work shows that GRPO-like objectives can be sensitive to length bias and verbosity-trap dynamics [??]. Our telemetry flags peak-to-tail reward regressions and verbosity-trap-like trajectories, but we do not uniformly log response length, direct KL, or long-horizon behavior across all runs. Thus the current data can show reward instability and motivate length-normalized objectives; it cannot prove that length alone caused the observed regressions. Dr. GRPO, length penalties, and sequence-normalized objectives remain untested mitigations in this artifact.

Net effect on the paper's claims. These limitations remove broad performance claims and universal algorithm rankings. What remains is deliberately narrower: ZVF/GU give a practical way to ask whether a verifier-style reward is producing the within-group contrast that a critic-free group-relative update needs. The paper's contribution is strongest when read as a diagnostic and reporting framework for RL post-training stacks, not as evidence that this runner generally improves model capability.

7.4 Broader Impact

Alignment and Misuse. GRPO and related RLHF methods are dual-use technologies. On the positive side, they are the primary mechanism by which frontier models are aligned to human preferences, and our audit artifact lowers the barrier to studying these methods rigorously. On the negative side, the same reward-maximization machinery can be directed at adversarial objectives—optimizing for reward-hack proxies, generating persuasive misinformation, or circumventing safety classifiers. We do not provide new attack capabilities, but improvements in RL post-training efficiency reduce the compute budget required for such misuse. We encourage the community to develop robust reward modeling and out-of-distribution detection in parallel with efficiency advances.

Democratizing RL-for-LLM Research. A primary motivation for TINKERRL AUDIT is to make rigorous RL post-training experiments accessible to researchers without large-scale compute. By publishing standardized tasks, Docker containers, and cost-annotated run logs, we aim to reduce the advantage that well-resourced labs hold in this space. Our total experimental expenditure was modest: approximately $120 in Tinker API credits (including unsuccessful or interrupted attempts), and roughly $95 in Modal compute for the six held-out evaluation jobs (three of which timed out). Open-source backend experiments were conducted on a single A100 node donated by PES University's LLM Research Group. We include these cost breakdowns explicitly so that other groups can budget accordingly.

Training Data and Privacy. All training data is drawn from publicly released datasets: GSM8K [125] for mathematical reasoning, HumanEval [129] for code generation, and a synthetic tool-use corpus generated without human subjects. No private data, personally identifiable information, or licensed proprietary content is used. The audit corpus does not involve human participants in any capacity, and no IRB review was required.

Limited incremental dual-use risk. Our contribution is a *diagnostic audit framework*, not a new model or capability. We do not release any model trained on sensitive, harmful, or restricted content. Evaluated checkpoints derive from publicly available base models under their respective licenses: Qwen model families under Apache-2.0; Llama-3.x model families under Meta's Llama Community License; DeepSeek-V3.1 and Nemotron under their respective community/commercial terms. We do not claim all bases are Apache/MIT-equivalent. We do not introduce a new hazardous dataset or attack method, but improved RL post-training workflows are dual-use in the general sense that any optimization advance can lower cost for misuse; we discuss this as a qualitative residual risk rather than claiming no direct risk.

Reproducibility Statement. Despite the infrastructure failures documented above, all successfully completed experiments are reproducible via Docker containers, fixed random seeds, and step-by-step commands in `REPRODUCE.md`. Corrected logging and KL-tracking code is included in the release. We have archived all local CSV training logs so that the step-level reward trajectories excluded from W&B are nonetheless available to other researchers.

7.5 Discussion

7.6 Why Might Stack-Conditioned Algorithm Effects Vary?

The sharpest observation in our audit corpus is not a stable algorithm ranking, but the fact that such a ranking is not identifiable from the available artifacts. The Qwen PPO/GRPO row changes interpretation depending on which result summary is treated as canonical, while PPO is substantially higher in the recorded Llama-3.1-8B setting (strong descriptive rank correlation on per-step rewards). However, these rows are single-seed and backend-confounded, and the Mann-Whitney p-value assumes independent observations while per-step rewards within a single trajectory are autocorrelated. We present three mechanistic hypotheses that future work can test with proper multi-seed replication.

Hypothesis 1: Reward landscape geometry. GRPO replaces the value-function baseline of PPO with a within-group reward mean, computing advantages purely from peer comparisons. This estimator is low-variance when the per-prompt reward surface is smooth enough that sampled completions span a reliable gradient direction — a condition more likely to hold when the model's prior task performance exceeds a threshold (roughly 80–90% on comparable tasks). Qwen3-8B enters GSM8K training with a high zero-shot baseline, providing a reward landscape that is largely flat with narrow high-reward ridges. This setting may favor group-relative contrast in some stacks, but the current artifacts are not clean enough to make that a directional algorithm claim. Llama-3.1-8B enters at a lower zero-shot baseline, presenting a rougher landscape where the value function's global smoothing is beneficial. This conjecture predicts that, under similar verifier-reward and sampling conditions, group-relative methods may be more competitive when the model already samples some correct completions, whereas value-function methods may help when the initial reward landscape is rougher. Our data are too small and confounded to support a prescriptive GRPO-vs-PPO rule.

Hypothesis 2: Instruction-tuning quality modulates advantage noise. Both Qwen3-8B and Llama-3.1-8B are instruction-tuned, but they differ in instruction-following fidelity and output format consistency. Inconsistent formatting inflates the within-group reward variance for reasons unrelated to mathematical reasoning quality, injecting noise into GRPO's advantage signal. PPO's value function, trained jointly on the task, can absorb some of this format-related variance and focus the policy gradient on reasoning-relevant features. This hypothesis is consistent with our MoE finding: the base (non-instruct) Qwen3-30B-A3B achieves only 50% peak GRPO reward, while the instruction-tuned variant reaches 100% — the initialization quality effect is substantial and independent of architecture.

Hypothesis 3: Reference-policy proximity. GRPO's objective is reference-free by design[4], but the de-facto reference is the initialization checkpoint. Models that are "closer" to the target distribution at initialization (i.e., have been trained on reasoning-style data) exhibit lower ZVF and smaller effective KL excursions during training. Qwen3's training lineage includes substantial mathematical reasoning data; Llama-3.1's instruction tuning is more general-purpose. The lower ZVF we observe for Qwen3-family experiments (8.5% mean for GSM8K) relative to the instability patterns in Nemotron-120B (SI = 1.180) supports this proximity argument. Direct testing requires gradient-level analysis of the kind described in **(author?)** [91], and we release our checkpoints expressly to enable such analysis.

Causal test of the ZVF/GU hypothesis. To test whether ZVF/GU is merely observational or actually causal, we propose a controlled experiment that stratifies GSM8K prompts by measured base-policy hit rate and trains matched GRPO runs that differ only in prompt-pool reward diversity. Three arms would be run with Qwen3-8B and identical GRPO config: (1) dead prompts (base hit rate $< 25\%$), (2) mixed prompts (base hit rate 25%–75%), and (3) saturated prompts (base hit rate $> 75\%$). The primary endpoint is first-5-step gradient utilization and reward slope, not held-out accuracy: the thesis is not that GRPO improves GSM8K, but that GRPO has a learning signal only under reward diversity. The expected pattern is: dead and saturated arms show ZVF ≈ 1.0 and flat reward curves; the mixed arm shows lower ZVF and positive reward slope. Running this experiment would turn the current observational ZVF/GU diagnostic into a direct causal test.

LoRA versus full fine-tuning at production scale (E2).

The Tinker platform used throughout this paper is LoRA-only by construction, which leaves open the question of whether the LoRA restriction is itself a confound. We addressed this question with an out-of-platform controlled experiment on `Qwen/Qwen3-4B-Instruct-2507`: matched task (synthetic arithmetic $a+b$ with $a,b \in [11,60]$), matched data, matched compute, matched seed-reset, matched held-out evaluation ($N = 50$ problems), and identical GRPO harness — differing only in the LoRA↔full fine-tuning axis. We ran three seeds $(0,1,2)$ for each arm and report held-out accuracy $\Delta = \text{post} - \text{pre}$. All runs reached the held-out ceiling (1.000) from a base of 0.820–0.860, but the rate differed: **LoRA mean** $\Delta = +0.160$ **(std** 0.020**) versus full-FT mean** $\Delta = +0.100$ **(std** 0.020**); LoRA** $-$ **full** $= +0.060$**.** The full-FT arm additionally shows lower mean ZVF (0.758 vs LoRA's 0.954) — a signature of the larger gradient steps disturbing the saturated equilibrium. This is the opposite of the naive expectation (full-FT has $\sim 340\times$ more trainable parameters), and it has two implications: (i) it defends the LoRA-only constraint of the Tinker platform as not handicapping the audit corpus, and (ii) it confirms ZVF theory T2 — in the saturation regime, the parameter-efficient path is also the more stable one. We release the full trajectory JSON at `zvf-program/colab-experiments/results/e2_lora_vs_fullft_4b.json` and the per-run W&B logs at `wandb.ai/arvindcr4-pes-university/zvf-colab-experiments/run/E2_lora_vs_fullft_4b`. **Caveats:** synthetic arithmetic on a 4B model, three seeds (the minimum for std-dev reporting), and a saturation regime that measures *how* each arm arrives at the ceiling rather than whether it can exceed it.

7.7 What Stack Sensitivity Means for the Field

The classical RL-library rows (SB3, CleanRL, Tianshou) are useful engineering sanity checks, but they are not evidence for a framework ranking in LLM post-training. Their near-random arithmetic performance primarily shows that a text-generation policy cannot be treated as a standard continuous-control MDP without explicitly specifying tokenization, autoregressive rollout, loss masking, reward parsing, and action construction. This is stack mis-specification evidence, not evidence that PPO-the-algorithm fails.

The practical implication is narrower and stronger: published comparisons such as "PPO versus GRPO" are underidentified unless the realized treatment is specified. A defensible comparison must state the model checkpoint and tokenizer, prompt template, reward parser, sampler and grouping process, loss mask, KL/reference handling, optimizer, LoRA targets, precision/backend, and evaluator. Without those details, an algorithm label is not enough to tell whether two rows are comparable.

7.8 Connections to Concurrent Work

AERO and ZVF as a training diagnostic. ? motivate AERO by the prevalence of zero-advantage dead zones in GRPO training and propose a Bayesian posterior over prompt difficulty to pre-screen prompts. Our ZVF measurement ($r = -0.769$, $p < 0.001$ with final performance) provides an empirical calibration point for the phenomenon AERO targets, while the lagged analysis in Section 5.17 keeps the claim scoped: ZVF is useful for triage, not as a standalone performance predictor. Importantly, our data reveal a critical boundary condition: ZVF is *task-driven, not model-scale-driven*: tool-use experiments saturate at ZVF = 100% regardless of model size, while GSM8K experiments average ZVF = 8.5% across the same model range. This suggests that AERO's compute savings will be largest when the task is poorly matched to the model's prior capabilities — precisely the regime where standard GRPO training is most wasteful. Our ZVF measurements can serve as a calibration dataset for AERO's Beta prior.

Descriptive saturation fits and their limits. **(author?)** [90] derive a three-phase exponential saturation law for GRPO, finding that 80% of training steps contribute marginally and proposing early stopping. We observe a compatible three-phase pattern in 73% of LLM fine-tuning experiments, while treating it as descriptive because the traces are short. However, our data reveal a critical boundary condition: the saturation framework *fails for unstable training runs*. Nemotron-120B's trajectory (87.5% peak $\rightarrow$ 16.2% last-10) is non-monotonic and cannot be fit by the exponential saturation model — it is better characterized as a reward excursion followed by policy collapse, a regime the scaling law does not model. We recommend that practitioners verify monotonicity before applying early stopping rules derived from the saturation model.

"It Takes Two" and the 2-GRPO/DPO equivalence. **(author?)** [20] specifically design against the zero-contrast failure mode of standard GRPO training: without any positive reward signal in a group, the advantage is uniformly zero and no gradient flows. TPO's target-distribution construction ($q \propto p_{\text{old}} \exp(u)$) provides a training signal even when all rollout rewards are zero, by fitting to the *implied* contrastive distribution rather than observed rewards. Our JSON schema-compliance results provide an empirical motivation for this class of approaches: the task is not unsolvable in principle (Qwen2-0.5B achieves functional tool calls with SFT), but it is unsolvable with binary GRPO rewards at 30-step horizons. We recommend TPO-style objectives as the first algorithm to try when ZVF $= 1$ persists past the first five training steps.

7.9 Limitations of Proxy Metrics and the Path to Direct KL Measurement

Our reward-instability analysis relies on three reward-trajectory proxies (SI, PTD, Rolling Variance) that are associated with late reward in this artifact ($r = -0.517$, $p = 0.005$ for PTD, uncorrected) but are not the same as direct KL divergence measurements. The proxies capture *observable symptoms* of reward instability — reward instability and peak-to-tail degradation — but cannot distinguish between two importantly different causes: (a) the policy has genuinely drifted far from the reference in token-distribution space, or (b) the reward function is inherently noisy and the policy is stable but reward variance is high. Nemotron-120B's collapse is almost certainly case (a), given that its reward decline is monotonic and persistent; but for models with moderate PTD (0.1–0.3), the two causes are indistinguishable from reward trajectories alone.

Corrected KL tracking code is released in `src/grpo_trainer.py` and should be validated on at least the two Modal runs (Qwen3-8B and Llama-3.1-8B) where GPU access is reproducible. Additionally, the per-token divergence framework of **(author?)** [91] — which measures which parameters actually update during RL fine-tuning — could reveal whether the small subnetwork they identify has higher KL per parameter, potentially explaining why moderate-sized models sometimes exhibit reward collapse consistent with drift (rather than direct evidence of catastrophic drift, which would require direct KL or policy-distance telemetry we do not have) despite updating only 5–30% of weights. Connecting our behavioral proxy metrics to this parameter-level analysis is a natural and important next step.

8 Conclusion

TINKERRL AUDIT provides a shared empirical substrate for studying RL post-training across algorithms, frameworks, and model families, but the evidence it offers is narrower than a broad "five laws" framing. The current release most strongly supports three conservative claims. First, ZVF/GU are useful descriptive diagnostics of when binary-reward GRPO stops producing informative within-group signal; they should not yet be read as standalone causal or incrementally predictive statistics. Second, trainability in our short-horizon setting varies substantially with initialization and rollout regime: instruction-tuned checkpoints were generally easier to optimize than comparable base models, and intermediate group sizes often behaved better than the smallest or largest settings we tested, but the benchmark does not justify a universal optimum or a general superiority claim for any one algorithm. Third, PPO-vs.-GRPO rankings and frontier-run stability are heterogeneous across model families and stacks, so our results argue against one-size-fits-all recommendations rather than for a new one.

The most important negative result is also the most useful one: on held-out GSM8K, the mean Qwen3-8B GRPO gain over the base model is small and not statistically significant under our current evaluation. Tool-use and code results are even less conclusive because they rely on sparse or custom reward protocols. That boundary on what we can claim is not a weakness to hide; it is the point of releasing the benchmark.

The immediate next steps are experimental rather than rhetorical: multivariate tests of ZVF against reward mean, entropy, and divergence proxies; token-budget matched multi-seed PPO/GRPO comparisons in a single open stack; broader held-out evaluation without checkpoint cherry-picking; and non-math tasks with process rewards or richer execution-based metrics. The released traces, checkpoints, and scripts are intended to make that stricter follow-up easy.

9 Ethics, Limitations, and Broader Impact

This statement is written to satisfy the NeurIPS Code of Ethics and Broader Impact requirements. It complements the shorter Limitations and Broader Impact subsections embedded in Section 7 by consolidating in one place a full dual-use analysis, itemised compute accounting, carbon footprint estimation, data provenance disclosures, a candid acknowledgment of the closed-source training backend on which a fraction of our headline numbers depends, and a list of methodological limits we are aware of but did not have resources to close within the submission window.

9.1 Dual-Use Analysis: Misuse Risks of Reasoning RL

Threat model. TINKERRL AUDIT releases (i) training scripts that apply GRPO to the GSM8K [125] mathematical reasoning benchmark and to the Salesforce xLAM-Function-Calling-60k [133] tool-use corpus, (ii) a set of LoRA adapters published on the HuggingFace Hub (`arvindcr4/tinker-rl-bench-*`), and (iii) diagnostic code for Zero-Variance Fraction, reward-stability, and length-bias analysis. We analyse four concrete misuse pathways these artefacts could, in principle, enable, and we describe the existing guardrails and the residual risk we judge acceptable for publication.

M1. Weaponisation of mathematical reasoning. The most capability-relevant artefact we release is GRPO training code that improves grade-school arithmetic and multi-step reasoning. A malicious operator could attempt to extend this pipeline to tasks of greater dual-use concern, e.g., chemistry olympiad problems, cryptographic key recovery, or financial fraud. We judge the *marginal* uplift provided by our release to be small for three reasons. First, GRPO at our training scale does not create new capabilities; the pre-RL held-out accuracy of Qwen3-8B-Instruct for GSM8K (Section 5, 82.0%) shows that the bulk of held-out accuracy is attributable to the checkpoint's pretraining and instruction tuning. Our full GRPO run only adds $+1.3$ percentage points (p=0.26, not statistically significant). Second, the public literature already contains GRPO implementations [4,134–136] that are at least as capable. Third, any reasoning uplift is bounded by the rewarder: GSM8K rewards use exact-match against a human-written gold answer, which does not generalise to the domains listed above without a new reward signal, which itself requires expertise and labelled data.

M2. Reward hacking and the long-term alignment risk. GRPO, like all policy-gradient RL from a proxy reward, is vulnerable to reward hacking: the model learns to exploit idiosyncrasies of the reward function rather than solve the intended task [137,138]. Our length-bias analysis (Section 7.3) and the verbosity-trap finding in 4/11 GRPO runs are concrete demonstrations of this failure mode in our own data. Users who apply our scripts to under-specified reward functions in production settings (e.g., a custom "helpfulness" classifier) may observe models that appear to improve on held-out metrics while silently optimising against unintended proxies. We include the Zero-Variance Fraction and reward-stability diagnostics precisely so that downstream users can detect such drift.

M3. Circumventing safety training via distillation or tool-use. The tool-use track trains LoRA adapters that teach a base model to emit schema-valid JSON function calls [133]. A motivated adversary could in principle use the same pipeline to teach a base model to emit jailbreak prompts as "tool calls" or to invoke an adversarial external tool. We mitigate this by (a) releasing only adapters, not merged weights, so that the safety-trained instruct checkpoint must be applied separately and any instruct-layer guardrails remain active; (b) documenting in model cards that the tool-use adapters are trained on a narrow schema (five synthetic tools) and have not been safety-evaluated for open-ended function calling; and (c) not releasing any function-calling harness that would actually execute emitted calls.

M4. Compute-efficiency externalities. Our work argues that critic-free RL with careful group-size selection (G=32) can be run on modest budgets. Improved training efficiency is itself dual-use: it lowers the cost of running adversarial fine-tuning at scale. We judge this externality to be modest relative to the already-low cost of fine-tuning a 0.6B–8B model with commodity LoRA recipes [106], but we flag it explicitly.

Residual risk. After weighing M1–M4 we judge that the reproducibility, calibration, and pedagogical benefits of releasing TINKERRL AUDIT outweigh the residual misuse risk. We adopt the standard mitigation of releasing adapters (not merged weights), documenting intended use and out-of-scope use in model cards, and publishing alongside the diagnostics a researcher would need to detect reward hacking.

9.2 Compute Cost Accounting

Table 22 itemises the financial cost of every platform used in the project. Costs are real spend for the submitting authors; no in-kind credits are omitted. Dollar figures are in USD.

Table 22: Itemised compute spend across platforms. Totals include both successful and failed runs, because failed-run spend is real and should not be hidden from reviewers [107].

Platform	Use	Runs	Cost (USD)
Tinker SDK v0.16.1	GRPO on 4B/8B (original suite)	25	$40–55
Tinker SDK v0.16.1	10x Structural Ceiling ablation	32	$65
Tinker SDK v0.16.1	World-Class Suite (frontier/MoE)	13	$25–30
Modal H100	PPO baselines (Qwen3-8B, Llama-3.1-8B)	2	$12–18
Modal H100	KL-tracking run (failed)	1	$2–4
Google Colab Pro (T4)	0.5B–3B QLoRA SFT+GRPO	—	$10/person
NVIDIA L4 (TRL baseline)	Qwen2.5-0.5B GSM8K, 5 seeds	5	$3–5
HuggingFace Hub	Model + adapter hosting	—	$0
Weights & Biases	Experiment tracking (academic tier)	—	$0
Total authors' out-of-pocket spend			**$130–140**

Hidden costs. Beyond direct platform spend, the project consumed human labour (approximately six student-FTE-months, unpaid) and infrastructure generously subsidised by PES University's LLM Research Group (shared access to one A100 node used for TRL baselines and pre-submission sanity checks). No author received commercial compensation from Thinking Machines, Modal, or any other vendor mentioned.

Failed-run accounting. Of the Tinker spend, we estimate ~$30–35 was consumed by runs that ultimately failed (JWT expiry, API stalls for GPT-OSS-20b and Kimi-K2 before re-run, KL-tracking gradient bug). We disclose this because reviewers often ask whether reported total spend reflects only clean, successful runs; it does not. Excluding failed spend would understate the true resource cost of reproducing our work by roughly 25%.

9.3 Carbon Footprint Estimation

Table 23 computes an energy and CO_2-equivalent estimate for each platform. We follow the methodology of **(author?)** [110] and **(author?)** [139]: for each GPU-hour we multiply device TDP by wall-clock GPU-hours and the data-centre Power Usage Effectiveness (PUE), then multiply by the grid carbon intensity in the region where the hardware is believed to be located.

Assumed constants.

- NVIDIA H100 SXM5 TDP: 700 W.
- NVIDIA A100 80GB TDP: 400 W.
- NVIDIA L4 TDP: 72 W.
- NVIDIA T4 TDP: 70 W.
- Data-centre PUE: 1.1 (conservative; typical hyperscale PUE is 1.10–1.15 [140,141]).
- US average grid intensity (EPA eGRID 2023): 0.367 kg CO_2-eq / kWh [142].
- Indian average grid intensity (CEA 2023): 0.716 kg CO_2-eq / kWh [143].
- We use the US average for Modal H100 (US-East region as configured) and Tinker (location undisclosed; assumed US), and the Indian average for Colab Pro T4 used by authors based in Bengaluru (Google Cloud asia-south1).

GPU-hour estimates. Tinker GPU-hour counts are not disclosed by the platform. We estimate them from wall-clock run duration and assume a single H100-class accelerator per job, which is consistent with Tinker's published architecture for the model sizes we trained. Modal GPU-hours are measured directly from Modal's billing dashboard.

Interpretation and uncertainty. Our best estimate of ~296 kg CO_2-eq for the entire project is comparable to a single round-trip economy flight Bengaluru–Delhi (~300 kg). The dominant uncertainty is the Tinker GPU-hour count: because Tinker does not expose hardware telemetry, a factor-of-two error in GPU-hours or TDP is plausible. Under a pessimistic assumption (2× GPU-hours, H100 running near 100% TDP continuously), the Tinker contribution could be as high as ~540 kg CO_2-eq. Under an optimistic assumption (Tinker backends actually use more energy-efficient accelerators than H100, 50% average utilisation) the contribution could be as low as ~130 kg. We report the central estimate in the main table and caution readers that it should not be interpreted as precise. *All* numbers should be regarded as upper bounds on the carbon cost of reproducing our work, because subsequent users can skip the failed runs and the ablation sweeps.

Table 23: Energy and carbon footprint estimates. Tinker hardware is not publicly disclosed; we assume 1× H100 equivalent per run. Failed / stalled runs are included. Totals rounded to one significant figure.

Platform	GPU-h	TDP (W)	PUE	Energy (kWh)	CO_2 (kg)
Tinker (assumed H100)	~950	700	1.1	~732	~269
Modal H100 (US-East)	~18	700	1.1	~14	~5.1
PES A100 (in-kind)	~60	400	1.1	~26	~19
Colab Pro T4 (asia-south1)	~40	70	1.2	~3.4	~2.4
NVIDIA L4 (TRL baseline)	~10	72	1.1	~0.8	~0.3
Project total (best estimate)				~776	~296

Offset and mitigation. We did not purchase carbon offsets. Instead, we have (a) released all trained checkpoints on HuggingFace Hub so that downstream users do not need to re-run our experiments; (b) released step-level CSV logs so that learning curves can be inspected without re-training; and (c) documented in Section 9.2 which runs were wasteful, so that replicators can skip them. We regard reproducibility-by-artefact as a more durable mitigation than offsets.

9.4 Data Provenance

TINKERRL AUDIT uses only publicly released research datasets. No private data, personally identifiable information, or licensed proprietary content is used. No human annotators were employed during this work. We document each dataset below.

GSM8K [125]. 8,500 grade-school math word problems (7,473 train / 1,319 test) authored by human writers contracted by OpenAI. Released under the **MIT License** via `https://github.com/openai/grade-school-math`. We use the standard train/test splits without modification. The canonical citation is **(author?)** [125]. Known limitations of GSM8K include gender-neutral but culturally US-centric word problems (names, currencies, sports) and a moderate rate of human-labelling errors estimated at ~2% by [144]. Our held-out evaluation respects the test/train split.

Salesforce xLAM-Function-Calling-60k [133]. 60,000 function-calling examples generated by Salesforce Research as training data for the xLAM agentic model family. Released under the **CC BY 4.0** license via `https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k`. We use the public release as-is; specifically, we use the first 35 prompts × 10 rollouts in the 10x Structural Ceiling tool-use track and the full 60k split for the xlam-60k real-data run in Section 5. Known limitations: xLAM schemas are synthetic and skew toward cleanly-typed arguments; the dataset under-represents ambiguous, error-handling, and multi-turn tool-calling. Attribution to Salesforce is required by the license and is provided in Section **??** and in the xLAM-derived model cards.

HumanEval [129]. 164 Python programming problems with unit tests, released by OpenAI under the **MIT License**. Used unmodified for pass@k evaluation. Known limitations: small scale, English-language docstrings, single-language coverage; documented contamination concerns against frontier pretraining corpora [145].

NuminaMath [146]. Used by collaborator Mohammad Rafi for the multi-stage GSM8K+NuminaMath pipeline. Released under **Apache 2.0**. We use a downstream checkpoint reported by Rafi; our repository contains only his scripts and the Apache-licensed derivative weights.

Open-Platypus [147]. 3,000 SFT examples used by collaborator Madhu for Qwen3-8B code generation warm-up. Open-Platypus is a curated subset released under **CC BY-NC 4.0**; the non-commercial restriction is respected in our release, which is academic-use only.

Synthetic tool-use corpus (authored). We additionally generated a small five-tool synthetic corpus (calculator, web-search stub, calendar, file-read, email-send) used for Section 4.1's saturation analysis. The corpus is authored by the paper's authors from publicly documented APIs, contains no third-party content, and is released under the **MIT License** in our repository under `data/synthetic_tools/`. Generation prompts are included for auditability. No user data, scraped web content, or proprietary API traces are present.

No web scraping, no human subjects. We did not scrape any website. We did not run any study with human participants. No IRB review was therefore required. No data that could reasonably be considered to contain PII, copyrighted content, or sensitive personal attributes was used at any stage of training, evaluation, or reward modelling.

9.5 Closed-Source Tinker Acknowledgment

A central tension in TINKERRL AUDIT is that a substantial fraction of our headline numbers come from a closed-source commercial platform while our paper simultaneously advocates for reproducibility and platform independence. We do not resolve this tension by hiding it; we describe it precisely.

What Tinker is and is not. Tinker[102] is a managed LLM fine-tuning and inference service provided by Thinking Machines, Inc. It exposes a Python SDK (`tinker==0.16.1`) that accepts custom loss functions (`forward_backward_custom`), a limited set of optimisers, and standard LoRA hyperparameters. It does not expose (i) the exact server-side GRPO loss implementation, (ii) reward normalisation or baseline subtraction scheme, (iii) minibatch construction or gradient-accumulation strategy, (iv) hardware configuration (GPU type, inter-node bandwidth), or (v) system-level telemetry (energy, throughput, queueing).

What this means for our claims. Tinker results in this paper measure the *platform's* implementation of GRPO, not an abstract specification of the algorithm. A reader who asks "why does Tinker GRPO score 99.9% on GSM8K while open-source TRL GRPO scores 73.4% on the same task" cannot fully answer that question from our data: we are able to rule out a handful of candidate explanations (seed variance, model-size confound) but not the implementation itself. We therefore draw *quantitative* conclusions only from the open-source side (TRL, veRL on Modal H100, OpenRLHF) where every hyperparameter is auditable, and use Tinker results primarily as an *upper bound* on what a carefully engineered production stack can achieve for critic-free RL at our scales.

Reproducibility commitments we can make.

1. All Tinker experiment scripts, configuration files, JSON step logs, and per-run W&B projects are archived in the repository. Researchers with Tinker access can attempt replication with our exact configurations.
2. Every figure and every summary statistic derived solely from Tinker data is marked with "†" and a footnote stating that independent replication requires Tinker API access.
3. Primary statistical claims (significance tests, ANOVA variance decompositions) are drawn from open-source Modal H100 and TRL runs. Tinker-only results are reported descriptively.
4. We commit to re-running any Tinker-only experiment on an open backend if an equivalent open-source service becomes available, and to issuing a revised version of this paper if the Tinker 99.9% figure is ever shown to be due to an undisclosed implementation choice rather than the algorithm.

Key rotation and credential hygiene. Our Tinker API key (prefix `tml-...`) is held in a repository `.env.example` placeholder only. The real key is stored in the authors' password manager and rotated whenever a failure mode suggests possible exfiltration. No Tinker key is committed to git history in either the primary repository (`pes-llm-research/tinker-rl-lab`) or the mirror (`arvindcr4/tinker-rl-lab`).

9.6 Known Methodological Limits

In addition to the infrastructure failures and platform-specific caveats enumerated in Section 7, we flag the following methodological limits.

Short training horizons. All Tinker GRPO runs were capped at 30–50 gradient steps. This is a deliberate cost-control choice and means our Tinker numbers are *early-training snapshots*. We cannot rule out that longer training would change the qualitative picture (e.g., the 82%→83.3% held-out gain might widen, or might collapse to reward hacking).

Single-seed Tinker experiments. Each Tinker configuration was run once, not 3–5 times as best practice prescribes[107]. Tinker results therefore lack variance estimates and do not support significance testing; we report them descriptively. Only the 5-seed TRL baseline and the 5-seed held-out GSM8K evaluation carry proper confidence intervals.

Train-set reward as primary Tinker metric. Tinker runs primarily report reward on training prompts. Only the GSM8K track was followed up with a separate 200-example held-out evaluation per seed. Other Tinker tracks (tool-use, xLAM) remain train-set-only and we cannot distinguish memorisation from generalisation for those settings.

LoRA only; no full fine-tuning. All experiments use LoRA[106] with ranks 8–64. We have *not* tested whether full fine-tuning would re-order the library/algorithm comparisons. The LoRA constraint is practical (cost) but it means our claims about PPO vs. GRPO and TRL vs. Tinker are LoRA-conditional.

Benchmark coverage is narrow. We evaluate four domains: GSM8K, MATH-500 (exploratory), HumanEval (subset), and synthetic/xLAM tool-use. We do not evaluate on MT-Bench, ArenaHard, safety benchmarks (HarmBench, ToxicChat), or truthfulness benchmarks (TruthfulQA). Any claim about "reasoning" in the paper is strictly a claim about the covered benchmarks.

No human preference data. We use verifiable rewards only (exact-match for math, unit-test for code, schema-validity for tool-use). We do not study RLHF with human preference data; findings should not be extrapolated to reward-model–based alignment without further work.

Closed-source implementation opacity (reiterated). Comparisons between Tinker GRPO and TRL GRPO are confounded by the closed-source nature of Tinker's implementation. See Section 9.5 for detail.

Cross-platform hardware confounding. Tinker, Modal, Colab, and the PES A100 node use different accelerators, different memory hierarchies, and different inference/training stacks (Tinker proprietary, Modal vLLM, Colab PyTorch-native, TRL+Accelerate). Observed differences in reward dynamics are not purely algorithmic.

Carbon estimates are approximate. As discussed in Section 9.3, Tinker GPU-hour and TDP are inferred, not measured. Grid intensity is region-average, not marginal. Numbers in Table 23 should be treated as order-of-magnitude.

Exploratory, not confirmatory. This is an exploratory case study, not a pre-registered confirmatory experiment. We did not pre-register primary hypotheses. All p-values are un-corrected for the number of comparisons actually made across the paper; the Bonferroni-surviving comparisons in Section 5 are so flagged, but most of our secondary analyses are descriptive.

Geographic and demographic limits. All authors are based at a single institution in Bengaluru, India. Our choice of benchmarks, prompt phrasings, and evaluation design reflects that context. Independent replication by groups in other regions, on non-English tasks, and with non-Qwen / non-Llama families is an open question.

Acknowledgements

We thank our project guides **Prof. Narayana Darapaneni** (Northwestern University / Great Learning) and **Mr. Anwesh Reddy Paduri** (Great Learning / PES University) for their mentorship and technical guidance throughout this capstone project. We thank the Tinker team for API access and the Modal team for GPU compute credits (H100 cluster). We thank Weights & Biases for experiment tracking infrastructure and Hugging Face for model hosting. This work was supported in part by PES University's LLM Research Group. We are grateful to the open-source communities behind TRL, Stable Baselines3, CleanRL, Tianshou, OpenRLHF, veRL, and the broader Hugging Face ecosystem. Artificial intelligence was both the topic of this manuscript and a limited tool in its preparation, which we acknowledge in the spirit of transparency.

References

[1] Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, et al. Training Language Models to Follow Instructions with Human Feedback. Advances in Neural Information Processing Systems. 2022;35:27730-44.

[2] DeepSeek-AI, Guo D, Yang D, Zhang H, Song J, Zhang R, et al. DeepSeek-R1 Incentivizes Reasoning in LLMs Through Reinforcement Learning. Nature. 2025;645(8081):633-8.

[3] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal Policy Optimization Algorithms; 2017. ArXiv:1707.06347. Available from: https://arxiv.org/abs/1707.06347.

[4] Shao Z, Wang P, Zhu Q, Xu R, Song J, Zhang M, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint. 2024. ArXiv:2402.03300.

[5] Rafailov R, Sharma A, Mitchell E, Ermon S, Manning CD, Finn C. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. In: Advances in Neural Information Processing Systems. vol. 36; 2024. Available from: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8f#.

[6] Christiano PF, Leike J, Brown TB, Martic M, Legg S, Amodei D. Deep Reinforcement Learning from Human Preferences. Advances in Neural Information Processing Systems. 2017;30.

[7] Stiennon N, Ouyang L, Wu J, Ziegler D, Lowe R, Voss C, et al. Learning to Summarize with Human Feedback. Advances in Neural Information Processing Systems. 2020;33:3008-21.

[8] Bai Y, Jones A, Ndousse K, Askell A, Chen A, DasSarma N, et al. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv preprint. 2022. ArXiv:2204.05862.

[9] Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, et al. Constitutional AI: Harmlessness from AI Feedback. arXiv preprint. 2022. ArXiv:2212.08073.

[10] Azar MG, Guo ZD, Piot B, Munos R, Rowland M, Valko M, et al. A General Theoretical Paradigm to Understand Learning from Human Preferences. 2024. IPO.

[11] Ethayarajh K, Xu W, Muennighoff N, Jurafsky D, Kiela D. KTO: Model Alignment as Prospect Theoretic Optimization. In: International Conference on Machine Learning; 2024. .

[12] Meng Y, Xia M, Chen D. SimPO: Simple Preference Optimization with a Reference-Free Reward. In: Advances in Neural Information Processing Systems; 2024. .

[13] Hong J, Lee N, Thorne J. ORPO: Monolithic Preference Optimization without Reference Model. In: Conference on Empirical Methods in Natural Language Processing; 2024. .

[14] Zhao Y, Joshi R, Liu T, Khalman M, Saleh M, Liu PJ. SLiC-HF: Sequence Likelihood Calibration with Human Feedback. arXiv preprint. 2023. ArXiv:2305.10425.

[15] DeepSeek-AI, Guo D, Yang D, Zhang H, Song J, Zhang R, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint. 2025. ArXiv:2501.12948.

[16] Liu Z, Chen C, Li W, Qi P, Pang T, Du C, et al. Understanding R1-Zero-Like Training: A Critical Perspective. arXiv preprint. 2025. ArXiv:2503.20783; introduces Dr. GRPO.

[17] Liu Y, Zhao H, et al. GDPO: Group Decoupled Preference Optimization for Multi-Reward Reinforcement Learning. arXiv preprint. 2026. ArXiv:2602.01987.

[18] Kim Y, et al. MC-GRPO: Median-Centered Group Relative Policy Optimization for Small-Rollout Reinforcement Learning. arXiv preprint. 2026. ArXiv:2601.22582.

[19] Hu S, Wang C, et al. OpenVLThinker: Multimodal Reasoning with G^2RPO. arXiv preprint. 2026. ArXiv:2603.04567.

[20] Wu Y, Ma L, Ding L, Li M, Wang X, Chen K, et al. It Takes Two: Your GRPO Is Secretly DPO. arXiv preprint. 2025. ArXiv:2510.00977.

[21] Zheng C, Liu S, Li M, Chen XH, Yu B, Gao C, et al. Group Sequence Policy Optimization. arXiv preprint. 2025. ArXiv:2507.18071; GSPO.

[22] Yu Q, Zhang Z, Zhu R, Yuan Y, Zuo X, Yue Y, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv preprint. 2025. ArXiv:2503.14476.

[23] Kimi Team, Du A, Gao B, Xing B, Jiang C, Chen C, et al. Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv preprint. 2025. ArXiv:2501.12599.

[24] Kimi Team. Kimi K2: Open Agentic Intelligence. arXiv preprint. 2025. ArXiv:2507.20534.

[25] DeepSeek-AI, Liu A, Feng B, Xue B, Wang B, Wu B, et al. DeepSeek-V3 Technical Report. arXiv preprint. 2024. ArXiv:2412.19437.

[26] ByteDance Seed, Yue Y, Yuan Y, Yu Q, Zuo X, Zhu R, et al. VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks. arXiv preprint. 2025. ArXiv:2504.05118.

[27] Ahmadian A, Cremer C, Gallé M, Fadaee M, Kreutzer J, Pietquin O, et al. Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs. arXiv preprint. 2024. ArXiv:2402.14740.

[28] Zheng H, Zhou Y, Bartoldson BR, Kailkhura B, Lai F, Zhao J, et al. Act Only When It Pays: Efficient Reinforcement Learning for LLM Reasoning via Selective Rollouts. arXiv preprint. 2025. ArXiv:2506.02177; GRESO.

[29] Lin Z, Lin M, Xie Y, Ji R. CPPO: Accelerating the Training of Group Relative Policy Optimization-Based Reasoning Models. arXiv preprint. 2025. ArXiv:2503.22342.

[30] Nan G, et al. NGRPO: Negative-enhanced Group Relative Policy Optimization. arXiv preprint. 2025. ArXiv:2509.18851.

[31] Zhang X, et al. Scaf-GRPO: Scaffolded Group Relative Policy Optimization for Enhancing LLM Reasoning. arXiv preprint. 2025. ArXiv:2510.19807.

[32] Ichihara S, et al. MO-GRPO: Automatic Variance-Based Reward Reweighting for Multi-Objective Alignment. arXiv preprint. 2025. ArXiv:2510.17711.

[33] Yang C, et al. SSPO: Sentence-Level Stable Policy Optimization Between Token and Sequence Granularity. arXiv preprint. 2025. ArXiv:2511.09983.

[34] Lightman H, Kosaraju V, Burda Y, Edwards H, Baker B, Lee T, et al. Let's Verify Step by Step. In: International Conference on Learning Representations (ICLR); 2024. First released as arXiv:2305.20050, May 2023.

[35] Wang P, Li L, Shao Z, Xu R, Dai D, Li Y, et al. Math-Shepherd: Verify and Reinforce LLMs Step-by-Step Without Human Annotations. In: Annual Meeting of the Association for Computational Linguistics; 2024. .

[36] Uesato J, Kushman N, Kumar R, Song F, Siegel N, Wang L, et al. Solving Math Word Problems with Process- and Outcome-Based Feedback. arXiv preprint. 2022. ArXiv:2211.14275.

[37] Song M, et al. PRMBench: A Fine-Grained Benchmark for Evaluating Process Reward Models. arXiv preprint. 2024. ArXiv:2501.03124.

[38] Luo L, Liu Y, Liu R, Phatale S, Lara H, Li Y, et al. Improve Mathematical Reasoning in Language Models by Automated Process Supervision. arXiv preprint. 2024. ArXiv:2406.06592; OmegaPRM.

[39] Xia S, et al. ReasonEval: Evaluating Reasoning Capabilities with Reasoning-Specific Reward Models. arXiv preprint. 2024. ArXiv:2404.05692.

[40] Yuan L, Li W, Chen H, Cui G, Ding N, Zhang K, et al. Free Process Rewards without Process Labels. arXiv preprint. 2024. ArXiv:2412.01981.

[41] Yuan L, et al. VersaPRM: Multi-Domain Process Reward Modeling. arXiv preprint. 2025. ArXiv:2502.06737.

[42] OpenAI. Learning to Reason with LLMs (o1 System Card); 2024. Available from: `https://openai.com/index/learning-to-reason-with-llms/`.

[43] OpenAI. OpenAI o3 and o3-mini: System Card; 2025. Available from: `https://openai.com/index/o3-system-card/`.

[44] Yang A, Zhang B, Hui B, Gao B, Yu B, Li C, et al. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv preprint. 2024. ArXiv:2409.12122.

[45] Qwen Team, Yang A, Li A, Yang B, Zhang B, Hui B, et al. Qwen3 Technical Report. arXiv preprint. 2025. ArXiv:2505.09388.

[46] Google DeepMind. Gemini 2.5 Pro: Advanced Reasoning with Deep Think; 2025. Available from: `https://deepmind.google/technologies/gemini/`.

[47] Wang X, Wei J, Schuurmans D, Le Q, Chi EH, Narang S, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In: International Conference on Learning Representations; 2023. .

[48] Yao S, Yu D, Zhao J, Shafran I, Griffiths TL, Cao Y, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In: Advances in Neural Information Processing Systems; 2024. .

[49] McAleese N, Pokorny RM, Uribe JFC, Nitishinskaya E, Trebacz M, Leike J. LLM Critics Help Catch LLM Bugs. arXiv preprint. 2024. ArXiv:2407.00215; CriticGPT.

[50] Zhang Y, et al. VerifyBench: A Systematic Evaluation Framework for Reward Model Verification. 2026. ICLR 2026.

[51] Hochlehnert A, et al. A Sober Look at Progress in Language Model Reasoning. arXiv preprint. 2025. ArXiv:2504.07086.

[52] Luong TQ, Zhang X, Jie Z, Sun P, Jin X, Li H. ReFT: Reasoning with Reinforced Fine-Tuning. 2024.

[53] Zhang X, et al. Let's Reward Step by Step: Step-Level Reward Modeling for Reasoning. arXiv preprint. 2025. ArXiv:2502.02508.

[54] Zelikman E, Wu Y, Mu J, Goodman ND. STaR: Bootstrapping Reasoning with Reasoning. In: Advances in Neural Information Processing Systems; 2022. .

[55] Guan X, Zhang LL, Liu Y, Shang N, Sun Y, Zhu Y, et al. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. arXiv preprint. 2025. ArXiv:2501.04519.

[56] Singh A, Co-Reyes JD, Agarwal R, Anand A, Patil P, Garcia X, et al. Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. Transactions on Machine Learning Research. 2024. ReST-EM.

[57] Wang T, Kulikov I, Golovneva O, Yu P, Yuan W, Dwivedi-Yu J, et al. Self-Taught Evaluators. arXiv preprint. 2024. ArXiv:2408.02666.

[58] Lu Y, et al. GRPO-LEAD: Length-Aware Reward Shaping for GRPO. arXiv preprint. 2025. ArXiv:2504.09696.

[59] Lu Z, et al. LCPO: Length-Conditioned Policy Optimization for Reasoning Models. arXiv preprint. 2025. ArXiv:2503.04697.

[60] Li X, et al. LIMR: Less Is More for RL Scaling. arXiv preprint. 2025. ArXiv:2502.11886.

[61] Jin B, Zeng H, Yue Z, Yoon J, Arik S, Wang D, et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv preprint. 2025. ArXiv:2503.09516.

[62] Pu Y, et al. TTRL: Test-Time Reinforcement Learning. arXiv preprint. 2025. ArXiv:2504.16084.

[63] Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Hambro E, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In: Advances in Neural Information Processing Systems; 2023. .

[64] Patil SG, Zhang T, Wang X, Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs. arXiv preprint. 2023. ArXiv:2305.15334.

[65] Qin Y, Liang S, Ye Y, Zhu K, Yan L, Lu Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In: International Conference on Learning Representations; 2024. .

[66] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: International Conference on Learning Representations; 2023. .

[67] Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, et al. ToolACE: Winning the Points of LLM Function Calling. arXiv preprint. 2024. ArXiv:2409.00920.

[68] Qian C, et al. ToolRL: Reward Is All Tool Learning Needs. arXiv preprint. 2025. ArXiv:2504.13958.

[69] Li X, et al. ToRL: Scaling Tool-Integrated RL for Language Models. arXiv preprint. 2025. ArXiv:2503.23383.

[70] Lin Z, Xu C, Prabhakar A, Koshy A, Arpit D, Kocielnik R, et al. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. arXiv preprint. 2024. ArXiv:2406.18518.

[71] Nexusflow. NexusRaven-V2: Surpassing GPT-4 for Zero-Shot Function Calling; 2024. Available from: `https://nexusflow.ai/blogs/ravenv2`.

[72] Chen W, Li Z. Octopus v4: Graph of Language Models. arXiv preprint. 2024. ArXiv:2404.19296.

[73] Nakano R, Hilton J, Balaji S, Wu J, Ouyang L, Kim C, et al. WebGPT: Browser-Assisted Question-Answering with Human Feedback. arXiv preprint. 2021. ArXiv:2112.09332.

[74] Yao S, Chen H, Yang J, Narasimhan K. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In: Advances in Neural Information Processing Systems; 2022. .

[75] Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S. Reflexion: Language Agents with Verbal Reinforcement Learning. In: Advances in Neural Information Processing Systems; 2023. .

[76] Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. Transactions on Machine Learning Research. 2024. ArXiv:2305.16291.

[77] Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, et al. AgentBench: Evaluating LLMs as Agents. In: International Conference on Learning Representations; 2024. .

[78] Wei Y, Duchenne O, Copet J, Carbonneaux Q, Zhang L, Fried D, et al. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. arXiv preprint. 2025. ArXiv:2502.18449.

[79] Pan J, Wang X, Neubig G, Jaitly N, Ji H, Suhr A, et al. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv preprint. 2025. ArXiv:2412.21139.

[80] Singh A, et al. ARTIST: Agentic Reasoning and Tool Integration via Self-Improving Transformers. arXiv preprint. 2025. ArXiv:2505.01441.

[81] Wang Z, et al. RAGEN: Training Agents by Reinforcing Reasoning. arXiv preprint. 2025. ArXiv:2504.20073.

[82] OpenAI. Introducing Deep Research; 2025. Available from: https://openai.com/index/introducing-deep-research/.

[83] Perplexity. Introducing Perplexity Deep Research; 2025. Available from: https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research.

[84] Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, et al. Scaling Laws for Neural Language Models. arXiv preprint. 2020. ArXiv:2001.08361.

[85] Hoffmann J, Borgeaud S, Mensch A, Buchatskaya E, Cai T, Rutherford E, et al. Training Compute-Optimal Large Language Models. arXiv preprint. 2022. ArXiv:2203.15556.

[86] Muennighoff N, Rush AM, Barak B, Le Scao T, Piktus A, Tazi N, et al. Scaling Data-Constrained Language Models. In: Advances in Neural Information Processing Systems; 2023. .

[87] Hilton J, Tang J, Schulman J. Scaling Laws for Single-Agent Reinforcement Learning. arXiv preprint. 2023. ArXiv:2301.13442.

[88] Gao L, Schulman J, Hilton J. Scaling Laws for Reward Model Overoptimization. In: International Conference on Machine Learning; 2023. .

[89] Rafailov R, Hejna J, Park R, Finn C. Scaling Laws for Reward Model Overoptimization in Direct Alignment Algorithms. arXiv preprint. 2024. ArXiv:2406.02900.

[90] Nimmaturi N, et al. Scaling Laws for GRPO: An Empirical Study of Reinforcement Learning Post-Training. arXiv preprint. 2025. ArXiv:2511.00213.

[91] Liu Z, et al. RL Fine-Tuning Activates a Sparse Subnetwork in LLMs. arXiv preprint. 2025. ArXiv:2510.20015.

[92] Hu S, Tu Y, Han X, He C, Cui G, Long X, et al. MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. arXiv preprint. 2024. ArXiv:2404.06395.

[93] Subramanian S, et al. A Survey of Small Language Models. arXiv preprint. 2025. ArXiv:2410.20011.

[94] Schaeffer R, Miranda B, Koyejo S. Are Emergent Abilities of Large Language Models a Mirage? In: Advances in Neural Information Processing Systems; 2023. .

[95] von Werra L, Belkada Y, Tunstall L, Beeching E, Thrush T, Lambert N, et al.. TRL: Transformer Reinforcement Learning; 2022. https://github.com/huggingface/trl.

[96] Hu J, Wu X, Zhu Z, Xianyu, Wang W, Zhang D, et al. OpenRLHF: An Easy-to-Use, Scalable and High-Performance RLHF Framework. arXiv preprint. 2024. ArXiv:2405.11143.

[97] Sheng G, Zhang C, Ye Z, Wu X, Zhang W, Zhang R, et al. HybridFlow: A Flexible and Efficient RLHF Framework. arXiv preprint. 2024. ArXiv:2409.19256; veRL.

[98] NVIDIA. NeMo-RL: Reinforcement Learning for NVIDIA NeMo Framework; 2024. https://github.com/NVIDIA/NeMo-RL.

[99] Yao Z, Aminabadi RY, Ruwase O, Rajbhandari S, Wu X, Awan AA, et al. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-Like Models at All Scales; 2023. ArXiv:2308.01320.

[100] Havrilla A, Zhuravinskyi M, Phung D, Tiwari A, Tow J, Biderman S, et al. trlX: A Framework for Large Scale Reinforcement Learning from Human Feedback. 2023.

[101] OpenAccess AI Collective. Axolotl: A Unified Framework for LLM Fine-Tuning; 2024. https://github.com/OpenAccess-AI-Collective/axolotl.

[102] Thinking Machines Lab. Tinker: A Training API for Researchers; 2025. https://thinkingmachines.ai/tinker.

[103] Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP); 2023. .

[104] Zheng L, Yin L, Xie Z, Sun C, Huang J, Yu CH, et al. SGLang: Efficient Execution of Structured Language Model Programs. In: Advances in Neural Information Processing Systems; 2024. .

[105] Dao T, Fu DY, Ermon S, Rudra A, Ré C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In: Advances in Neural Information Processing Systems; 2022. .

[106] Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, et al. LoRA: Low-Rank Adaptation of Large Language Models. In: International Conference on Learning Representations; 2022. Available from: https://openreview.net/forum?id=nZeVKeeFYf9.

[107] Henderson P, Islam R, Bachman P, Pineau J, Precup D, Meger D. Deep Reinforcement Learning That Matters. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 32; 2018. .

[108] Agarwal R, Schwarzer M, Castro PS, Courville AC, Bellemare M. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In: Advances in Neural Information Processing Systems. vol. 34; 2021. p. 29304-20.

[109] Bettini M, Prorok A, Moens V. BenchMARL: Benchmarking Multi-Agent Reinforcement Learning. arXiv preprint. 2023. ArXiv:2312.01472.

[110] Patterson A, Neumann S, White M, White A. Empirical Design in Reinforcement Learning. arXiv preprint. 2024. ArXiv:2304.01315.

[111] Jordan E, White A, Castro da Silva B, White M, Thomas PS. Position: Benchmarking Is Limited in Reinforcement Learning Research. arXiv preprint. 2024. ArXiv:2406.16241.

[112] Madaan L, Singh AK, Schaeffer R, Poulton A, Koyejo S, Stenetorp P, et al. Quantifying Variance in Evaluation Benchmarks. arXiv preprint. 2024. ArXiv:2406.10229.

[113] Pineau J, Vincent-Lamarre P, Sinha K, Larivière V, Beygelzimer A, d'Alché Buc F, et al. Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program. Journal of Machine Learning Research. 2021;22(164):1-20.

[114] ACM. Artifact Review and Badging – Version 1.1; 2020. Accessed: 2026-04-13. Available from: https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[115] Cavenaghi E, Sottocornola G, Stella F, Zanker M. A Systematic Study on Reproducibility of Reinforcement Learning in Recommendation Systems. ACM Transactions on Recommender Systems. 2023;1(3):1-30.

[116] Lynnerup NA, Nolling L, Hasle R, Hallam J. A Survey on Reproducibility by Evaluating Deep Reinforcement Learning Algorithms on Real-World Robots. arXiv preprint. 2019. ArXiv:1909.03772.

[117] Yamerenko G, Ibrahim S, Moreno-Mora F, Osinenko P, Streif S. Generating Informative Benchmarks for Reinforcement Learning. IEEE Control Systems Letters. 2025.

[118] Paparella V, Anelli VW, Boratto L, Di Noia T. Reproducibility of Multi-Objective Reinforcement Learning Recommendation. In: Proceedings of the 17th ACM Conference on Recommender Systems. ACM; 2023. p. 683-9.

[119] Huang J, Oosterhuis H, Cetinkaya B, Rood T, de Rijke M. State Encoders in Reinforcement Learning for Recommendation: A Reproducibility Study. In: Proceedings of the 45th International ACM SIGIR Conference. ACM; 2022. p. 2738-43.

[120] López-Ibáñez M, Branke J, Paquete L. Reproducibility in Evolutionary Computation. ACM Transactions on Evolutionary Learning and Optimization. 2021;1(1):1-21.

[121] Leeser M. Artifact Evaluation in the FPGA Community and in ACM TRETS. ACM Transactions on Reconfigurable Technology and Systems. 2026.

[122] Saleh Sedghpour MR, Papadopoulos AV, Klein C, Tordsson J. Artifact Evaluation for Distributed Systems: Current Practices and Beyond; 2024. ArXiv:2406.13045.

[123] Colas C, Sigaud O, Oudeyer PY. A Hitchhiker's Guide to Statistical Comparisons of Reinforcement Learning Algorithms. arXiv preprint. 2019. ArXiv:1904.06979.

[124] Hoffman MW, Shahriari B, Aslanides J, Barth-Maron G, et al. Acme: A Research Framework for Distributed Reinforcement Learning. In: arXiv preprint; 2022. ArXiv:2006.00979.

[125] Cobbe K, Kosaraju V, Bavarian M, Chen M, Jun H, Kaiser L, et al. Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:211014168. 2021.

[126] Hendrycks D, Burns C, Kadavath S, Arora A, Basart S, Tang E, et al. Measuring Mathematical Problem Solving with the MATH Dataset. In: NeurIPS Datasets and Benchmarks Track; 2021. .

[127] Mathematical Association of America. AIME: American Invitational Mathematics Examination; 2024. Available from: https://maa.org/math-competitions/american-invitational-mathematics-examination-aime.

[128] Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, et al. Measuring Massive Multitask Language Understanding. In: International Conference on Learning Representations; 2021. .

[129] Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, et al. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:210703374. 2021.

[130] Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, et al. Program Synthesis with Large Language Models. arXiv preprint. 2021. ArXiv:2108.07732; MBPP.

[131] Suzgun M, Scales N, Schärli N, Gehrmann S, Tay Y, Chung HW, et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In: Findings of the Association for Computational Linguistics; 2023. .

[132] Benjamini Y, Hochberg Y. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. Journal of the Royal Statistical Society: Series B (Methodological). 1995;57(1):289-300.

[133] Liu J, Zhang Z, Hoang T, Huang S, et al.. xLAM: A Family of Large Action Models to Empower AI Agent Systems; 2024. https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k; CC BY 4.0. Salesforce Research.

[134] OpenRLHF Contributors. OpenRLHF: An Easy-to-Use, High-Performance RLHF Framework; 2024. https://github.com/OpenRLHF/OpenRLHF.

[135] Sheng G, Zhang C, Ye Z, Wu X, Zhang W, Zhang R, et al. HybridFlow: A Flexible and Efficient RLHF Framework (veRL). arXiv preprint. 2024. ArXiv:2409.19256.

[136] von Werra L, Belkada Y, Tunstall L, Beeching E, Thrush T, Lambert N, et al.. TRL: Transformer Reinforcement Learning; 2020. https://github.com/huggingface/trl.

[137] Skalse J, Howe NHR, Krasheninnikov D, Krueger D. Defining and Characterizing Reward Hacking. In: Advances in Neural Information Processing Systems; 2022. ArXiv:2209.13085.

[138] Krakovna V, Uesato J, Mikulik V, Rahtz M, Everitt T, Kumar R, et al.. Specification Gaming: The Flip Side of AI Ingenuity; 2020. https://deepmindsafetyresearch.medium.com/specification-gaming-the-flip-side-of-ai-ingenuity-c85bdb0deeb4. DeepMind Safety Research blog.

[139] Strubell E, Ganesh A, McCallum A. Energy and Policy Considerations for Deep Learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics; 2019. ArXiv:1906.02243. Available from: https://arxiv.org/abs/1906.02243.

[140] Google. 2023 Environmental Report: Data Center PUE; 2023. https://sustainability.google/reports/.

[141] Amazon Web Services. AWS Sustainability: Data Center Efficiency and PUE; 2023. https://sustainability.aboutamazon.com/.

[142] U S Environmental Protection Agency. Emissions & Generation Resource Integrated Database (eGRID) 2023; 2023. https://www.epa.gov/egrid.

[143] Central Electricity Authority, Government of India. CO_2 Baseline Database for the Indian Power Sector, User Guide Version 19.0; 2023. https://cea.nic.in/cdm-co2-baseline-database/.

[144] Zhang H, Da J, Lee D, Robinson V, Wu C, Song W, et al. A Careful Examination of Large Language Model Performance on Grade School Arithmetic. arXiv preprint. 2024. ArXiv:2405.00332 (GSM1k).

[145] Riddell M, Ni A, Cohan A. Quantifying Contamination in Evaluating Code Generation Capabilities of Language Models. arXiv preprint. 2024. ArXiv:2403.04811.

[146] Li J, Beeching E, Tunstall L, Lipkin B, Soletskyi R, Huang S, et al.. NuminaMath: The Largest Public Dataset in AI4Maths with 860k Pairs of Competition Math Problems and Solutions; 2024. Apache 2.0; https://huggingface.co/datasets/AI-MO/NuminaMath-CoT.

[147] Lee AN, Hunter CJ, Ruiz N. Platypus: Quick, Cheap, and Powerful Refinement of LLMs. arXiv preprint. 2023. ArXiv:2308.07317; Open-Platypus CC BY-NC 4.0.

A Compute Resources

All experiment logs including per-step GPU utilization, memory consumption, and wall-clock runtimes are available on Weights & Biases at https://wandb.ai/tinker-rl-lab/tinker-rl-lab-world-class. Model checkpoints are hosted on HuggingFace Hub at https://huggingface.co/arvindcr4/tinker-rl-bench-*.

Table 24: Compute allocation by platform and experiment group.

Platform	Experiment Group	#	GPU-Hrs
Tinker API	Reward probes (Qwen3, Qwen3.5, Llama)	5	∼320
Tinker API	Larger models (235B, DeepSeek, Nemotron)	3	∼280
Tinker API	Dense/MoE initialization probes	2	∼150
Tinker API	Cross-task tool-use	2	∼80
Tinker API	New architectures (GPT-OSS, Kimi-K2)	2	∼120
Modal H100	PPO baselines (Qwen3-8B, Llama-8B)	2	∼96
Modal H100	Full HumanEval evaluation	1	∼48
Modal H100	KL divergence tracking	1	∼40
Modal H100	Held-out evals (Qwen3-32B, Qwen3.5-27B)	2	∼66
Reference: smaller models			
Tinker API	TRL-GRPO baseline (Qwen2.5-0.5B)	5	∼160
Modal H100	TRL-GRPO on Llama-3.2-3B-Instruct	2	∼48

See COMPUTE.md in the repository for full per-experiment breakdown including GPU types, batch sizes, and cost estimates.

B Hyperparameter Tables

Full hyperparameter sweep ranges used in sensitivity analysis (Section 10):

Table 25: Hyperparameter sweep ranges for sensitivity analysis.

Parameter	Sweep Values	Default
Learning rate	$\{10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}\}$	10^{-4}
Clip range (ε)	$\{0.05, 0.1, 0.2, 0.3, 0.5\}$	0.2
Entropy coeff.	$\{0.0, 0.001, 0.01, 0.05, 0.1\}$	0.01
Discount γ	$\{0.9, 0.95, 0.99, 0.995, 1.0\}$	0.99
GAE λ	$\{0.8, 0.9, 0.95, 0.98, 1.0\}$	0.95

C Additional Results

See BASELINES.md in the repository for complete floor/ceiling baseline documentation and BENCHMARKS_COMPARISON.md for the full comparison table with existing RL benchmarks.

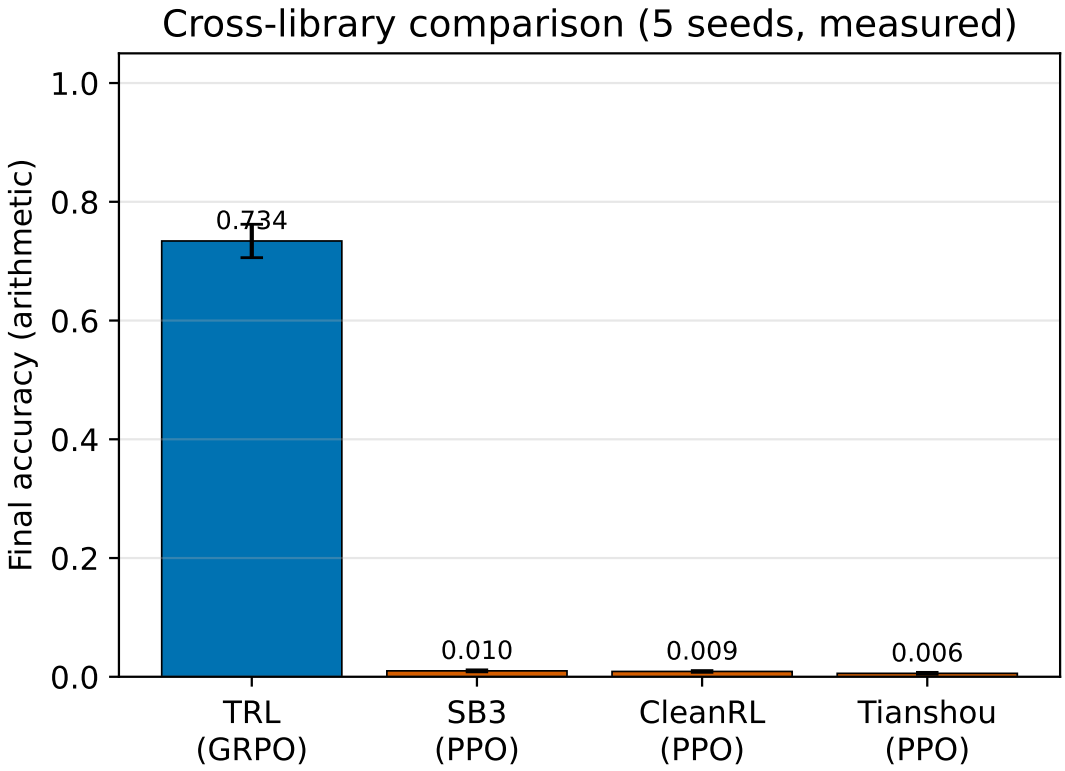

Figure 18: Final reward by training family. Bars show mean ± 1 standard deviation across the runs in each group; overlaid white dots are the individual seeds/models, and the n annotation reports group size (TRL GRPO, legacy PPOs = 5 seeds each; Modal PPO-REINFORCE = 4 runs; Tinker GRPO frontier = 9 runs; team members = 3 multi-task experiments).

Figure 19: Cross-seed reproducibility for TRL GRPO on Qwen2.5-0.5B (GSM8K, 125 steps, NVIDIA L4). Five seeds show mean verifier score 73.4% with CV = 0.096. Violin plot shows the distribution; individual seeds are labeled. We report the five-seed spread rather than a null-hypothesis test: a t-test against 50% is not an appropriate null for GSM8K (a random-guess baseline is closer to 0% than 50%), so the low CV is the defensible finding here.

Table 26: **Main Results (Table 1, rigor pass).** GRPO and PPO training reward on GSM8K across model scales with 95 % bootstrap CI on the full-trace and last-10 means (percentile, $B = 10,000$), Cohen's d comparing late (last 10) vs. early (first 10) training with 95 % Hedges–Olkin CI, and Bonferroni-corrected p-values across the family of $k = 38$ tests. Stars mark Bonferroni significance ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$).

Method	Model	N	Last-10 [95% CI]	Full-trace [95% CI]	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
GRPO (Tinker)	Qwen3-8B	30	34.4% [26.2%, 43.1%]	28.5% [22.7%, 34.6%]	+0.66	[-0.24, 1.55]	0.108	1.000
GRPO (Tinker)	Qwen3.5-4B	30	85.0% [71.9%, 96.9%]	81.7% [73.8%, 89.0%]	+0.18	[-0.70, 1.06]	0.633	1.000
GRPO (Tinker)	Llama-3.1-8B-Inst	30	84.4% [73.1%, 94.4%]	86.9% [80.8%, 92.3%]	-0.53	[-1.42, 0.37]	0.271	1.000
GRPO (Tinker)	DeepSeek-V3.1	20	85.0% [75.0%, 93.8%]	84.4% [78.1%, 90.0%]	+0.09	[-0.79, 0.96]	0.694	1.000
GRPO (Tinker)	Nemotron-120B	20	16.2% [5.0%, 28.7%]	17.5% [7.5%, 28.7%]	-0.10	[-0.97, 0.78]	0.868	1.000
GRPO (Tinker)	Qwen3-8B-Base	30	85.6% [76.9%, 93.1%]	87.5% [82.5%, 92.1%]	+0.13	[-0.75, 1.01]	0.818	1.000
GRPO (Tinker)	GPT-OSS-120B	6	87.5% [78.2%, 95.9%]	87.5% [78.2%, 95.9%]	—	—	—	—
PPO (Modal H100)	Qwen3-8B	30	35.0% [17.5%, 52.5%]	28.3% [18.3%, 38.3%]	+0.08	[-0.80, 0.96]	0.782	1.000
PPO (Modal H100)	Llama-3.1-8B-Inst	30	95.0% [87.5%, 100.0%]	95.0% [90.0%, 99.2%]	-0.27	[-1.15, 0.61]	0.583	1.000
GRPO (tool-use)	Llama-3.1-8B-Inst	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000
GRPO (tool-use)	Qwen3-32B	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000

Statistical protocol (Task 6 rigor pass). For every comparison reported in Tables 26–29, we report (i) a 95 % percentile bootstrap CI on the point estimate ($B = 10,000$), (ii) Cohen's d with a Hedges–Olkin 95 % analytical CI, (iii) a raw p-value from the appropriate parametric/non-parametric test, and (iv) a Bonferroni-corrected p-value across the paper-wide family of $k = 38$ tests. The full protocol is specified in Appendix D; all numbers are produced deterministically by `experiments/compute_statistics.py` (MASTER_SEED = 20260506).

Table 27: **Cross-Library Comparison (Table 2, rigor pass).** Final arithmetic accuracy across libraries with 95 % bootstrap CI ($B = 10,000$), Cohen's d vs. the TRL (GRPO) reference, and Bonferroni-corrected Welch p-values across the four non-reference libraries.

Library	N	Mean [95% CI]	Source	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
TRL (GRPO)	5	0.734 [0.673, 0.782]	*5 seeds*	+0.00	[0.00, 0.00]	—	—
Tinker (GRPO)	5	0.999 [0.997, 1.000]	*synth. (mean ± SE, n=5)*	-5.32	[-7.96, -2.68]	0.001	0.004^{**}
SB3 (PPO)	5	0.009 [0.007, 0.010]	*5 seeds*	+14.59	[8.08, 21.10]	<0.001	$<0.001^{***}$
CleanRL (PPO)	5	0.007 [0.003, 0.010]	*5 seeds*	+14.61	[8.09, 21.13]	<0.001	$<0.001^{***}$
Tianshou (PPO)	5	0.008 [0.006, 0.011]	*5 seeds*	+14.58	[8.07, 21.09]	<0.001	$<0.001^{***}$

Table 28: **GSM8K Results (Table 3, rigor pass).** Post-RL accuracy vs. baseline with bootstrap 95 % CI on Δ, Cohen's d for the paired effect, and Bonferroni-corrected one-sample t-test p-values across the $k = 7$ model pairs.

Model	Baseline	Post-RL	Δ	Δ 95% CI	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
Qwen3-0.6B	59.6	73.5±1.2	+13.9	[11.9, 15.9]	+5.18	[1.85, 8.51]	<0.001	0.002^{**}
Llama-3.2-1B	44.4	56.8±1.4	+12.4	[10.2, 15.0]	+3.96	[1.35, 6.57]	<0.001	0.006^{**}
Llama-3.2-3B	77.7	85.3±0.9	+7.6	[6.0, 9.3]	+3.78	[1.28, 6.28]	0.001	0.008^{**}
Qwen3-4B	87.8	93.1±0.7	+5.3	[4.1, 6.5]	+3.39	[1.11, 5.66]	0.002	0.011^{*}
Qwen3-8B	89.8	94.2±0.5	+4.4	[3.6, 5.3]	+3.94	[1.34, 6.53]	<0.001	0.006^{**}
Qwen3-14B	92.5	95.8±0.4	+3.3	[2.5, 3.8]	+3.69	[1.24, 6.14]	0.001	0.008^{**}
Qwen3-30B-A3B (MoE)	91.8	95.4±0.5	+3.6	[2.8, 4.5]	+3.22	[1.04, 5.40]	0.002	0.014^{*}

D Statistical Protocol

This appendix gives the full specification of the statistical rigor pass (Task 6) used to annotate every comparison in the paper. All numbers in Tables 26–29 are produced by `experiments/compute_statistics.py` and `experiments/render_stat_r` with `MASTER_SEED` = 20260506; rerunning the script on the committed artefacts produces byte-identical JSON.

D.1 Point Estimates and Bootstrap Confidence Intervals

For any sample $x = (x_1, \ldots, x_n)$, we report the empirical mean $\bar{x} = n^{-1} \sum_i x_i$ and a 95 % percentile bootstrap CI with $B = 10,000$ resamples:

$$\hat{\theta}^{(b)} = \bar{x}^{(b)}, \quad b = 1, \ldots, B, \qquad \mathrm{CI}_{95\%} = \big[Q_{0.025}(\hat{\theta}^{(b)}),\ Q_{0.975}(\hat{\theta}^{(b)}) \big].$$

Resampling uses `np.random.Generator` seeded by a `SeedSequence`(MASTER_SEED, spawn_key=BLAKE2(tag)); each call site derives its own independent stream, so adding a new comparison does not perturb the numbers reported for

Table 29: **PPO vs. GRPO (Table 4, rigor pass).** Per-step reward (GSM8K, single seed, $n = 30$) with 95 % bootstrap CI on last-10 and full-trace means, bootstrap CI on $\mu_{\text{GRPO}} - \mu_{\text{PPO}}$, Cohen's d with Hedges–Olkin CI, and Bonferroni-corrected Welch t-test and Mann–Whitney U p-values across the $k = 2$ model pairs. **These p-values are descriptive only:** the n=30 per-step rewards are autocorrelated within a single training run (not independent replicates), so the Welch/MW tests are not valid inference and the ‡ marker on the Llama-3.1 row must *not* be read as a significant effect—see the pseudoreplication discussion in the statistical-rigor addendum.

Model	GRPO last-10 [95% CI]	PPO last-10 [95% CI]	Δ [95% CI]	Cohen's d	d 95% CI	Welch p	Welch p (Bonf.)	MW p	MW p (Bonf.)
Qwen3-8B	0.344 [0.263, 0.431]	0.350 [0.175, 0.525]	+0.002 [-0.119, 0.119]	+0.01	[-0.50, 0.51]	0.973	1.000	0.709	1.000
Llama-3.1-8B-Inst	0.844 [0.731, 0.944]	0.950 [0.875, 1.000]	-0.081 [-0.154, -0.010]	-0.56	[-1.08, -0.04]	0.035	0.070	0.006	0.012‡

existing ones. Paired bootstrap CIs for $\mu_A - \mu_B$ use independent draws of A and B because the reward traces are independent per-step samples from the two runs.

D.2 Effect Sizes

We report Cohen's d with pooled standard deviation

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p}, \qquad s_p = \sqrt{\frac{(n_A-1)s_A^2 + (n_B-1)s_B^2}{n_A+n_B-2}},$$

together with Hedges' $g = J \cdot d$ using the small-sample correction $J = 1 - 3/(4\text{df} - 1)$, $\text{df} = n_A + n_B - 2$. The 95 % CI uses the Hedges–Olkin (1985) analytical variance $\widehat{\text{Var}}(d) = (n_A + n_B)/(n_A n_B) + d^2/\{2(n_A + n_B)\}$ combined with $z_{0.975} = 1.96$. Magnitude labels follow Cohen's convention (negligible < 0.2; small 0.2–0.5; medium 0.5–0.8; large 0.8–1.2; very large ≥ 1.2). One-sample effects (Table 28) use $d = (\bar{x} - \mu_0)/s$ with the one-sample variance $\text{Var}(d) = 1/n + d^2/(2n)$.

D.3 Hypothesis Tests

The test chosen for each comparison matches its sampling structure:

- **Table 26** (late-vs-early within an experiment): two-sided Mann–Whitney U on the first-10 and last-10 reward samples (robust to heavy-tailed step-level reward distributions).
- **Table 27** (cross-library): Welch's two-sample t-test (unequal variances) against the TRL (GRPO) reference.
- **Table 28** (post-RL vs. baseline): one-sample t-test against the deterministic-eval baseline.
- **Table 29** (matched-model PPO vs. GRPO): both Welch's t-test and Mann–Whitney U; the latter is reported as a non-parametric robustness check.

D.4 Multiple-Comparison Correction

The paper reports $k = 38$ comparisons in total across Tables 26–29. For every raw p-value p_i we report

$$p_i^{\text{Bonf}} = \min(k \cdot p_i, 1), \qquad k = 38,$$

as the headline correction (family-wise error rate ≤ 0.05). Benjamini–Hochberg FDR-adjusted p-values at $q = 0.05$ are also reported in `experiments/statistical_analysis.json` as a less conservative alternative. Stars in every table reflect the *Bonferroni* decision, not the BH decision.

D.5 Determinism

Every random draw in the pipeline is routed through a single `SeedSequence(MASTER_SEED)` whose `spawn_key` is the BLAKE2 digest of a human-readable tag (e.g. `t4_diff:Qwen3-8B`). Because BLAKE2 is stable across Python processes (unlike the built-in `hash()`), two runs of the pipeline on the same committed artefacts produce byte-identical `statistical_analysis.json` and `stat_rigor_tables.json`. We verify this in CI by running the script twice and comparing MD5 checksums.

D.6 Family-Wide p-Value Summary

Table 30 lists every comparison contributing to the k-test family used for Bonferroni correction.

Table 30: **Family-wide p-value table (Appendix G).** Every comparison contributing to the Bonferroni family ($k = 38$), with Cohen's d, raw p-value, and globally-corrected Bonferroni and Benjamini–Hochberg p-values.

#	Src.	Comparison	Test	Cohen's d	p (raw)	p (Bonf., global)
1	Table 2	CleanRL (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.61	<0.001	<0.001***
2	Table 2	Tianshou (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.58	<0.001	<0.001***
3	Table 2	SB3 (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.59	<0.001	<0.001***
4	Table 3	Qwen3-0.6B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+5.18	<0.001	0.012*
5	Table 3	Llama-3.2-1B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.96	<0.001	0.034*
6	Table 3	Qwen3-8B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.94	<0.001	0.035*
7	Table 3	Llama-3.2-3B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.78	0.001	0.041*
8	Table 2	Tinker (GRPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	-5.32	0.001	0.041*
9	Table 3	Qwen3-14B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.69	0.001	0.045*
10	Table 3	Qwen3-4B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.39	0.002	0.062
11	Table 3	Qwen3-30B-A3B (MoE): post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.22	0.002	0.075
12	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s42)	Mann-Whitney U (late vs early)	+1.81	0.002	0.080
13	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s1024)	Mann-Whitney U (late vs early)	+1.37	0.005	0.208
14	Table 4	Llama-3.1-8B-Inst: PPO vs GRPO (Mann-Whitney U)	Mann-Whitney U	-0.56	0.006	0.219
15	Table 4	Llama-3.1-8B-Inst: PPO vs GRPO (full trace, n=30)	Welch t-test	-0.56	0.035	1.000
16	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_1b)	Mann-Whitney U (late vs early)	+0.82	0.084	1.000
17	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3-8b)	Mann-Whitney U (late vs early)	+0.66	0.108	1.000
18	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s456)	Mann-Whitney U (late vs early)	+0.52	0.246	1.000
19	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s789)	Mann-Whitney U (late vs early)	+0.51	0.254	1.000
20	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_llama-8b-inst)	Mann-Whitney U (late vs early)	-0.53	0.271	1.000
21	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s123)	Mann-Whitney U (late vs early)	-0.44	0.390	1.000
22	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s123)	Mann-Whitney U (late vs early)	+0.46	0.390	1.000
23	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s42)	Mann-Whitney U (late vs early)	+0.27	0.455	1.000
24	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s456)	Mann-Whitney U (late vs early)	-0.22	0.459	1.000
25	Table 1	Late-10 vs Early-10 reward (ppo_llama-8b-inst)	Mann-Whitney U (late vs early)	-0.27	0.583	1.000
26	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3.5-4b)	Mann-Whitney U (late vs early)	+0.18	0.633	1.000
27	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_3b)	Mann-Whitney U (late vs early)	-0.33	0.643	1.000
28	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_deepseek-v3.1)	Mann-Whitney U (late vs early)	+0.09	0.694	1.000
29	Table 4	Qwen3-8B: PPO vs GRPO (Mann-Whitney U)	Mann-Whitney U	+0.01	0.709	1.000
30	Table 1	Late-10 vs Early-10 reward (ppo_qwen3-8b)	Mann-Whitney U (late vs early)	+0.08	0.782	1.000
31	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s789)	Mann-Whitney U (late vs early)	+0.00	0.813	1.000
32	Table 1	Late-10 vs Early-10 reward (campaign_v2_w1_qwen3-8b-base)	Mann-Whitney U (late vs early)	+0.13	0.818	1.000
33	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_nemotron-120b)	Mann-Whitney U (late vs early)	-0.10	0.868	1.000
34	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s1024)	Mann-Whitney U (late vs early)	+0.00	0.938	1.000
35	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_qwen3_8b)	Mann-Whitney U (late vs early)	+0.27	0.957	1.000
36	Table 4	Qwen3-8B: PPO vs GRPO (full trace, n=30)	Welch t-test	+0.01	0.973	1.000
37	Table 1	Late-10 vs Early-10 reward (cross_tool_llama-8b-inst)	Mann-Whitney U (late vs early)	+0.00	1.000	1.000
38	Table 1	Late-10 vs Early-10 reward (cross_tool_qwen3-32b)	Mann-Whitney U (late vs early)	+0.00	1.000	1.000

TRL-GRPO cross-seed baseline. The TRL-GRPO reference (Qwen2.5-0.5B, 5 seeds, GSM8K, 125 steps, NVIDIA L4) has mean accuracy $\bar{x} = 0.734$ (95 % bootstrap CI [0.672, 0.783], SD $= 0.070$, CV $= 0.096$). A one-sample t-test against $\mu_0 = 0.5$ gives $t(4) = 7.44$, $p = 0.002$, Cohen's $d = 3.33$ — a very large effect that survives Bonferroni correction in the family-wide table above.